



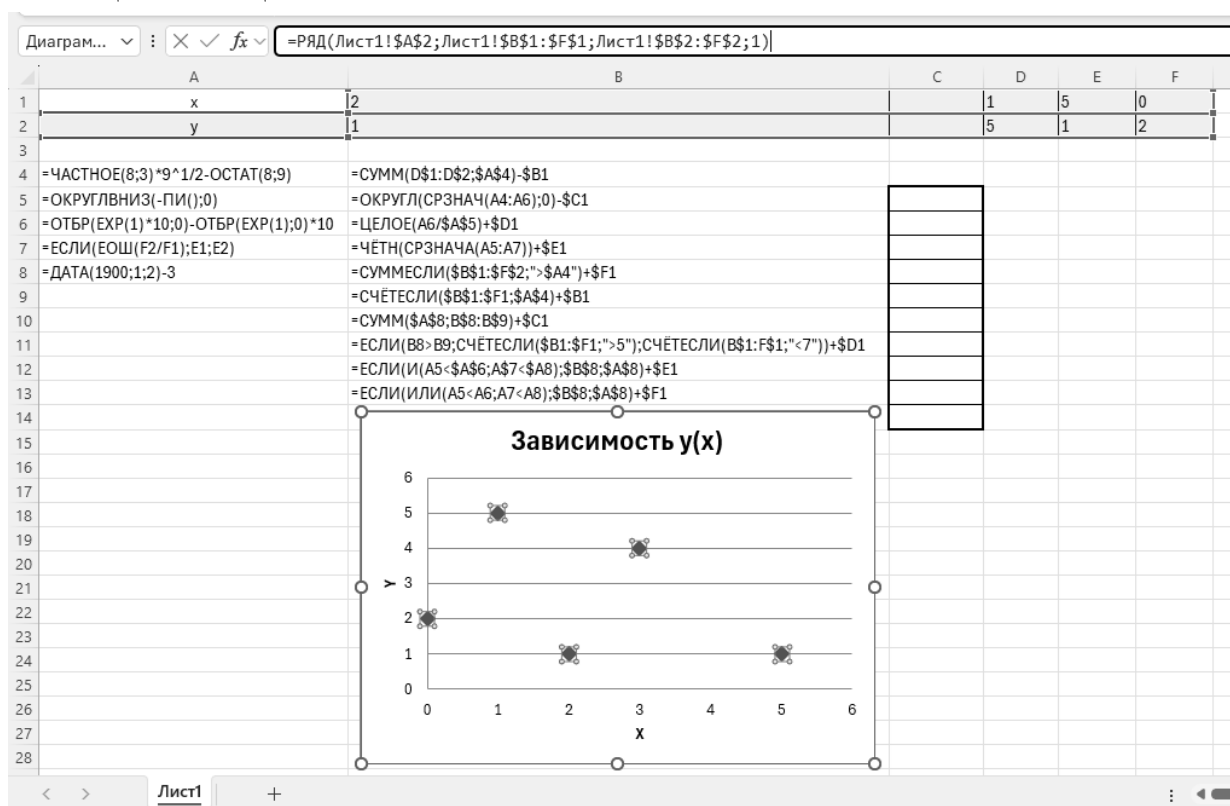
ВАРИАНТ 1

Задача 1(5 баллов)

Разведчики

От разведчиков были получены сведения, закодированные в виде электронной таблицы. Определите числа в ячейках C5:C14, которые будут получены в результате копирования в них ячеек B4:B13. Также определите недостающие числа в блоке B1:F2 на основании диаграммы.

В ответе записать, в одну строчку через запятую, значения ячеек C5:C14, начиная с ячейки C5 и заканчивая ячейкой C14. Формат ячеек в таблице – «Общий».



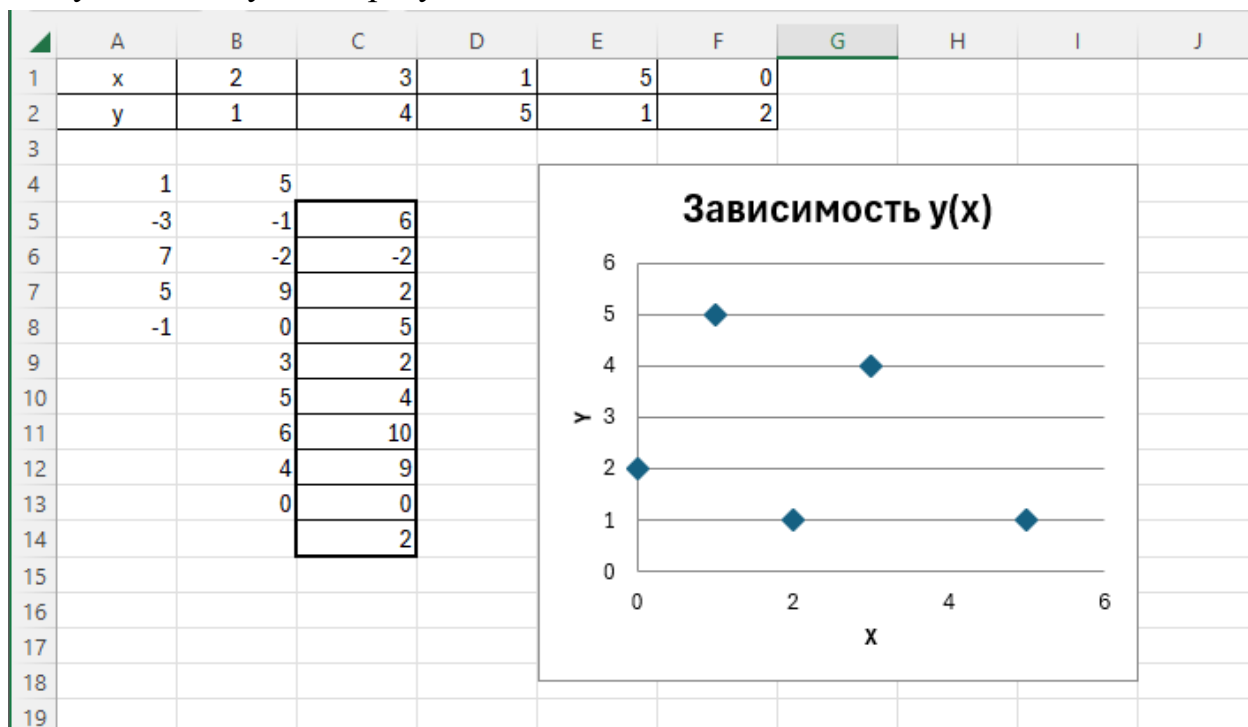
Решение

На основании диаграммы определяем, что в ячейках A1:F2 не отображена точка ($x=3$, $y=4$) расположенная в ячейках C1:C2. Далее копируем формулы, расположенные в ячейках B4:B13 в ячейки C5:C14. В результате получаем следующие формулы:



	C	D
5	=СУММ(E\$1:E\$2;\$A\$4)-\$B2	
6	=ОКРУГЛ(СРЗНАЧ(B5:B7);0)-\$C2	
7	=ЦЕЛОЕ(B7/\$A\$5)+\$D2	
8	=ЧЁТН(СРЗНАЧА(B6:B8))+E2	
9	=СУММЕСЛИ(\$B\$1:\$F\$2;">\$A4")+F2	
10	=СЧЁТЕСЛИ(\$B\$1:\$F2;\$A\$4)+\$B2	
11	=СУММ(\$A\$8;C\$8:C\$9)+\$C2	
12	=ЕСЛИ(C9>C10;СЧЁТЕСЛИ(\$B2:\$F2;">5");СЧЁТЕСЛИ(C\$1:G\$1;"<7"))+\$D2	
13	=ЕСЛИ(И(B6<\$A\$6;B\$7<\$A9);B\$8;\$A\$8)+E2	
14	=ЕСЛИ(ИЛИ(B6<B7;B8<B9);B\$8;\$A\$8)+F2	
15		

Вычисляем значения по формулам в ячейках A4:A8, затем переходим к вычислениям значений в ячейках B4:B13 и, наконец, в ячейках C5:C14. Получаем следующие результаты:



Ответ: 6, -2, 2, 5, 2, 4, 10, 9, 0, 2

**Задача 2 (10 баллов)****Лаборатория кибербезопасности**

Алиса работает в отделе кибербезопасности и тестирует новый алгоритм фильтрации данных. Логическая функция F описывает условия допуска к защищённым файлам:

$$F = x \wedge z \vee w \equiv (y \rightarrow w \vee z \wedge y)$$

Алиса составила таблицу истинности, но для проверки знаний своих коллег зашифровала некоторые значения логическими выражениями, в которых a, b, c – следующие высказывания:

a – «1 Мбайт = 1024 Кбайт»

b – «21 делится на 3 с остатком»

c – « $4 \times 5 = 20$ »

Ниже приведен фрагмент таблицы истинности функции F , содержащий неповторяющиеся строки. Определите, какому столбцу таблицы истинности функции F соответствует каждая из переменных w, x, y, z .

?	?	?	?	F
	$a \wedge b \wedge \neg c$		$\neg a \wedge b \vee \neg c$	0
	$a \rightarrow \neg b \vee c$	$a \wedge c \rightarrow b$	$a \wedge b \wedge c$	0
$a \rightarrow b \rightarrow c$	$a \wedge b \vee c$			0
$a \oplus b \wedge \neg c$	$a \vee c \rightarrow b$			0

В ответе напишите буквы w, x, y, z в том порядке, в котором идут соответствующие им столбцы.

Решение

Определим значения логических высказываний. Таким образом, получим следующее:

$$a = 1$$

$$b = 0$$

$$c = 1$$

Теперь определим значения логических выражений в таблице. В результате получим следующую таблицу:

?	?	?	?	F
	0		0	0
	1	0	0	0
1	1			0
1	0			0

Получаем следующую таблицу истинности:

w	x	y	z	$x \wedge z$	$x \wedge z \vee w$	$y \rightarrow w$	$z \wedge y$	$y \rightarrow w \vee z \wedge y$	F
0	0	0	0	0	0	1	0	1	0



0	0	0	1	0	0	1	0	1	0
0	0	1	0	0	0	0	0	0	1
0	0	1	1	0	0	0	1	1	0
0	1	0	0	0	0	1	0	1	0
0	1	0	1	1	1	1	0	1	1
0	1	1	0	0	0	0	0	0	1
0	1	1	1	1	1	0	1	1	1
1	0	0	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1	1
1	0	1	0	0	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1
1	1	0	0	0	1	1	0	1	1
1	1	0	1	1	1	1	0	1	1
1	1	1	0	0	1	1	0	1	1
1	1	1	1	1	1	1	1	1	1

Выберем из нее строки, где функция $F = 0$.

w	x	y	z	F
0	0	0	0	0
0	0	0	1	0
0	0	1	1	0
0	1	0	0	0

Сравнивая эту таблицу с заданной в задании, можем сделать вывод, что исходная и полученная таблицы не могут быть соотнесены и указать правильный порядок переменных невозможно.

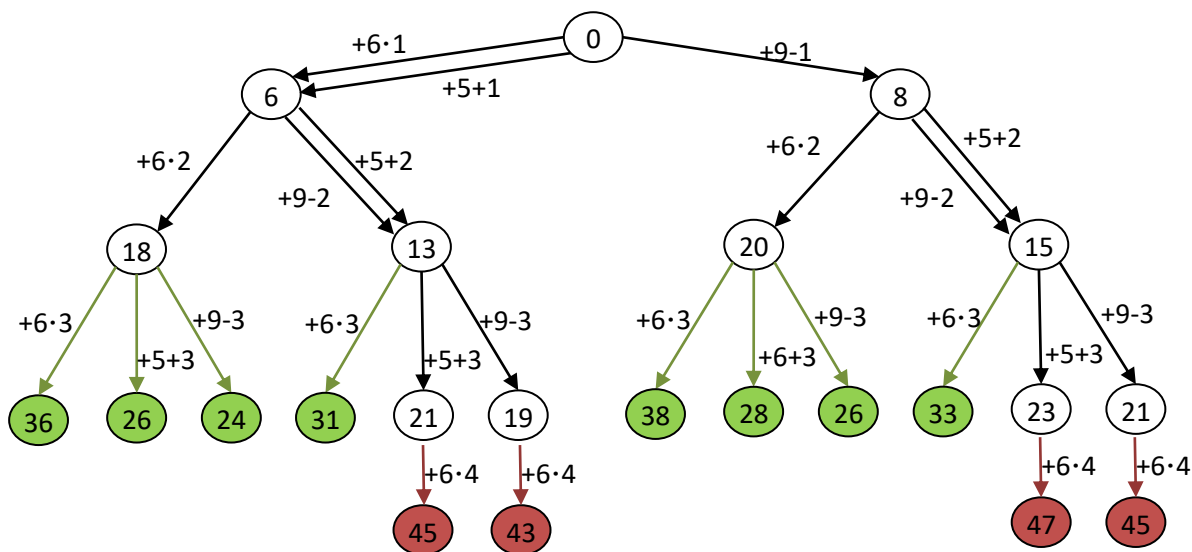
**Задача 3 (15 баллов)****Гномы и драгоценные камни: стратегии и выигрыши**

Два богатых гнома решили сыграть в следующую игру. По очереди они складывают драгоценные камни в кучку. За один ход можно либо положить в кучку количество камней, определяемое формулой: либо 6 умноженное на номер хода в игре, либо 5 плюс номер хода в игре, либо 9 минус номер хода в игре. При достижении или превышении количества 24 камня в кучке, игра заканчивается. Кто первым, сделав ход, получит в кучке 24 камня или больше – считается победителем.

Кто выигрывает при безошибочной игре – гном, делающий первый ход, или гном, делающий второй ход? Каково минимальное число ходов до победы, если выигрывающий гном пытается закончить игру за меньшее число ходов, а проигрывающий пытается максимально задержать свое поражение? Какое максимальное число камней может оказаться в кучке в конце игры? Какой гном при этом победит? Поясните алгоритм графически, используя ориентированные графы в виде деревьев, с обозначением выигрышных и проигрышных позиций как вершин, а действий игроков – дугами графа с указанием численных изменений.

Решение

Построим ориентированный граф, где зеленым цветом выделены выигрышные позиции первого гнома, красным цветом – выигрышные позиции второго гнома, а номер соответствует количеству камней в кучке. Рядом со стрелкой указано изменение числа камней в кучке на этом ходу. В случае, если один из вариантов хода приводит к победе, то второй вариант не отображается, за исключением самой правой ветви, которая демонстрирует игру с максимально возможным числом камней в конце игры.



Из графа видно, что при любой игре второго гнома, гарантированно



победит первый гном на 3 ходу. Максимально возможное число камней можно получить, если увеличивать количество камней в кучке таким образом, чтобы получить как можно более высокий номер хода (4) и количество камней в кучке (23) перед последним ходом, в котором берется вариант с большим значением $(+6 \cdot 4)$. При этом побеждает второй гном с количеством камней 47.

Ответ: гарантированно победит первый гном за 3 хода, максимум 47 камней при победе второго гнома.



Задача 4 (20 баллов)

Аномальный криокамень

В ходе арктической экспедиции обнаружена горная порода с аномальной кристаллической решеткой, демонстрирующая свойства высокотемпературной сверхпроводимости. Старший геолог Пётр, опасаясь промышленного шпионажа, передал коллеге Марии, используя многоуровневую систему защиты, зашифрованные данные, содержащие предварительное название породы из двух слов.

Мария получила два файла:

- 1) rock_sample.dat – файл с основным зашифрованным отчётом;
- 2) thin_section.png – микрофотография тонкого среза породы, в которую Петром был внедрён секретный ключ методом стеганографии, имитирующим природные включения. Микрофотография представляет собой 8-битное RGB-изображение.

Алгоритм действий для Марии выглядит следующим образом:

1) Извлечь скрытые данные из изображения thin_section.png. Для этого необходимо проанализировать третьи значащие биты цветовых каналов пикселей, записать их последовательно в том порядке, в котором они были закодированы, и получить битовую последовательность.

2) Эту битовую последовательность необходимо декодировать. В результате она должна превратиться в строку в формате JSON, которая и содержит секретные ключи для алгоритма RSA. В данной задаче строка в формате JSON выглядит следующим образом: {ключ1: значение1, ключ2: значение2, ключ3: значение3}.

3) Используя полученные ключи, можно приступить к дешифровке основного отчёта, хранящегося в файле rock_sample.dat.

Для сокрытия информации в изображении thin_section.png Пётр применил LSB-метод, производя замену третьего значащего младшего бита в компонентах R, G и B каждого пикселя.

Обычное LSB-стеганографирование заменяет младший бит (bit_0) в каждом цветовом канале R, G и B. Однако Пётр использует усложнённую модификацию метода, где он заменяет третий значащий бит.

Например, необходимо зашифровать бит сообщения = 0. Имеется цвет пикселя изображения: R = 101. Двоичное представление данного компонента канала: $101 \rightarrow 01100101_2$

1) Читаем текущий bit_2 для замены: $bit_2 = 1$

2) Записываем «0» вместо него, изменяя только данный бит.

Таким образом, до шифрования было: 01100101_2 (= 101). После шифрования станет: 01100001_2 (= 97)

После проведения спектрального и статистического анализа изображения thin_section.png Мария выделила последовательность пикселей,



в каждом цветовом канале которых наблюдались отклонения, характерные для внедрённого стеганографического сообщения. Мария извлекла из этих пикселей их значения каналов RGB, получив следующую последовательность.

Пиксель	R	G	B
1	205	172	139
2	2	11	96
3	157	229	219
4	18	87	193
5	195	113	252
6	231	206	17
7	179	125	63
8	47	88	216
9	237	158	125
10	46	135	174
11	145	199	214
12	27	237	243
13	186	113	190
14	57	135	69
15	144	104	116
16	231	239	94
17	93	200	111
18	226	172	242
19	201	84	15
20	212	162	183
21	178	32	193
22	177	194	86
23	190	248	219
24	111	143	110
25	143	79	208
26	178	159	78
27	184	106	147
28	28	134	167
29	49	117	245
30	253	220	97
31	74	174	128
32	95	133	105
33	249	98	182

Пётр, известный своей любовью к горным треккигам, в файле thin_section.png для последующей защиты дешифрованной битовой последовательности применил специальный «Горный код» стеганографии. Его принцип основан на использовании неравномерного кодирования, где



частым символам соответствуют короткие коды, что минимизирует общий размер секретного сообщения. Дополнительный уровень защиты обеспечивают «мусорные» символы – специальные ловушки, встроенные в код. При расшифровке легитимный получатель должен распознать и отфильтровать этот мусор, чтобы получить исходные данные. Таблица используемых Петром кодов представлена ниже.

Символ	Код	Символ	Код
{	110000	7	111100
}	110001	8	111101
"	110010	9	111110
:	110011	n	000
,	110100	e	001
0	110101	d	010
1	110110	p	0110
2	110111	q	0111
3	111000	x	1000
4	111001	z	1001
5	111010	y	1010
6	111011		

Неравномерный код – это код, в котором кодовые слова, например, последовательности битов, имеют разную длину.

Например, при декодировании битовой последовательности 000001010 с помощью вышепредставленной таблицы, процесс декодирования будет заключаться в следующем:

1) Читаем биты слева-направо: 0 → 00 → 000.

000 есть в таблице? Да, это n.

Результат: n.

2) Продолжаем со следующего бита: 0 → 00 → 001.

001 есть в таблице? Да, это e.

Результат: e.

3) Продолжаем со следующего бита: 0 → 01 → 010.

010 есть в таблице? Да, это d.

Результат: d.

Расшифрованное сообщение n e d.

Файл rock_sample.dat представляет собой последовательность десятичных чисел, каждая из которых – зашифрованная буква передаваемого сообщения. Исходный текст отчёта перед шифрованием был преобразован в числа по правилу: каждый символ кодировался своим порядковым номером в алфавите ($a=1$, $b=2$, ..., $z=33$). Для шифрования Петром использовалась формула: $C_i = M_i^e \bmod n$,

где C_i – i -й элемент последовательности шифротекста C ;

M_i – i -й элемент последовательности шифра M ;



e – открытая экспонента – параметр открытого ключа RSA – число, используемое при шифровании в качестве степени;

n – модуль операции взятия остатка;

mod – операция взятия остатка от деления.

Для дешифрования должна использоваться формула $M_i = C_i^d \text{mod } n$,

где M_i – i -й элемент последовательности шифра M ;

C_i – i -й элемент последовательности шифротекста C ;

d – секретная экспонента – параметр закрытого ключа RSA – число, используемое при дешифровке в качестве степени;

n – модуль операции взятия остатка;

mod – операция взятия остатка от деления.

Совокупность значений (e, n) и (d, n) является открытым и закрытым ключами, используемыми при шифровании и дешифровании соответственно.

Например, RSA-шифрование с открытым ключом: $(e, n) = (17, 3233)$ – и закрытым ключом: $(d, n) = (2753, 3233)$ заключается в следующем.	
При шифровании сообщения $M = 65$ отправляемый шифротекст $C = M^e \text{mod } n = 65^{17} \text{mod } 3233 = 2790$	
При дешифровке полученного шифротекста $C = 2790$ восстановленное сообщение $M = C^d \text{mod } n = 2790^{2753} \text{mod } 3233 = 65$.	

Примечание

Чтобы вычислить выражение вида $M = C^d \text{mod } n$, иногда невыгодно сначала находить большое число C^d , ведь оно может быть огромным. Удобнее использовать алгоритм быстрого возведения в степень по модулю.

Идея алгоритма:

1. Разложить показатель степени на степени двойки.
2. Последовательно возводить число в квадрат.
3. После каждого шага брать остаток по модулю.
4. Умножать только те степени, которые нужны по разложению.

Например, если нужно найти значение выражения $11^5 \text{mod } 9$, то степень 5 можно представить как сумму степеней 4 и 1: $5 = 4 + 1$. Следовательно:

1. Находим остаток: $11^2 \text{mod } 9 = 4$, т.к. $9 \times 13 = 117$ (остаток 4).
 2. Используем найденный остаток, чтобы получить $11^4 \text{mod } 9$.
 $11^4 \text{mod } 9 = (11^2)^2 \text{mod } 9 \equiv 4^2 \text{mod } 9 = 7$, т.к. $9 \times 1 = 9$ (остаток 7)
 3. Перемножаем найденный остаток с 11 – и снова берём остаток.
 $11^5 \text{mod } 9 = (11^4 \times 11) \text{mod } 9 \equiv (7 \times 11) \text{mod } 9 = 77 \text{mod } 9 = 5$, т.к. $9 \times 8 = 72$ (остаток 5)
- Таким образом, $11^5 \text{mod } 9 = 5$.

Используя данные, извлечённые из стеганографического сообщения, расшифруйте `rock_sample.dat` и узнайте предварительное название новой горной породы. Содержимое `rock_sample.dat`:

Первое слово – 33 16 12 39 19; второе слово – 7 16 44 25 20 8 28 20 11.

В ответе укажите два слова расшифрованного сообщения в формате



«Первое слово» «Второе слово» (например, Медное золото). Для получения дополнительных баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю, используемого при дешифровке.

Решение

На первом этапе необходимо определить секретный ключ, который был внедрен Петром методом стеганографии в файл `thin_section.png`. Для сокрытия информации в изображении `thin_section.png` Пётр применил LSB-метод, производя замену третьего значащего бита в компонентах R, G и B каждого пикселя. Следовательно, необходимо последовательно извлечь биты сначала из красного канала первого пикселя, затем из зеленого, затем из синего, записать в том порядке, в котором они были закодированы, а потом перейти к следующему пикселю.

Каждый цветовой канал пикселя представлен числом от 0 до 255. В связи с тем, что замена третьего значащего бита осуществляется в двоичной записи числа, необходимо перевести значения извлечённой последовательности каналов RGB в двоичную систему, выделить третий бит каждого числа и записать получившуюся последовательность.

Для перевода числа из десятичной системы счисления в двоичную необходимо разделить число в десятичной системе на 2 и записать остаток до тех пор, пока неполное частное не будет равно нулю. Полученные остатки записываются в обратном порядке. В рамках задачи можно ограничиться делением числа до момента нахождения значения третьего значащего бита.

Например, извлечение битов из первого пикселя выглядит следующим образом.

Пиксель	Каналы	Число в 10-ой системе счисления	Перевод в 2-ую систему счисления	Третий значащий бит
1	R	205_{10}	$\begin{array}{l} 1) \frac{205}{2} = 102 \text{ (остаток 1)} \\ 2) \frac{102}{2} = 51 \text{ (остаток 0)} \\ 3) \frac{51}{2} = 25 \text{ (остаток 1)} \end{array}$	1
	G	172_{10}	$\begin{array}{l} 1) \frac{172}{2} = 86 \text{ (остаток 0)} \\ 2) \frac{86}{2} = 43 \text{ (остаток 0)} \\ 3) \frac{43}{2} = 21 \text{ (остаток 1)} \end{array}$	1
	B	139_{10}	$\begin{array}{l} 1) \frac{139}{2} = 69 \text{ (остаток 1)} \\ 2) \frac{69}{2} = 34 \text{ (остаток 1)} \\ 3) \frac{34}{2} = 17 \text{ (остаток 0)} \end{array}$	0

Руководствуясь аналогичным алгоритмом, получаем следующие значения.



Пиксель/ Каналы	Число в 10-ой системе счисления			Третий значащий бит		
	R	G	B	R	G	B
1	205	172	139	1	1	0
2	2	11	96	0	0	0
3	157	229	219	1	1	0
4	18	87	193	0	1	0
5	195	113	252	0	0	1
6	231	206	17	1	1	0
7	179	125	63	0	1	1
8	47	88	216	1	0	0
9	237	158	125	1	1	1
10	46	135	174	1	1	1
11	145	199	214	0	1	1
12	27	237	243	0	1	0
13	186	113	190	0	0	1
14	57	135	69	0	1	1
15	144	104	116	0	0	1
16	231	239	94	1	1	1
17	93	200	111	1	0	1
18	226	172	242	0	1	0
19	201	84	15	0	1	1
20	212	162	183	1	0	1
21	178	32	193	0	0	0
22	177	194	86	0	0	1
23	190	248	219	1	0	0
24	111	143	110	1	1	1
25	143	79	208	1	1	0
26	178	159	78	0	1	1
27	184	106	147	0	0	0
28	28	134	167	1	1	1
29	49	117	245	0	1	1
30	253	220	97	1	1	0
31	74	174	128	0	1	0
32	95	133	105	1	1	0
33	249	98	182	0	0	1

Таким образом, в результате дешифровки пикселей изображения получена следующая битовая строка:

110000110010001110011100111110110100010110011111010100111010
00001100111110011000111011110010110001

В задаче сказано, что Пётр использовал для шифрования специальный вод, который основан на использовании неравномерного кодирования. Алгоритм дешифровки в данном случае заключается в следующем: читаем строку по битам слева направо, пошагово накапливая последовательность, пока она не совпадёт с одним из кодов в таблице. Как только совпадение



найденно, записываем символ и начинаем формировать следующую последовательность заново.

Начнём процесс декодирования. Если код не найден сразу (например, при чтении первой 1), то показан процесс накопления последовательности: $1 \rightarrow 11 \rightarrow 110$ и т.д., до момента совпадения с записью в таблице кодов, используемых Петром при шифровании. Максимальная длина кодового слова в таблице кодировки – 6 бит (например, 110000, 110011, 111110 и т.д.), поэтому в процессе накопления битовой последовательности последовательности длиннее 6 символов не рассматриваются.

Шаг	Индекс начала	Биты (извлечённые)	Найденный код	Символ
1	0	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11000 \rightarrow 110000$	110000	{
2	6	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110010$	110010	"
3	12	$0 \rightarrow 00 \rightarrow 001$	001	e
4	15	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 110011$	110011	:
5	21	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1001$	1001	z
6	25	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1111 \rightarrow 11111 \rightarrow 111110$	111110	9
7	31	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1101 \rightarrow 110100$	110100	,
8	37	$0 \rightarrow 01 \rightarrow 010$	010	d
9	40	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 110011$	110011	:
10	46	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 111010$	111010	5
11	52	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1001$	1001	z
12	56	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1101 \rightarrow 110100$	110100	,
13	62	$0 \rightarrow 00 \rightarrow 000$	000	n
14	65	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 110011$	110011	:
15	71	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 111001$	111001	4
16	77	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1000$	1000	x
17	81	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 111011$	111011	6
18	87	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 110010$	110010	"
19	93	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 110001$	110001	}

Собирая полученные символы в порядке их дешифровки, получаем точную расшифровку битовой строки: {"e:z9,d:5z,n:4x6"}. В условии задачи говорится про «мусорные символы» – ловушки, которые Пётр вставляет в сообщение. В таблице кодов есть набор букв-«включений» x, y, z (и т.д.), и они выступают как мусор в конкретном сообщении: они появляются внутри ключей/значений, но не несут смысла для конечного JSON.

Если удалить все мусорные буквы x, y, z из полученной строки {"e:z9,d:5z,n:4x6"}, то получим корректную JSON строку: {"e:9,d:5,n:46"}. Следовательно, получены данные для расшифровки текстового сообщения из файла rock_sample.dat, а именно модуль $n=46$ и секретная экспонента $d=5$.

Файл rock_sample.dat содержит две последовательности целых чисел, где каждое число – зашифрованный символ шифротекста:

- Первое слово: 33 16 12 39 19



- Второе слово: 7 16 44 25 20 8 28 20 11

Из условия задачи воспользуемся формулой дешифрования: $M_i = C_i^d \bmod n$,

где M_i – i -й элемент последовательности шифра M (восстановленный номер буквы (1 = а, 2 = б, ..., 33 = я));

C_i – i -й элемент последовательности шифротекста C ;

d – секретная экспонента;

n – модуль операции взятия остатка;

$\bmod$ – операция взятия остатка от деления.

Дешифруем первое слово: $C = [33 \ 16 \ 12 \ 39 \ 19]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 33$

$$33^2 = 1089 \rightarrow 1089 \bmod 46 = 31, \text{ т. к. } 46 \times 23 = 1058 \text{ (остаток 31)}$$

$$\text{Таким образом, } 33^2 \equiv 31 \pmod{46}$$

$$33^4 = (33^2)^2 \equiv 31^2 = 961 \rightarrow 961 \bmod 46 = 41, \text{ т. к. } 46 \times 20 = 920 \text{ (остаток 41)}$$

$$\text{Таким образом, } 33^4 \equiv 41 \pmod{46}$$

$$33^5 = 33^4 \times 33 \equiv 41 \times 33 = 1353 \rightarrow 1353 \bmod 46 = 19, \text{ т. к. } 46 \times 29 = 1334 \text{ (остаток 19)}$$

$$\text{Таким образом, } M_1 = 33^5 \pmod{46} = 19$$

Следовательно, $M_1 = 19$, что соответствует букве алфавита «с».

2. $C_2 = 16$

$$16^2 = 256 \rightarrow 256 \bmod 46 = 26, \text{ т. к. } 46 \times 5 = 230 \text{ (остаток 26)}$$

$$\text{Таким образом, } 16^2 \equiv 26 \pmod{46}$$

$$16^4 = (16^2)^2 \equiv 26^2 = 676 \rightarrow 676 \bmod 46 = 32, \text{ т. к. } 46 \times 14 = 644 \text{ (остаток 32)}$$

$$\text{Таким образом, } 16^4 \equiv 32 \pmod{46}$$

$$16^5 = 16^4 \times 16 \equiv 32 \times 16 = 512 \rightarrow 512 \bmod 46 = 6, \text{ т. к. } 46 \times 11 = 506 \text{ (остаток 6)}$$

$$\text{Таким образом, } M_2 = 16^5 \pmod{46} = 6$$

Следовательно, $M_2 = 6$, что соответствует букве алфавита «е».

3. $C_3 = 12$

$$12^2 = 144 \rightarrow 144 \bmod 46 = 6, \text{ т. к. } 46 \times 3 = 138 \text{ (остаток 6)}$$

$$\text{Таким образом, } 12^2 \equiv 6 \pmod{46}$$

$$12^4 = (12^2)^2 \equiv 6^2 = 36 \rightarrow 36 \bmod 46 = 36$$

$$\text{Таким образом, } 12^4 \equiv 36 \pmod{46}$$

$$12^5 = 12^4 \times 12 \equiv 36 \times 12 = 432 \rightarrow 432 \bmod 46 = 18, \text{ т. к. } 46 \times 9 = 414 \text{ (остаток 18)}$$

$$\text{Таким образом, } M_3 = 12^5 \pmod{46} = 18$$

Следовательно, $M_3 = 18$, что соответствует букве алфавита «р».

4. $C_4 = 39$



$$39^2 = 1521 \rightarrow 1521 \bmod 46 = 3, \text{ т. к. } 46 \times 33 = 1518 \text{ (остаток 3)}$$

$$\text{Таким образом, } 39^2 \equiv 3 \pmod{46}$$

$$39^4 = (39^2)^2 \equiv 3^2 = 9$$

$$39^5 = 39^4 \times 39 \equiv 9 \times 39 = 351 \rightarrow 351 \bmod 46 = 29, \text{ т. к. } 46 \times 7 = 322 \text{ (остаток 29)}$$

$$\text{Таким образом, } M_4 = 39^5 \pmod{46} = 29$$

Следовательно, $M_4 = 29$, что соответствует букве алфавита «ы».

5. $C_5 = 19$

$$19^2 = 361 \rightarrow 361 \bmod 46 = 39, \text{ т. к. } 46 \times 7 = 322 \text{ (остаток 39)}$$

$$\text{Таким образом, } 19^2 \equiv 39 \pmod{46}$$

$$19^4 = (19^2)^2 \equiv 39^2 = 1521 \bmod 46 = 3$$

$$\text{Таким образом, } 19^4 \equiv 3 \pmod{46}$$

$$19^5 = 19^4 \times 19 \equiv 3 \times 19 = 57 \rightarrow 57 \bmod 46 = 11$$

$$\text{Таким образом, } M_5 = 19^5 \pmod{46} = 11$$

Следовательно, $M_5 = 11$, что соответствует букве алфавита «й».

Соединив расшифрованные буквы, получаем слово «серый».

Дешифруем второе слово: $C = [7 \ 16 \ 44 \ 25 \ 20 \ 8 \ 28 \ 20 \ 11]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 7$

$$7^2 = 49 \rightarrow 49 \bmod 46 = 3, \text{ т. к. } 46 \times 1 = 46 \text{ (остаток 3)}$$

$$\text{Таким образом, } 7^2 \equiv 3 \pmod{46}$$

$$7^4 = (7^2)^2 \equiv 3^2 = 9 \rightarrow 9 \bmod 46 = 9$$

$$\text{Таким образом, } 7^4 \equiv 9 \pmod{46}$$

$$7^5 = 7^4 \times 7 \equiv 9 \times 7 = 63 \rightarrow 63 \bmod 46 = 17, \text{ т. к. } 46 \times 1 = 46 \text{ (остаток 17)}$$

$$\text{Таким образом, } M_1 = 7^5 \pmod{46} = 17$$

Следовательно, $M_1 = 17$, что соответствует букве алфавита «п».

2. $C_2 = 16$

Аналогично $C_2 = 16$ для первого слова $M_2 = M_2 = 6$, что соответствует букве алфавита «е».

3. $C_3 = 44$

$$44^2 = 1936 \rightarrow 1936 \bmod 46 = 4, \text{ т. к. } 46 \times 42 = 1932 \text{ (остаток 8)}$$

$$\text{Таким образом, } 44^2 \equiv 8 \pmod{46}$$

$$44^4 = (44^2)^2 \equiv 8^2 = 16 \rightarrow 16 \bmod 46 = 16$$

$$\text{Таким образом, } 44^4 \equiv 16 \pmod{46}$$



$$44^5 = 44^4 \times 44 \equiv 16 \times 44 = 704 \rightarrow 704 \bmod 46 = 14, \text{ т. к. } 46 \times 15 = 690 \text{ (остаток 14)}$$

$$\text{Таким образом, } M_3 = 44^5 \bmod 46 = 14$$

Следовательно, $M_3 = 14$, что соответствует букве алфавита «м».

4. $C_4 = 25$

$$25^2 = 625 \rightarrow 625 \bmod 46 = 27, \text{ т. к. } 46 \times 13 = 598 \text{ (остаток 27)}$$

$$\text{Таким образом, } 25^2 \equiv 27 \pmod{46}$$

$$25^4 = (25^2)^2 \equiv 27^2 = 729 \rightarrow 729 \bmod 46 = 39, \text{ т. к. } 46 \times 15 = 690 \text{ (остаток 39)}$$

$$\text{Таким образом, } 25^4 \equiv 39 \pmod{46}$$

$$25^5 = 25^4 \times 25 \equiv 39 \times 25 = 975 \rightarrow 975 \bmod 46 = 9, \text{ т. к. } 46 \times 21 = 966 \text{ (остаток 9)}$$

$$\text{Таким образом, } M_3 = 25^5 \bmod 46 = 9$$

Следовательно, $M_4 = 9$, что соответствует букве алфавита «з».

5. $C_5 = 20$

$$20^2 = 400 \rightarrow 400 \bmod 46 = 32, \text{ т. к. } 46 \times 8 = 368 \text{ (остаток 32)}$$

$$\text{Таким образом, } 20^2 \equiv 32 \pmod{46}$$

$$20^4 = (20^2)^2 \equiv 32^2 = 1024 \rightarrow 1024 \bmod 46 = 12, \text{ т. к. } 46 \times 22 = 1012 \text{ (остаток 12)}$$

$$\text{Таким образом, } 20^4 \equiv 12 \pmod{46}$$

$$20^5 = 20^4 \times 20 \equiv 12 \times 20 = 240 \rightarrow 240 \bmod 46 = 10, \text{ т. к. } 46 \times 5 = 230 \text{ (остаток 10)}$$

$$\text{Таким образом, } M_5 = 20^5 \bmod 46 = 10$$

Следовательно, $M_5 = 10$, что соответствует букве алфавита «и».

6. $C_6 = 8$

$$8^2 = 64 \rightarrow 64 \bmod 46 = 18, \text{ т. к. } 46 \times 1 = 46 \text{ (остаток 18)}$$

$$\text{Таким образом, } 8^2 \equiv 18 \pmod{46}$$

$$8^4 = (8^2)^2 \equiv 18^2 = 324 \rightarrow 324 \bmod 46 = 2, \text{ т. к. } 46 \times 7 = 322 \text{ (остаток 2)}$$

$$\text{Таким образом, } 8^4 \equiv 2 \pmod{46}$$

$$8^5 = 8^4 \times 8 \equiv 2 \times 8 = 16 \rightarrow 16 \bmod 46 = 16$$

$$\text{Таким образом, } M_2 = 8^5 \bmod 46 = 16$$

Следовательно, $M_6 = 16$, что соответствует букве алфавита «о».

7. $C_7 = 28$

$$28^2 = 784 \rightarrow 784 \bmod 46 = 2, \text{ т. к. } 46 \times 17 = 782 \text{ (остаток 2)}$$

$$\text{Таким образом, } 28^2 \equiv 2 \pmod{46}$$

$$28^4 = (28^2)^2 \equiv 2^2 = 4 \rightarrow 4 \bmod 46 = 4$$

$$\text{Таким образом, } 28^4 \equiv 4 \pmod{46}$$

$$28^5 = 28^4 \times 28 \equiv 4 \times 28 = 112 \rightarrow 112 \bmod 46 = 20, \text{ т. к. } 46 \times 2 = 92 \text{ (остаток 20)}$$

$$\text{Таким образом, } M_7 = 28^5 \bmod 46 = 20$$

Следовательно, $M_7 = 20$, что соответствует букве алфавита «т».

8. $C_8 = 20$



Аналогично $C_5 = 20$ $M_8 = M_2 = 10$, что соответствует букве алфавита «и».

9. $C_9 = 11$

$$11^2 = 121 \rightarrow 121 \bmod 46 = 29, \text{ т.к. } 46 \times 2 = 92 \text{ (остаток 29)}$$

Таким образом, $11^2 \equiv 29 \pmod{46}$

$$11^4 = (11^2)^2 \equiv 29^2 = 841 \rightarrow 841 \bmod 46 = 13, \text{ т.к. } 46 \times 18 = 828 \text{ (остаток 13)}$$

Таким образом, $11^4 \equiv 13 \pmod{46}$

$$11^5 = 11^4 \times 11 \equiv 13 \times 11 = 143 \rightarrow 143 \bmod 46 = 5, \text{ т.к. } 46 \times 3 = 138 \text{ (остаток 5)}$$

Таким образом, $M_9 = 11^5 \pmod{46} = 5$

Следовательно, $M_9 = 5$, что соответствует букве алфавита «д».

Соединив расшифрованные буквы, получаем слово «пемзиотид».

В ответе сказано указать два слова расшифрованного сообщения в формате «Первое слово» «Второе слово», соединяя получаем серый пемзиотид.

Ответ: серый пемзиотид

Для получения максимального количества баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю при дешифровке.

C++

```
#include <iostream>

int mod_exp(int base, int exp, int mod) {
    int result = 1;
    while (exp > 0) {
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

int main() {
    int base = 33;
    int exp = 5;
    int mod = 46;

    int result = mod_exp(base, exp, mod);
    std::cout << result << std::endl;

    return 0;
}
```

Python

```
def mod_exp(base, exp, mod):
    result = 1
    while exp > 0:
        if exp % 2 == 1:
```



```
        result = (result * base) % mod
    base = (base * base) % mod
    exp //= 2
    return result

base = 12
exp = 5
mod = 46

result = mod_exp(base, exp, mod)
print(result)
```

**Задача 5 (30 баллов)****Космический зонд на ледяной комете**

На длинной ледяной комете расположено $n \geq 2$ научно-исследовательских станций вдоль числовой оси с целочисленными координатами $b_1, b_2, \dots, b_n$, упорядоченными по возрастанию, где $b_1 = 0$. Космический зонд начинает свою миссию на станции b_1 и должен достичь финальной станции b_n , используя адаптивную систему навигации.

Принцип работы навигационной системы

Зонд использует двухфазную систему перемещения. Сначала происходит **калибровка**: зонд определяет ближайшую станцию впереди (обозначим её b_j), вычисляет эталонное расстояние D как разность между координатой этой станции и текущей позицией зонда, затем телепортируется на станцию b_j , фиксируя найденное значение D как текущий масштаб перемещения.

После калибровки начинается **поиск целевого расстояния**: зонд последовательно сканирует позиции впереди себя (текущая позиция + 1, текущая позиция + 2, текущая позиция + 3 и так далее). Когда зонд обнаруживает станцию на некоторой позиции X , возможны три сценария. Если X равно b_n , то зонд перемещается на нее и успешно завершает миссию. Если на позиции $(X - D)$ также существует станция, то зонд перемещается на станцию X и выполняет новую калибровку в этой точке. В противном случае зонд продолжает сканирование позиций дальше.

Формат входных данных

В первой строке задано целое число n — количество станций, различных целочисленных точек ($2 \leq n \leq 300000$). Во второй строке через пробел записаны n целых чисел $b_1, b_2, \dots, b_n$ — координаты станций в порядке возрастания. Самая левая из них имеет значение 0, самая правая не превосходит 10^9 .

Формат выходных данных

Требуется вывести одно целое число D — итоговое эталонное расстояние, при котором зонд достигает финальной станции b_n .

Примеры

стандартный ввод	стандартный вывод
7 0 3 4 5 6 8 13	2
9 0 3 4 5 6 8 9 10 13	3
12 0 3 4 5 7 8 11 12 14 17 18 24	2

Пояснения к примерам

Иллюстрация к первому примеру.

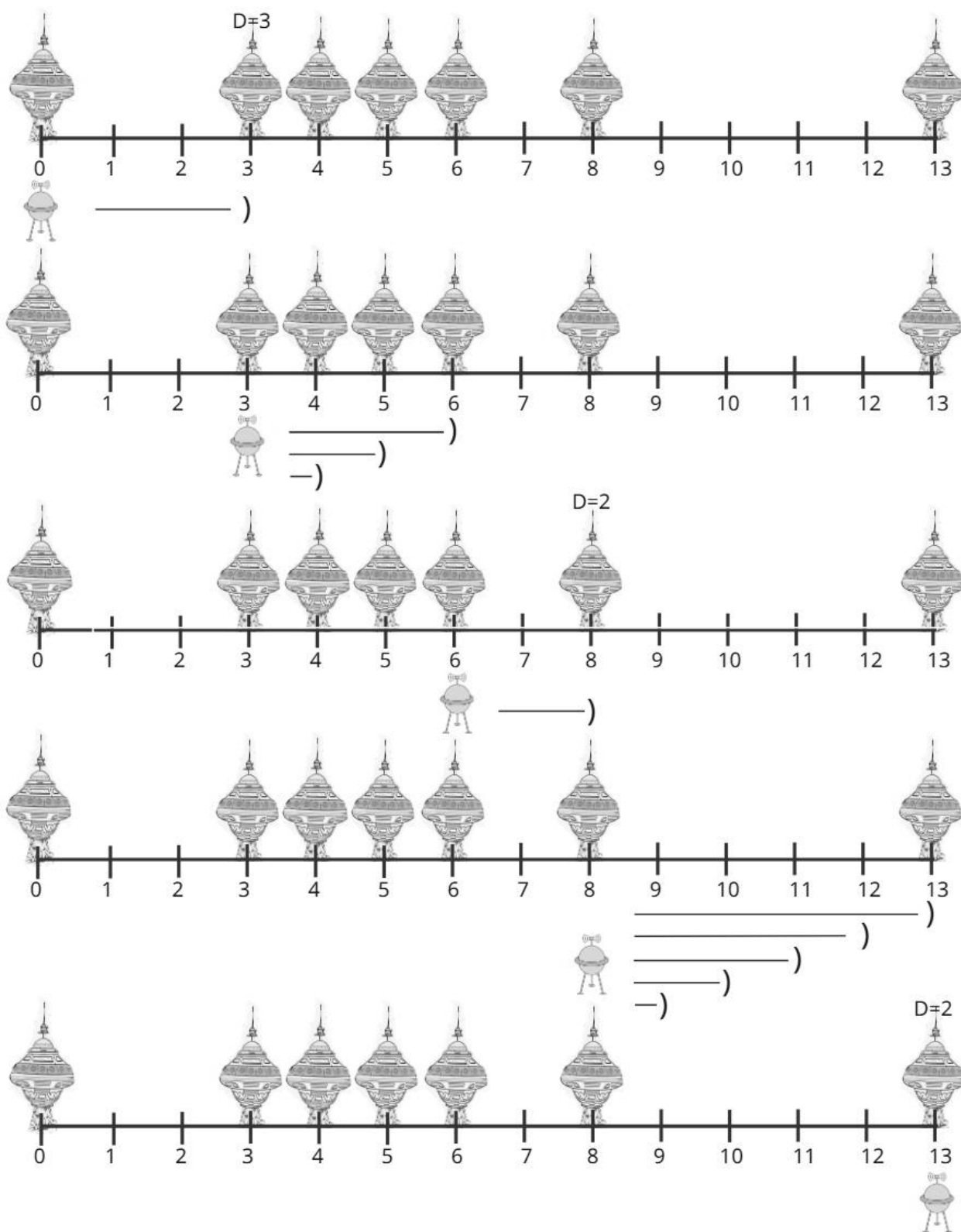
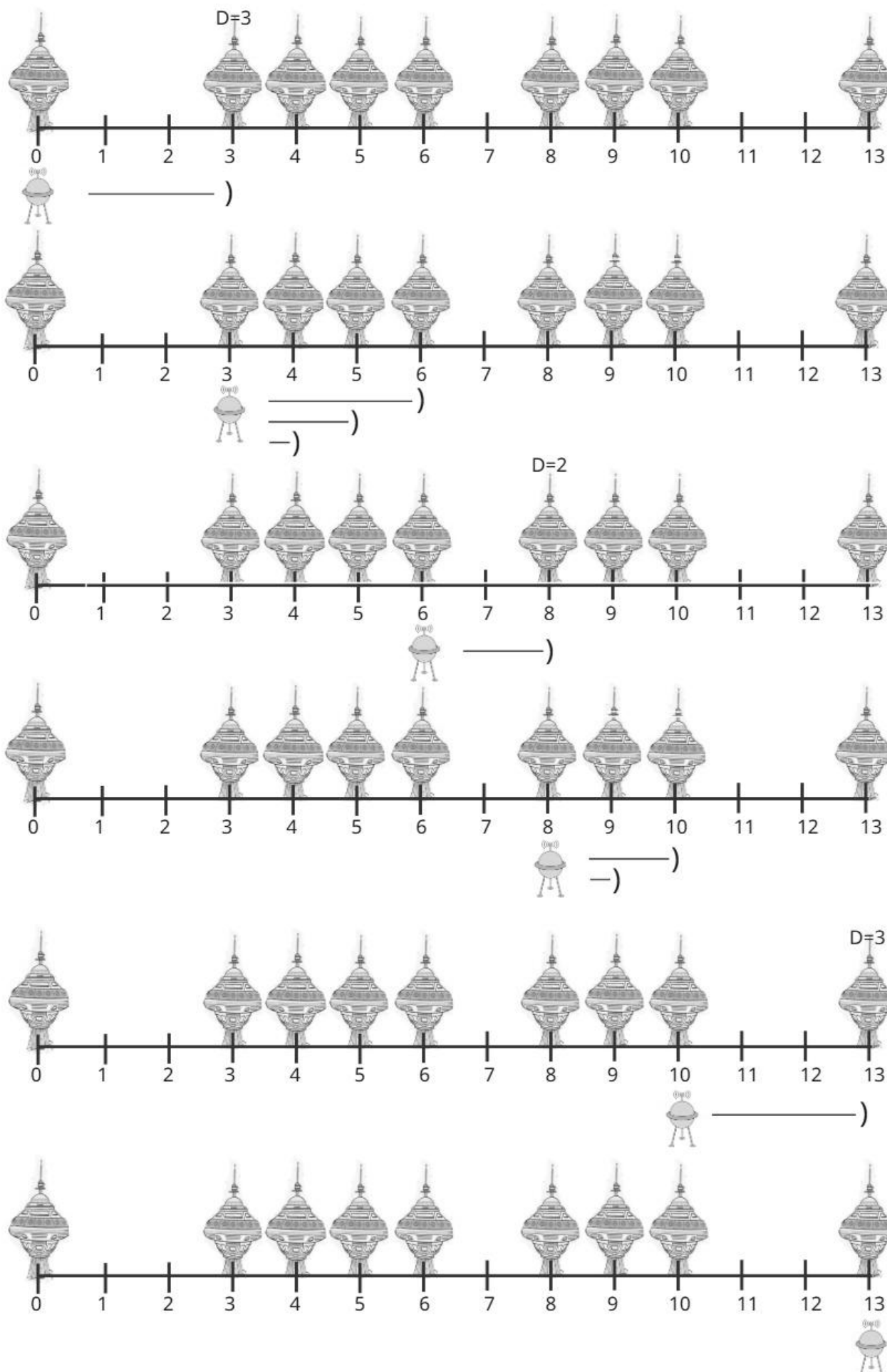


Иллюстрация ко второму примеру.





Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, обязательно **добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных, поясните решение. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
12 0 3 4 5 6 7 9 10 12 13 18 20 24	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

Результаты выполнения программы для заданных исходных данных:

Вариант 1

стандартный ввод	стандартный вывод
12 0 3 4 5 6 7 9 10 12 13 18 20	2
13 0 3 4 5 6 7 9 10 12 13 18 20 24	4

Вариант 2

стандартный ввод	стандартный вывод
12 0 3 4 5 6 7 9 10 12 13 14 24	10

Вариант 3

стандартный ввод	стандартный вывод
12 0 3 4 7 9 10 12 13 14 18 19 24	4

Вариант 4

стандартный ввод	стандартный вывод
12 0 2 3 4 7 9 10 11 15 17 18 24	6

Зонд стартует в точке 0 и должен добраться до самой правой станции b_n , используя особый принцип перемещений с калибровкой и сканированием.



Нужно найти итоговое значение эталонного расстояния D , при котором зонд доберётся до b_n .

Калибровка: зонд всегда сначала перемещается на ближайшую станцию впереди (то есть, просто переходит к следующей координате в отсортированном массиве станций) и считает разницу D .

Сканирование: после калибровки зонд начинает перебирать каждую позицию вперед по оси (по сути, по всем целым числам), пока не найдёт станцию. Когда станцию найдёт, он проверит: если станция под номером X и станция под номером $X-D$ существуют, он делает калибровку на этой новой позиции. Если попадает ровно в конечную станцию — задача решена.

Процесс повторяется, пока не будет достигнута финальная станция.

Решение на C++

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int solve_probe_fast() {
    int n;
    if (!(cin >> n)) return 0;

    vector<int> station_list(n);
    for (int i = 0; i < n; ++i) {
        cin >> station_list[i];
    }

    int ind = 0; // Текущий индекс станции, на которой стоит зонд
    int D = 0;
    int target = station_list[n - 1];

    while (station_list[ind] < target) {
        // --- Калибровка ---
        // Прыгаем на следующую станцию и фиксируем D
        int prev_pos = station_list[ind];
        ind++; // Переместились на b_j
        D = station_list[ind] - prev_pos;

        if (station_list[ind] == target) return D;

        // --- Сканирование ---
        bool found = false;
        // Перебираем только существующие станции впереди
        for (int j = ind + 1; j < n; ++j) {
            int X = station_list[j];

            // Если это финал, миссия всегда завершается успешно
            if (X == target) {
                ind = j;
                found = true;
                break;
            }

            // Провераем условие X - D существует.
            // Используем бинарный поиск по вектору - это O(log n)
            if (binary_search(station_list.begin(), station_list.end(), X - D)) {
```



```
        ind = j;
        found = true;
        break;
    }
}

    if (!found) return D; // Если не нашли следующую точку перехода
}
return D;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cout << solve_probe_fast() << endl;
    return 0;
}
```

Зонд перемещается по станциям, координаты которых хранятся в отсортированном массиве. Для быстрой проверки существования станции в произвольной точке используется set или бинарный поиск, что позволяет выполнять проверку за $O(\log n)$.

Заполнение массива и создание структуры для поиска занимает $O(n \log n)$. Несмотря на вложенный цикл (поиск станции внутри цикла перемещения), указатель текущей станции в массиве движется только вперед. Каждая станция просматривается не более одного раза. Внутри цикла выполняется поиск в set или бинарный поиск, что занимает $O(\log n)$.

Итоговая сложность: $O(n \log n)$ в худшем случае.

Для хранения координат n станций требуется $O(n)$ памяти.

Решение на Python

```
import sys

def solve():
    # Быстрое чтение данных
    data = sys.stdin.read().split()
    if not data:
        return

    n = int(data[0])
    stations = list(map(int, data[1:]))
    station_set = set(stations) # Оставляем для быстрой проверки X-D за
O(1)

    target = stations[-1]
    curr_idx = 0

    while stations[curr_idx] < target:
        # Фаза калибровки
        prev_pos = stations[curr_idx]
        curr_idx += 1 # Переместились на ближайшую станцию
        D = stations[curr_idx] - prev_pos

        if stations[curr_idx] == target:
            return D

    # Фаза сканирования
```



```
found = False
# Перебираем индексы станций впереди, а не координаты
for j in range(curr_idx + 1, n):
    X = stations[j]

    # Финальная станция — всегда успех
    if X == target:
        curr_idx = j
        found = True
        break

    # Условие: на позиции X-D должна быть станция
    if (X - D) in station_set:
        curr_idx = j
        found = True
        break

# Если сканирование не нашло подходящую станцию
if not found:
    return D

return D

# Выводим результат
print(solve())
```

Для этого решения чтение входных данных $O(n)$. Создание множества $O(n)$. Добавление n элементов в хеш-таблицу (которой является `set` в Python) требует $O(1)$ в среднем на каждый элемент.

Проверка условия $(X - D) \text{ in } \text{station_set}$: $O(1)$ в среднем. Это ключевое преимущество Python-решения. Поиск в `set` реализован через хеш-таблицу, что дает константное время доступа, в отличие от $O(\log n)$ при бинарном поиске.

Суммарно мы имеем $O(n) + O(n) + O(n) \sim O(n)$.

Решение перебором со сложностью $O(n^2 \max_coords)$.

```
def solve():
    n = int(input())
    station_list = list(map(int, input().split()))
    station_set = set(station_list)
    ind = 0
    current_pos = station_list[ind]
    target = station_list[-1]
    while current_pos != target:
        D = station_list[ind + 1] - station_list[ind]
        # Сканирование
        current_pos += D
        ind += 1
        if current_pos == n - 1:
            return D
        else:
            i = current_pos + 1

            while not (i in station_set and (i - D) in station_set) and i !=
target:
                i += 1
            current_pos = i
            ind = station_list.index(i)
    return D
```



```
solve()
```



Задача 6 (25 баллов)

Источник вечной энергии

Известный изобретатель Иван Иванович Иванов разработал принципиально новую установку, позволяющую генерировать сколько угодно безопасной энергии. Он поместил в точки прямой **0** и **D** два направленных друг на друга источника специально заряженных частиц. В момент времени **0** источник с координатой **0** генерирует **n** частиц, которые движутся по прямой в сторону второго генератора. Каждая из них имеет свою скорость v_i , скорости всех частиц попарно различны. Как только какая-то частица достигает источника с координатой **D**, она им поглощается и этот источник выпускает ровно такую же по скорости частицу в противоположном направлении. Эта частица движется обратно в сторону начала координат и снова поглощается источником с координатой **0**, после чего процесс испускания частицы в сторону второго источника снова повторяется.

Известно, что как только две частицы попадают в одну и ту же точку прямой, они генерируют одну единицу энергии. Если в одной точке оказываются несколько частиц, каждая пара среди встретившихся генерирует единицу энергии, в частности, в момент времени **0**, когда **n** частиц стартуют из точки **0**, они уже создают $\frac{n(n-1)}{2}$ единиц энергии. Если две частицы одновременно достигают какого-то из источников, то они генерируют одну единицу перед поглощением, и одну единицу уже после возрождения их двойников, движущихся в обратном направлении. За счет множественности встреч, процесс работы источников имеет большой положительный баланс по выработке энергии. Энергия может вырабатываться сколь угодно долго.

Теперь, для написания научной статьи про опыт использования источника вечной энергии, Иван Иванович хочет точно знать момент, когда будет выработана **m**-я по счету единица энергии.

Формат входных данных

В первой строке ввода через пробел содержатся два целых числа **D** и **n** ($0 \leq D \leq 200$, $2 \leq n \leq 2000$). Во второй строке содержатся **n** целых чисел v_i через пробел – скорости частиц ($1 \leq v_i \leq 2000$). Скорости даны в порядке строгого возрастания, то есть $v_1 < v_2 < \dots < v_n$.

В третьей строке содержится одно целое число **m** – номер требуемой по порядку единицы энергии, для которой следует найти точный момент её генерации ($1 \leq m \leq 10^9$).

Формат выходных данных

Вывести в ответ одно вещественное число – время, когда будет сгенерирована **m**-я по порядку единица энергии. Ответ вывести с точностью не менее 2 знаков после десятичной точки.

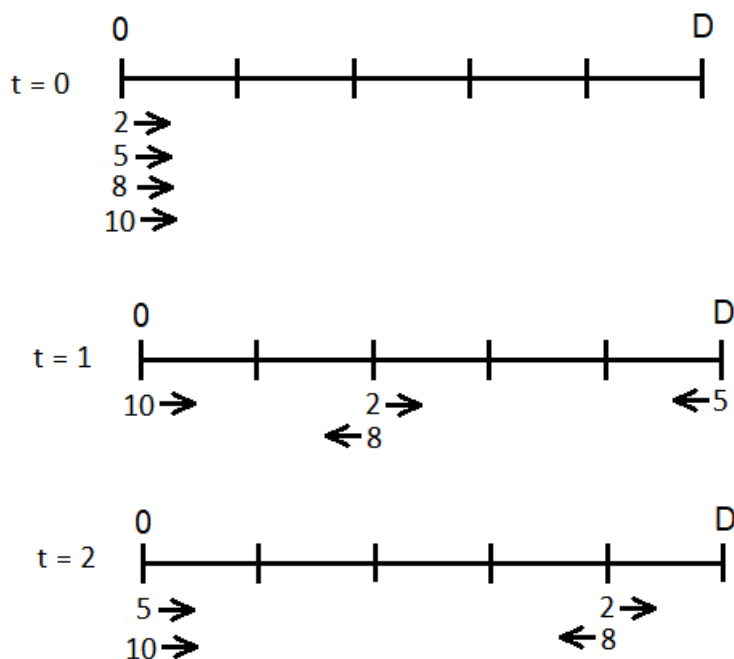


Примеры

стандартный ввод	стандартный вывод
5 4 2 5 8 10 6	0.00
5 4 2 5 8 10 7	0.56
5 4 2 5 8 10 11	1.00
5 4 2 5 8 10 17	1.67
5 4 2 5 8 10 19	1.67
5 4 2 5 8 10 21	2.00

Пояснения к примерам

На следующих рисунках приведено расположение частиц из примеров в моменты времени 0, 1 и 2. Частицы обозначены своими скоростями. Стрелками обозначены направления их движения в соответствующий момент. Если в этот момент частица была регенерирована, она имеет новое направление движения.



Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите



эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, **обязательно добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
5 4 2 5 8 10 15	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

Результаты выполнения программы для заданных исходных данных:

Вариант 1

стандартный ввод	стандартный вывод
5 4 2 5 8 10 15	1.43

Вариант 2

стандартный ввод	стандартный вывод
5 4 2 5 8 10 16	1.54

Вариант 3

стандартный ввод	стандартный вывод
5 4 2 5 8 10 14	1.33

Вариант 4

стандартный ввод	стандартный вывод
5 4 2 5 8 10 13	1.25

Частицы движутся с постоянными скоростями, отражаясь от стенок на расстоянии D . Для заданного времени подсчитываем все столкновения через



столкновения со стенками (каждая частица при ударе встречает все предыдущие) и проверку текущих позиций всех пар частиц. Используется бинарный поиск: ищем минимальное время, когда произойдет ровно k столкновений.

Временная сложность: $O(n^2 \cdot \log(T/\varepsilon))$, n^2 – для подсчета столкновений между всеми парами, $\log(T/\varepsilon)$ – количество итераций бинарного поиска (T – максимальное время, ε – точность).

Пространственная сложность: $O(n)$ для хранения массива скоростей.

Решение на C++

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#pragma GCC target("popcnt")

#define sz(a) (int)a.size()
#define pb push_back
#define all(a) a.begin(), a.end()
#define for0(i, n) for(int i = 0; i < n; i++)
#define for1(i, n) for(int i = 1; i <= n; i++)
#define x first
#define y second
#define int long long

using namespace std;
using namespace __gnu_pbds;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vector<int>> > vvi;
typedef vector<double> vd;
typedef vector<vd> vvd;
typedef tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> ordered_set;

const int INF = 1e18;
const int MOD = 1e9 + 7;
const int LG = 19;
const double EPS = 1e-7;

double x(double M, int V, int d){
    double dst = V * M;
    int cnt = floor((dst + EPS) / d);
    double pin = dst - cnt * d;

    if(cnt % 2 == 0){
        return pin;
    }
    else{
        return d - pin;
    }
}
```



```
int F(double M, vi &v, int d){
    int res = 0;
    for0(i, sz(v)){
        double dst = v[i] * M;
        int cnt = floor((dst + EPS) / d);
        res += cnt * i;

        double tx = x(M, v[i], d);

        for0(j, i){
            double txj = x(M, v[j], d);
            if(cnt % 2 == 0){
                res += txj <= tx + EPS;
            }
            else{
                res += txj >= tx - EPS;
            }
        }
    }
    return res;
}

signed main(){
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);

    int d, n;
    cin >> d >> n;
    vi v(n);
    for0(i, n){
        cin >> v[i];
    }
    int k;
    cin >> k;

    double L = 0, R = 1;
    while(F(R, v, d) <= k){
        R *= 2;
    }
    R *= 2;

    while(R - L > 1e-4){
        double M = (R + L) / 2.0;
        if(F(M, v, d) < k){
            L = M;
        }
        else{
            R = M;
        }
    }

    cout.precision(2);
    cout << fixed << L << endl;
}
```

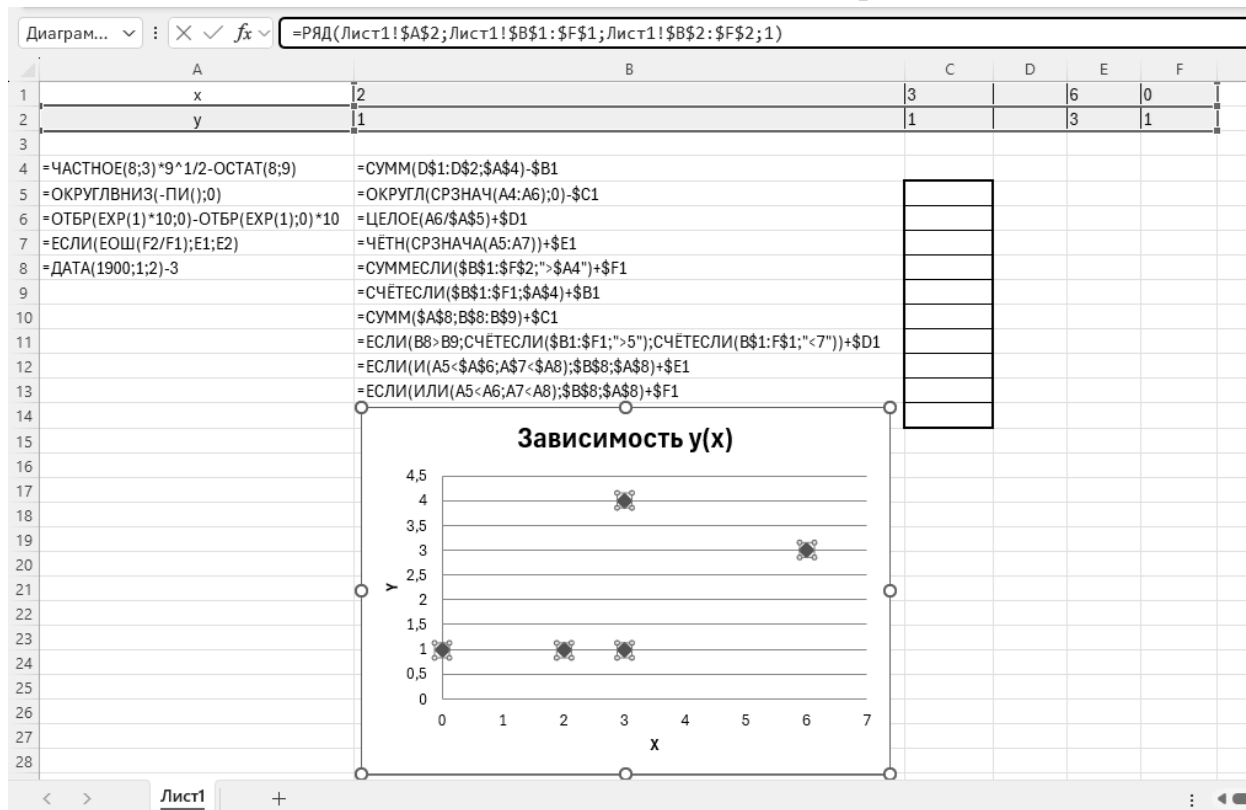


ВАРИАНТ 2

Задача 1 (5 баллов)

Разведчики

От разведчиков были получены сведения, закодированные в виде электронной таблицы. Определите числа в ячейках C5:C14, которые будут получены в результате копирования в них ячеек B4:B13. Также определите недостающие числа в блоке B1:F2 на основании диаграммы.



В ответе записать, в одну строчку через запятую, значения ячеек C5:C14, начиная с ячейки C5 и заканчивая ячейкой C14. Формат ячеек в таблице – «Общий».

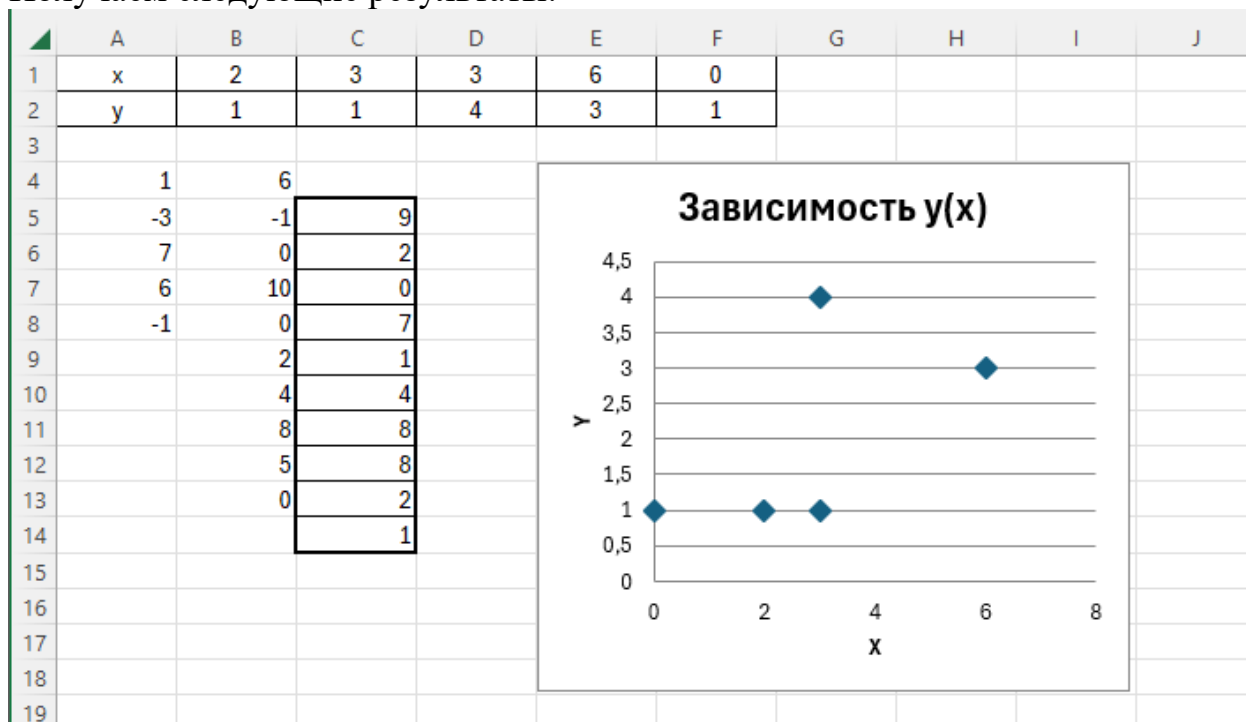
Решение

На основании диаграммы определяем, что в ячейках A1:F2 не отображена точка ($x=3$, $y=4$) расположенная в ячейках D1:D2. Далее копируем формулы, расположенные в ячейках B4:B13 в ячейки C5:C14. В результате получаем следующие формулы:



	C	D
5	=СУММ(E\$1:E\$2;\$A\$4)-\$B2	
6	=ОКРУГЛ(СРЗНАЧ(B5:B7);0)-\$C2	
7	=ЦЕЛОЕ(B7/\$A\$5)+\$D2	
8	=ЧЁТН(СРЗНАЧА(B6:B8))+\$E2	
9	=СУММЕСЛИ(\$B\$1:\$F\$2;">\$A4")+\$F2	
10	=СЧЁТЕСЛИ(\$B\$1:\$F2;\$A\$4)+\$B2	
11	=СУММ(\$A\$8;C\$8:C\$9)+\$C2	
12	=ЕСЛИ(C9>C10;СЧЁТЕСЛИ(\$B2:\$F2;">5");СЧЁТЕСЛИ(C\$1:G\$1;"<7"))+\$D2	
13	=ЕСЛИ(И(B6<\$A\$6;B\$7<\$A9);\$B\$8;\$A\$8)+\$E2	
14	=ЕСЛИ(ИЛИ(B6<B7;B8<B9);\$B\$8;\$A\$8)+\$F2	
15		

Вычисляем значения по формулам в ячейках A4:A8, затем переходим к вычислениям значений в ячейках B4:B13 и, наконец, в ячейках C5:C14. Получаем следующие результаты:



Ответ: 9, 2, 0, 7, 1, 4, 8, 8, 2, 1

**Задача 2 (10 баллов)****Лаборатория кибербезопасности**

Алиса работает в отделе кибербезопасности и тестирует новый алгоритм фильтрации данных. Логическая функция F описывает условия допуска к защищённым файлам:

$$\neg w \vee z \rightarrow (x \wedge y \wedge z \vee \neg x)$$

Алиса составила таблицу истинности, но для проверки знаний своих коллег зашифровала некоторые значения логическими выражениями, в которых **a, b, c** – следующие высказывания:

a – «1 байт = 8 бит»

b – «30 делится на 3 без остатка»

c – « $3 \times 6 = 15$ »

Ниже приведен фрагмент таблицы истинности функции F , содержащий неповторяющиеся строки. Определите, какому столбцу таблицы истинности функции F соответствует каждая из переменных **w, x, y, z**.

?	?	?	?	F
$\neg a \vee b \rightarrow c$		$a \rightarrow b \rightarrow c$	$\neg a \rightarrow b \wedge c$	0
$\neg a \vee \neg b \wedge \neg c$		$a \vee \neg b \vee c$	$a \vee \neg c \vee b$	0
	$a \vee b \vee c$			0
$a \vee b \wedge c$				0

В ответе напишите буквы **w, x, y, z** в том порядке, в котором идут соответствующие им столбцы.

Решение

Определим значения логических высказываний. Таким образом, получим следующее:

$$a=1$$

$$b=1$$

$$c=0$$

Теперь определим значения логических выражений в таблице. В результате получим следующую таблицу:

?	?	?	?	F
0		0	1	0
0		1	1	0
	1			0
1				0

Получаем следующую таблицу истинности:



w	x	y	z	$\neg w$	$\neg w \vee z$	$x \wedge y$	$x \wedge y \wedge z$	$\neg x$	$x \wedge y \wedge z \vee \neg x$	F
0	0	0	0	1	1	0	0	1	1	1
0	0	0	1	1	1	0	0	1	1	1
0	0	1	0	1	1	0	0	1	1	1
0	0	1	1	1	1	0	0	1	1	1
0	1	0	0	1	1	0	0	0	0	0
0	1	0	1	1	1	0	0	0	0	0
0	1	1	0	1	1	1	0	0	0	0
0	1	1	1	1	1	1	1	0	1	1
1	0	0	0	0	0	0	0	1	1	1
1	0	0	1	0	1	0	0	1	1	1
1	0	1	0	0	0	0	0	1	1	1
1	0	1	1	0	1	0	0	1	1	1
1	1	0	0	0	0	0	0	0	0	1
1	1	0	1	0	1	0	0	0	0	0
1	1	1	0	0	0	1	0	0	0	1
1	1	1	1	0	1	1	1	0	1	1

Выберем из нее строки, где функция $F = 0$.

w	x	y	z	F
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
1	1	0	1	0

Сравнивая эту таблицу с заданной в задании, и учитывая, что строки в этой таблице выписаны в случайном порядке, установим искомые наименования столбцов:

w	y	z	x	F
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
1	0	1	1	0

Ответ: w, y, z, x





Из графа видно, что при любой игре первого бельчонка, гарантированно победит второй бельчонок на 4 ходу. Максимально возможное число орехов можно получить, если увеличивать количество орехов в кучке таким образом, чтобы получить как можно более высокий номер хода (5) и количество орехов в кучке (32) перед последним ходом, в котором берется вариант с большим значением $(+5 \cdot 5)$. При этом побеждает первый бельчонок с количеством орехов 57.

Ответ: гарантированно победит второй бельчонок за 4 хода, максимум 57 орехов при победе первого бельчонка.



Задача 4 (20 баллов)

Аномальный криокамень

В ходе арктической экспедиции обнаружена горная порода с аномальной кристаллической решеткой, демонстрирующая свойства высокотемпературной сверхпроводимости. Старший геолог Пётр, опасаясь промышленного шпионажа, передал коллеге Марии, используя многоуровневую систему защиты, зашифрованные данные, содержащие предварительное название породы из двух слов.

Мария получила два файла:

- 1) rock_sample.dat – файл с основным зашифрованным отчётом;
- 2) thin_section.png – микрофотография тонкого среза породы, в которую Петром был внедрён секретный ключ методом стеганографии, имитирующим природные включения. Микрофотография представляет собой 8-битное RGB-изображение.

Алгоритм действий для Марии выглядит следующим образом:

1) Извлечь скрытые данные из изображения thin_section.png. Для этого необходимо проанализировать третьи значащие биты цветовых каналов пикселей, записать их последовательно в том порядке, в котором они были закодированы, и получить битовую последовательность.

2) Эту битовую последовательность необходимо декодировать. В результате она должна превратиться в строку в формате JSON, которая и содержит секретные ключи для алгоритма RSA. В данной задаче строка в формате JSON выглядит следующим образом: {ключ1: значение1, ключ2: значение2, ключ3: значение3}.

3) Используя полученные ключи, можно приступить к дешифровке основного отчёта, хранящегося в файле rock_sample.dat.

Для сокрытия информации в изображении thin_section.png Пётр применил LSB-метод, производя замену третьего значащего младшего бита в компонентах R, G и B каждого пикселя.

Обычное LSB-стеганографирование заменяет младший бит (bit_0) в каждом цветовом канале R, G и B. Однако Пётр использует усложнённую модификацию метода, где он заменяет третий значащий бит.

Например, необходимо зашифровать бит сообщения = 0. Имеется цвет пикселя изображения: R = 101. Двоичное представление данного компонента канала: $101 \rightarrow 01100101_2$

1) Читаем текущий bit_2 для замены: $bit_2 = 1$

2) Записываем «0» вместо него, изменяя только данный бит.

Таким образом, до шифрования было: 01100101_2 (= 101). После шифрования станет: 01100001_2 (= 97)

После проведения спектрального и статистического анализа изображения thin_section.png Мария выделила последовательность пикселей, в каждом цветовом канале которых наблюдались отклонения, характерные



для внедрённого стеганографического сообщения. Мария извлекла из этих пикселей их значения каналов RGB, получив следующую последовательность.

Пиксель	R	G	B
1	214	223	96
2	184	81	249
3	151	119	145
4	195	246	241
5	227	196	233
6	5	183	162
7	232	237	118
8	197	177	41
9	119	205	102
10	228	219	253
11	153	254	150
12	201	190	178
13	169	179	83
14	45	199	126
15	130	154	230
16	22	119	100
17	174	245	197
18	232	221	105
19	233	171	156
20	13	179	180
21	171	226	65
22	138	209	165
23	174	211	107
24	126	247	13
25	140	70	186
26	246	92	222
27	161	199	233
28	204	255	100
29	92	71	187
30	204	222	242
31	217	39	145
32	21	237	74
33	114	139	54

Пётр, известный своей любовью к горным треккигам, в файле thin_section.png для последующей защиты дешифрованной битовой последовательности применил специальный «Горный код» стеганографии. Его принцип основан на использовании неравномерного кодирования, где



частым символам соответствуют короткие коды, что минимизирует общий размер секретного сообщения. Дополнительный уровень защиты обеспечивают «мусорные» символы – специальные ловушки, встроенные в код. При расшифровке легитимный получатель должен распознать и отфильтровать этот мусор, чтобы получить исходные данные. Таблица используемых Петром кодов представлена ниже.

Символ	Код	Символ	Код
{	110000	7	111100
}	110001	8	111101
"	110010	9	111110
:	110011	n	000
,	110100	e	001
0	110101	d	010
1	110110	p	0110
2	110111	q	0111
3	111000	x	1000
4	111001	z	1001
5	111010	y	1010
6	111011		

Неравномерный код – это код, в котором кодовые слова, например, последовательности битов, имеют разную длину.

Например, при декодировании битовой последовательности 000001010 с помощью вышепредставленной таблицы, процесс декодирования будет заключаться в следующем:

1) Читаем биты слева-направо: 0 → 00 → 000.

000 есть в таблице? Да, это n.

Результат: n.

2) Продолжаем со следующего бита: 0 → 00 → 001.

001 есть в таблице? Да, это e.

Результат: e.

3) Продолжаем со следующего бита: 0 → 01 → 010.

010 есть в таблице? Да, это d.

Результат: d.

Расшифрованное сообщение n e d.

Файл rock_sample.dat представляет собой последовательность десятичных чисел, каждая из которых – зашифрованная буква передаваемого сообщения. Исходный текст отчёта перед шифрованием был преобразован в числа по правилу: каждый символ кодировался своим порядковым номером в алфавите ($a=1$, $b=2$, ..., $z=33$). Для шифрования Петром использовалась формула: $C_i = M_i^e \bmod n$,

где C_i – i -й элемент последовательности шифротекста C ;

M_i – i -й элемент последовательности шифра M ;



e – открытая экспонента – параметр открытого ключа RSA – число, используемое при шифровании в качестве степени;

n – модуль операции взятия остатка;

mod – операция взятия остатка от деления.

Для дешифрования должна использоваться формула $M_i = C_i^d \text{mod } n$,

где M_i – i -й элемент последовательности шифра M ;

C_i – i -й элемент последовательности шифротекста C ;

d – секретная экспонента – параметр закрытого ключа RSA – число, используемое при дешифровке в качестве степени;

n – модуль операции взятия остатка;

mod – операция взятия остатка от деления.

Совокупность значений (e, n) и (d, n) является открытым и закрытым ключами, используемыми при шифровании и дешифровании соответственно.

Например, RSA-шифрование с открытым ключом: $(e, n) = (17, 3233)$ – и закрытым ключом: $(d, n) = (2753, 3233)$ заключается в следующем.

При шифровании сообщения $M = 65$ отправляемый шифротекст $C = M^e \text{mod } n = 65^{17} \text{mod } 3233 = 2790$

При дешифровке полученного шифротекста $C = 2790$ восстановленное сообщение $M = C^d \text{mod } n = 2790^{2753} \text{mod } 3233 = 65$.

Примечание

Чтобы вычислить выражение вида $M = C^d \text{mod } n$, иногда невыгодно сначала находить большое число C^d , ведь оно может быть огромным. Удобнее использовать алгоритм быстрого возведения в степень по модулю.

Идея алгоритма:

1. Разложить показатель степени на степени двойки.
2. Последовательно возводить число в квадрат.
3. После каждого шага брать остаток по модулю.
4. Умножать только те степени, которые нужны по разложению.

Например, если нужно найти значение выражения $11^5 \text{mod } 9$, то степень 5 можно представить как сумму степеней 4 и 1: $5 = 4 + 1$.

Следовательно:

1. Находим остаток: $11^2 \text{mod } 9 = 4$, т.к. $9 \times 13 = 117$ (остаток 4).

2. Используем найденный остаток, чтобы получить $11^4 \text{mod } 9$.

$11^4 \text{mod } 9 = (11^2)^2 \text{mod } 9 \equiv 4^2 \text{mod } 9 = 7$, т.к. $9 \times 1 = 9$ (остаток 7)

3. Перемножаем найденный остаток с 11 – и снова берём остаток.

$11^5 \text{mod } 9 = (11^4 \times 11) \text{mod } 9 \equiv (7 \times 11) \text{mod } 9 = 77 \text{mod } 9 = 5$, т.к. $9 \times 8 = 72$ (остаток 5)

Таким образом, $11^5 \text{mod } 9 = 5$.

Используя данные, извлечённые из стеганографического сообщения, расшифруйте `rock_sample.dat` и узнайте предварительное название новой горной породы. Содержимое `rock_sample.dat`:

Первое слово – 13 49 63 29 31 27 43 65;



второе слово – 11 43 31 53 10 43 11.

В ответе укажите два слова расшифрованного сообщения в формате «Первое слово» «Второе слово» (например, Медное золото). Для получения дополнительных баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю, используемого при дешифровке.

Решение

На первом этапе необходимо определить секретный ключ, который был внедрен Петром методом стеганографии в файл `thin_section.png`. Для сокрытия информации в изображении `thin_section.png` Пётр применил LSB-метод, производя замену третьего значащего бита в компонентах R, G и B каждого пикселя. Следовательно, необходимо последовательно извлечь биты сначала из красного канала первого пикселя, затем из зеленого, затем из синего, записать в том порядке, в котором они были закодированы, а потом перейти к следующему пикселю.

Каждый цветовой канал пикселя представлен числом от 0 до 255. В связи с тем, что замена третьего значащего бита осуществляется в двоичной записи числа, необходимо перевести значения извлечённой последовательности каналов RGB в двоичную систему, выделить третий бит каждого числа и записать получившуюся последовательность.

Для перевода числа из десятичной системы счисления в двоичную необходимо разделить число в десятичной системе на 2 и записать остаток до тех пор, пока неполное частное не будет равно нулю. Полученные остатки записываются в обратном порядке. В рамках задачи можно ограничиться делением числа до момента нахождения значения третьего значащего бита.

Например, извлечение битов из первого пикселя выглядит следующим образом.

Пиксель	Каналы	Число в 10-ой системе счисления	Перевод в 2-ую систему счисления	Третий значащий бит
1	R	214_{10}	$\begin{array}{l} 1) \frac{214}{2} = 107 \text{ (остаток 0)} \\ 2) \frac{107}{2} = 53 \text{ (остаток 1)} \\ 3) \frac{53}{2} = 26 \text{ (остаток 1)} \end{array}$	1
	G	223_{10}	$\begin{array}{l} 1) \frac{223}{2} = 111 \text{ (остаток 1)} \\ 2) \frac{111}{2} = 55 \text{ (остаток 1)} \\ 3) \frac{55}{2} = 22 \text{ (остаток 1)} \end{array}$	1
	B	96_{10}	$\begin{array}{l} 1) \frac{96}{2} = 48 \text{ (остаток 0)} \\ 2) \frac{48}{2} = 24 \text{ (остаток 0)} \\ 3) \frac{24}{2} = 12 \text{ (остаток 0)} \end{array}$	0



Руководствуясь аналогичным алгоритмом, получаем следующие значения.

Пиксель/ Каналы	Число в 10-ой системе счисления			Третий значащий бит		
	R	G	B	R	G	B
1	214	223	96	1	1	0
2	184	81	249	0	0	0
3	151	119	145	1	1	0
4	195	246	241	0	1	0
5	227	196	233	0	1	0
6	5	183	162	1	1	0
7	232	237	118	0	1	1
8	197	177	41	1	0	0
9	119	205	102	1	1	1
10	228	219	253	1	0	1
11	153	254	150	0	1	1
12	201	190	178	0	1	0
13	169	179	83	0	0	0
14	45	199	126	1	1	1
15	130	154	230	0	0	1
16	22	119	100	1	1	1
17	174	245	197	1	1	1
18	232	221	105	0	1	0
19	233	171	156	0	0	1
20	13	179	180	1	0	1
21	171	226	65	0	0	0
22	138	209	165	0	0	1
23	174	211	107	1	0	0
24	126	247	13	1	1	1
25	140	70	186	1	1	0
26	246	92	222	1	1	1
27	161	199	233	0	1	0
28	204	255	100	1	1	1
29	92	71	187	1	1	0
30	204	222	242	1	1	0
31	217	39	145	0	1	0
32	21	237	74	1	1	0
33	114	139	54	0	0	1

Таким образом, в результате дешифровки пикселей изображения получена следующая битовая строка:

1100001100100101100111001111010110100001110011111110100011010
00001100111110111010111110110010110001

В задаче сказано, что Пётр использовал для шифрования специальный вод, который основан на использовании неравномерного кодирования. Алгоритм дешифровки в данном случае заключается в следующем: читаем



строку по битам слева направо, пошагово накапливая последовательность, пока она не совпадёт с одним из кодов в таблице. Как только совпадение найдено, записываем символ и начинаем формировать следующую последовательность заново.

Начнём процесс декодирования. Если код не найден сразу (например, при чтении первой 1), то показан процесс накопления последовательности: $1 \rightarrow 11 \rightarrow 110$ и т.д., до момента совпадения с записью в таблице кодов, используемых Петром при шифровании. Максимальная длина кодового слова в таблице кодировки – 6 бит (например, 110000, 110011, 111110 и т.д.), поэтому в процессе накопления битовой последовательности последовательности длиннее 6 символов не рассматриваются.

Шаг	Индекс начала	Биты (извлечённые)	Найденный код	Символ
1	0	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11000 \rightarrow 110000$	110000	{
2	6	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110010$	110010	"
3	12	$0 \rightarrow 01 \rightarrow 010$	010	d
4	15	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
5	21	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1001$	1001	z
6	25	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 11101 \rightarrow 111010$	111010	5
7	31	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1101 \rightarrow 11010 \rightarrow 110100$	110100	,
8	37	$0 \rightarrow 00 \rightarrow 001$	001	e
9	40	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
10	46	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1111 \rightarrow 11111 \rightarrow 111110$	111110	9
11	52	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1000$	1000	x
12	56	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1101 \rightarrow 11010 \rightarrow 110100$	110100	,
13	62	$0 \rightarrow 00 \rightarrow 000$	000	n
14	65	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
15	71	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 11101 \rightarrow 111011$	111011	6
16	77	$1 \rightarrow 10 \rightarrow 101 \rightarrow 1010$	1010	y
17	81	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1111 \rightarrow 11111 \rightarrow 111110$	111110	9
18	87	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 110010$	110010	"
19	93	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 110001$	110001	}

Собирая полученные символы в порядке их дешифровки, получаем точную расшифровку битовой строки: {"d:z5,e:5x,n:6y9"}. В условии задачи говорится про «мусорные символы» – ловушки, которые Пётр вставляет в сообщение. В таблице кодов есть набор букв-«включений» x, y, z (и т.д.), и они выступают как мусор в конкретном сообщении: они появляются внутри ключей/значений, но не несут смысла для конечного JSON.

Если удалить все мусорные буквы x, y, z из полученной строки {"d:z5,e:5x,n:6y9"}, то получим корректную JSON строку: {"d:5,e:5,n:69"}. Следовательно, получены данные для расшифровки текстового сообщения из файла rock_sample.dat, а именно модуль $n=69$ и секретная экспонента $d=5$.

Файл rock_sample.dat содержит две последовательности целых чисел, где каждое число – зашифрованный символ шифротекста:

- Первое слово: 13 49 63 29 31 27 43 65



• Второе слово: 11 43 31 53 10 43 11 Из условия задачи воспользуемся формулой дешифрования: $M_i = C_i^d \bmod n$,

где M_i – i -й элемент последовательности шифра M (восстановленный номер буквы ($1 = a, 2 = б, \dots, 33 = я$));

C_i – i -й элемент последовательности шифротекста C ;

d – секретная экспонента;

n – модуль операции взятия остатка;

$\bmod$ – операция взятия остатка от деления.

Дешифруем первое слово: $C = [13 \ 49 \ 63 \ 29 \ 31 \ 27 \ 43 \ 65]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 13$

$$13^2 = 169 \rightarrow 169 \bmod 69 = 31, \text{ т. к. } 69 \times 2 = 138 \text{ (остаток 31)}$$

Таким образом, $13^2 \equiv 31 \pmod{69}$

$$13^4 = (13^2)^2 \equiv 31^2 = 961 \rightarrow 961 \bmod 69 = 64, \text{ т. к. } 69 \times 13 = 897 \text{ (остаток 64)}$$

Таким образом, $13^4 \equiv 64 \pmod{69}$

$$13^5 = 13^4 \times 13 \equiv 64 \times 13 = 832 \rightarrow 832 \bmod 69 = 4, \text{ т. к. } 69 \times 12 = 828 \text{ (остаток 4)}$$

Таким образом, $M_1 = 13^5 \pmod{69} = 4$

Следовательно, $M_1 = 4$, что соответствует букве алфавита «г».

2. $C_2 = 49$

$$49^2 = 2401 \rightarrow 2401 \bmod 69 = 55, \text{ т. к. } 69 \times 34 = 2346 \text{ (остаток 55)}$$

Таким образом, $49^2 \equiv 55 \pmod{69}$

$$49^4 = (49^2)^2 \equiv 55^2 = 3025 \rightarrow 3025 \bmod 69 = 58, \text{ т. к. } 69 \times 43 = 2967 \text{ (остаток 58)}$$

Таким образом, $49^4 \equiv 58 \pmod{69}$

$$49^5 = 49^4 \times 49 \equiv 58 \times 49 = 2842 \rightarrow 2842 \bmod 69 = 13, \text{ т. к. } 69 \times 41 = 2829 \text{ (остаток 13)}$$

Таким образом, $M_2 = 49^5 \pmod{69} = 13$

Следовательно, $M_2 = 13$, что соответствует букве алфавита «л».

3. $C_3 = 63$

$$63^2 = 3969 \rightarrow 3969 \bmod 69 = 36, \text{ т. к. } 69 \times 57 = 3933 \text{ (остаток 36)}$$

Таким образом, $63^2 \equiv 36 \pmod{69}$

$$63^4 = (63^2)^2 \equiv 36^2 = 1296 \rightarrow 1296 \bmod 69 = 54, \text{ т. к. } 69 \times 18 = 1242 \text{ (остаток 54)}$$

Таким образом, $63^4 \equiv 54 \pmod{69}$

$$63^5 = 63^4 \times 63 \equiv 54 \times 63 = 3402 \rightarrow 3402 \bmod 69 = 21, \text{ т. к. } 69 \times 49 = 3381 \text{ (остаток 21)}$$

Таким образом, $M_3 = 63^5 \pmod{69} = 21$

Следовательно, $M_3 = 21$, что соответствует букве алфавита «у».

4. $C_4 = 29$

$$29^2 = 841 \rightarrow 841 \bmod 69 = 13, \text{ т. к. } 69 \times 12 = 828 \text{ (остаток 13)}$$

Таким образом, $29^2 \equiv 13 \pmod{69}$



$$29^4 = (29^2)^2 \equiv 13^2 = 169 \rightarrow 169 \bmod 69 = 31, \text{ т. к. } 69 \times 2 = 138 \text{ (остаток 31)}$$

Таким образом, $29^4 \equiv 31 \pmod{69}$

$$29^5 = 29^4 \times 29 \equiv 31 \times 29 = 899 \rightarrow 899 \bmod 69 = 2, \text{ т. к. } 69 \times 13 = 897 \text{ (остаток 2)}$$

Таким образом, $M_4 = 29^5 \pmod{69} = 2$

Следовательно, $M_4 = 2$, что соответствует букве алфавита «б».

5. $C_5 = 31$

$$31^2 = 961 \rightarrow 961 \bmod 69 = 64, \text{ т. к. } 69 \times 13 = 897 \text{ (остаток 64)}$$

Таким образом, $31^2 \equiv 64 \pmod{69}$

$$31^4 = (31^2)^2 \equiv 64^2 = 4096 \rightarrow 4096 \bmod 69 = 25, \text{ т. к. } 69 \times 59 = 4071 \text{ (остаток 25)}$$

Таким образом, $31^4 \equiv 25 \pmod{69}$

$$31^5 = 31^4 \times 31 \equiv 25 \times 31 = 775 \rightarrow 775 \bmod 69 = 16, \text{ т. к. } 69 \times 11 = 759 \text{ (остаток 16)}$$

Таким образом, $M_5 = 31^5 \pmod{69} = 16$

Следовательно, $M_5 = 16$, что соответствует букве алфавита «о».

6. $C_6 = 27$

$$27^2 = 2401 \rightarrow 2401 \bmod 69 = 31, \text{ т. к. } 69 \times 34 = 2346 \text{ (остаток 55)}$$

Таким образом, $27^2 \equiv 55 \pmod{69}$

$$27^4 = (27^2)^2 \equiv 55^2 = 3025 \rightarrow 3025 \bmod 69 = 58, \text{ т. к. } 69 \times 43 = 2967 \text{ (остаток 58)}$$

Таким образом, $27^4 \equiv 58 \pmod{69}$

$$27^5 = 27^4 \times 27 \equiv 58 \times 27 = 2842 \rightarrow 2842 \bmod 69 = 13, \text{ т. к. } 69 \times 41 = 2829 \text{ (остаток 13)}$$

Таким образом, $M_6 = 27^5 \pmod{69} = 13$

Следовательно, $M_6 = 13$, что соответствует букве алфавита «к».

7. $C_7 = 43$

$$43^2 = 1849 \rightarrow 1849 \bmod 69 = 55, \text{ т. к. } 69 \times 26 = 1794 \text{ (остаток 55)}$$

Таким образом, $43^2 \equiv 55 \pmod{69}$

$$43^4 = (43^2)^2 \equiv 55^2 = 3025 \rightarrow 3025 \bmod 69 = 58, \text{ т. к. } 69 \times 43 = 2967 \text{ (остаток 58)}$$

Таким образом, $43^4 \equiv 58 \pmod{69}$

$$43^5 = 43^4 \times 43 \equiv 58 \times 43 = 2494 \rightarrow 2494 \bmod 69 = 10, \text{ т. к. } 69 \times 36 = 2484 \text{ (остаток 10)}$$

Таким образом, $M_7 = 43^5 \pmod{69} = 10$

Следовательно, $M_7 = 10$, что соответствует букве алфавита «и».

8. $C_8 = 65$

$$65^2 = 4225 \rightarrow 4225 \bmod 69 = 16, \text{ т. к. } 69 \times 61 = 4209 \text{ (остаток 16)}$$

Таким образом, $65^2 \equiv 16 \pmod{69}$

$$65^4 = (65^2)^2 \equiv 16^2 = 256 \rightarrow 256 \bmod 69 = 49, \text{ т. к. } 69 \times 3 = 207 \text{ (остаток 49)}$$

Таким образом, $65^4 \equiv 49 \pmod{69}$

$$65^5 = 65^4 \times 65 \equiv 49 \times 65 = 3185 \rightarrow 3185 \bmod 69 = 11, \text{ т. к. } 69 \times 46 = 3174 \text{ (остаток 11)}$$

Таким образом, $M_8 = 65^5 \pmod{69} = 11$

Следовательно, $M_8 = 11$, что соответствует букве алфавита «й».



Соединив расшифрованные буквы, получаем слово «глубокий».

Дешифруем второе слово: $C = [11 \ 43 \ 31 \ 53 \ 10 \ 43 \ 11]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 11$

$$11^2 = 121 \rightarrow 121 \bmod 69 = 52, \text{ т. к. } 69 \times 1 = 69 \text{ (остаток 52)}$$

$$\text{Таким образом, } 11^2 \equiv 52 \pmod{69}$$

$$11^4 = (11^2)^2 \equiv 52^2 = 2704 \rightarrow 2704 \bmod 69 = 13, \text{ т. к. } 69 \times 39 = 2691 \text{ (остаток 13)}$$

$$\text{Таким образом, } 11^4 \equiv 13 \pmod{69}$$

$$11^5 = 11^4 \times 11 \equiv 13 \times 11 = 143 \rightarrow 143 \bmod 69 = 5, \text{ т. к. } 69 \times 2 = 138 \text{ (остаток 5)}$$

$$\text{Таким образом, } M_1 = 11^5 \pmod{69} = 5$$

Следовательно, $M_1 = 5$, что соответствует букве алфавита «д».

2. $C_2 = 43$

Аналогично C_7 для первого слова, $M_7 = M_2 = 10$, что соответствует букве алфавита «и».

3. $C_3 = 31$

Аналогично C_5 для первого слова, $M_5 = M_3 = 16$, что соответствует букве алфавита «о».

4. $C_4 = 53$

$$53^2 = 625 \rightarrow 625 \bmod 69 = 49, \text{ т. к. } 69 \times 40 = 2760 \text{ (остаток 49)}$$

$$\text{Таким образом, } 53^2 \equiv 49 \pmod{69}$$

$$53^4 = (53^2)^2 \equiv 49^2 = 2401 \rightarrow 2401 \bmod 69 = 55, \text{ т. к. } 69 \times 34 = 2346 \text{ (остаток 55)}$$

$$\text{Таким образом, } 53^4 \equiv 55 \pmod{69}$$

$$53^5 = 53^4 \times 53 \equiv 55 \times 53 = 2915 \rightarrow 2915 \bmod 69 = 17, \text{ т. к. } 69 \times 42 = 2898 \text{ (остаток 17)}$$

$$\text{Таким образом, } M_4 = 53^5 \pmod{69} = 17$$

Следовательно, $M_4 = 17$, что соответствует букве алфавита «п».

5. $C_5 = 10$

$$10^2 = 100 \rightarrow 100 \bmod 69 = 31, \text{ т. к. } 69 \times 1 = 69 \text{ (остаток 31)}$$

$$\text{Таким образом, } 10^2 \equiv 31 \pmod{69}$$

$$10^4 = (10^2)^2 \equiv 31^2 = 961 \rightarrow 961 \bmod 69 = 64, \text{ т. к. } 69 \times 13 = 897 \text{ (остаток 64)}$$

$$\text{Таким образом, } 10^4 \equiv 64 \pmod{69}$$

$$10^5 = 10^4 \times 10 \equiv 64 \times 10 = 640 \rightarrow 640 \bmod 69 = 19, \text{ т. к. } 69 \times 9 = 621 \text{ (остаток 19)}$$

$$\text{Таким образом, } M_5 = 10^5 \pmod{69} = 19$$

Следовательно, $M_5 = 19$, что соответствует букве алфавита «с».

6. $C_6 = 43$



Аналогично C_7 для первого слова, $M_6 = M_2 = 10$, что соответствует букве алфавита «и».

7. $C_7 = 31$

Аналогично C_1 , $M_1 = M_7 = 5$, что соответствует букве алфавита «д».

Соединив расшифрованные буквы, получаем слово «диопсид».

В ответе сказано указать два слова расшифрованного сообщения в формате «Первое слово» «Второе слово», соединяя получаем глубокий диопсид.

Ответ: глубокий диопсид

Для получения максимального количества баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю при дешифровке.

C++

```
#include <iostream>

int mod_exp(int base, int exp, int mod) {
    int result = 1;
    while (exp > 0) {
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

int main() {
    int base = 33;
    int exp = 5;
    int mod = 46;

    int result = mod_exp(base, exp, mod);
    std::cout << result << std::endl;

    return 0;
}
```

Python

```
def mod_exp(base, exp, mod):
    result = 1
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp //= 2
    return result

base = 12
exp = 5
mod = 46
result = mod_exp(base, exp, mod)
print(result)
```

**Задача 5 (20 баллов)****Космический зонд на ледяной комете**

На длинной ледяной комете расположено $n \geq 2$ научно-исследовательских станций вдоль числовой оси с целочисленными координатами $b_1, b_2, \dots, b_n$, упорядоченными по возрастанию, где $b_1 = 0$. Космический зонд начинает свою миссию на станции b_1 и должен достичь финальной станции b_n , используя адаптивную систему навигации.

Принцип работы навигационной системы

Зонд использует двухфазную систему перемещения. Сначала происходит **калибровка**: зонд определяет ближайшую станцию впереди (обозначим её b_j), вычисляет эталонное расстояние D как разность между координатой этой станции и текущей позицией зонда, затем телепортируется на станцию b_j , фиксируя найденное значение D как текущий масштаб перемещения.

После калибровки начинается **поиск целевого расстояния**: зонд последовательно сканирует позиции впереди себя (текущая позиция + 1, текущая позиция + 2, текущая позиция + 3 и так далее). Когда зонд обнаруживает станцию на некоторой позиции X , возможны три сценария. Если X равно b_n , то зонд перемещается на нее и успешно завершает миссию. Если на позиции $(X - D)$ также существует станция, то зонд перемещается на станцию X и выполняет новую калибровку в этой точке. В противном случае зонд продолжает сканирование позиций дальше.

Формат входных данных

В первой строке задано целое число n — количество станций, различных целочисленных точек ($2 \leq n \leq 300000$). Во второй строке через пробел записаны n целых чисел $b_1, b_2, \dots, b_n$ — координаты станций в порядке возрастания. Самая левая из них имеет значение 0, самая правая не превосходит 10^9 .

Формат выходных данных

Требуется вывести одно целое число D — итоговое эталонное расстояние, при котором зонд достигает финальной станции b_n .

Примеры

стандартный ввод	стандартный вывод
7 0 3 4 5 6 8 13	2
9 0 3 4 5 6 8 9 10 13	3
12 0 3 4 5 7 8 11 12 14 17 18 24	2

Пояснения к примерам



Иллюстрация к первому примеру.

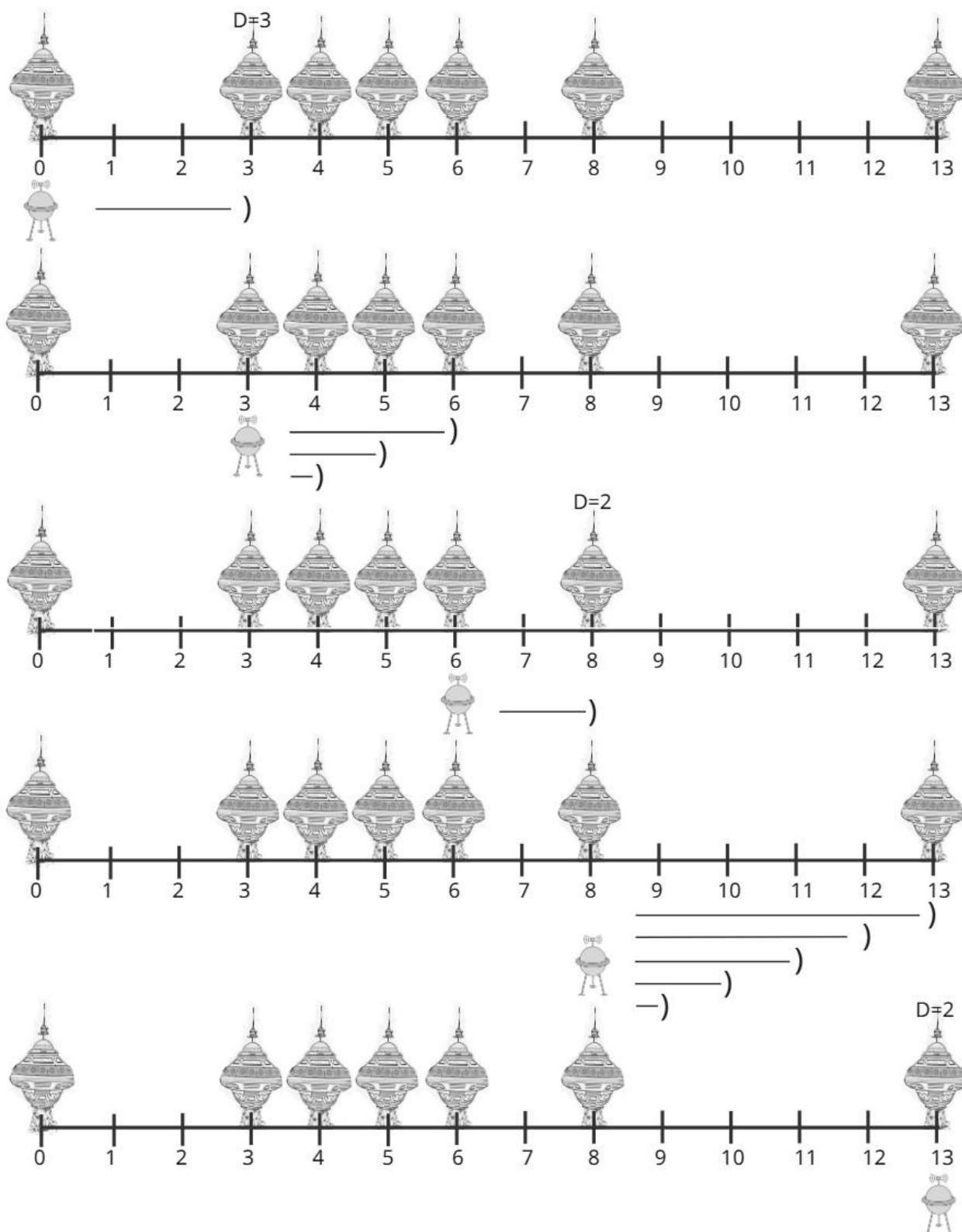
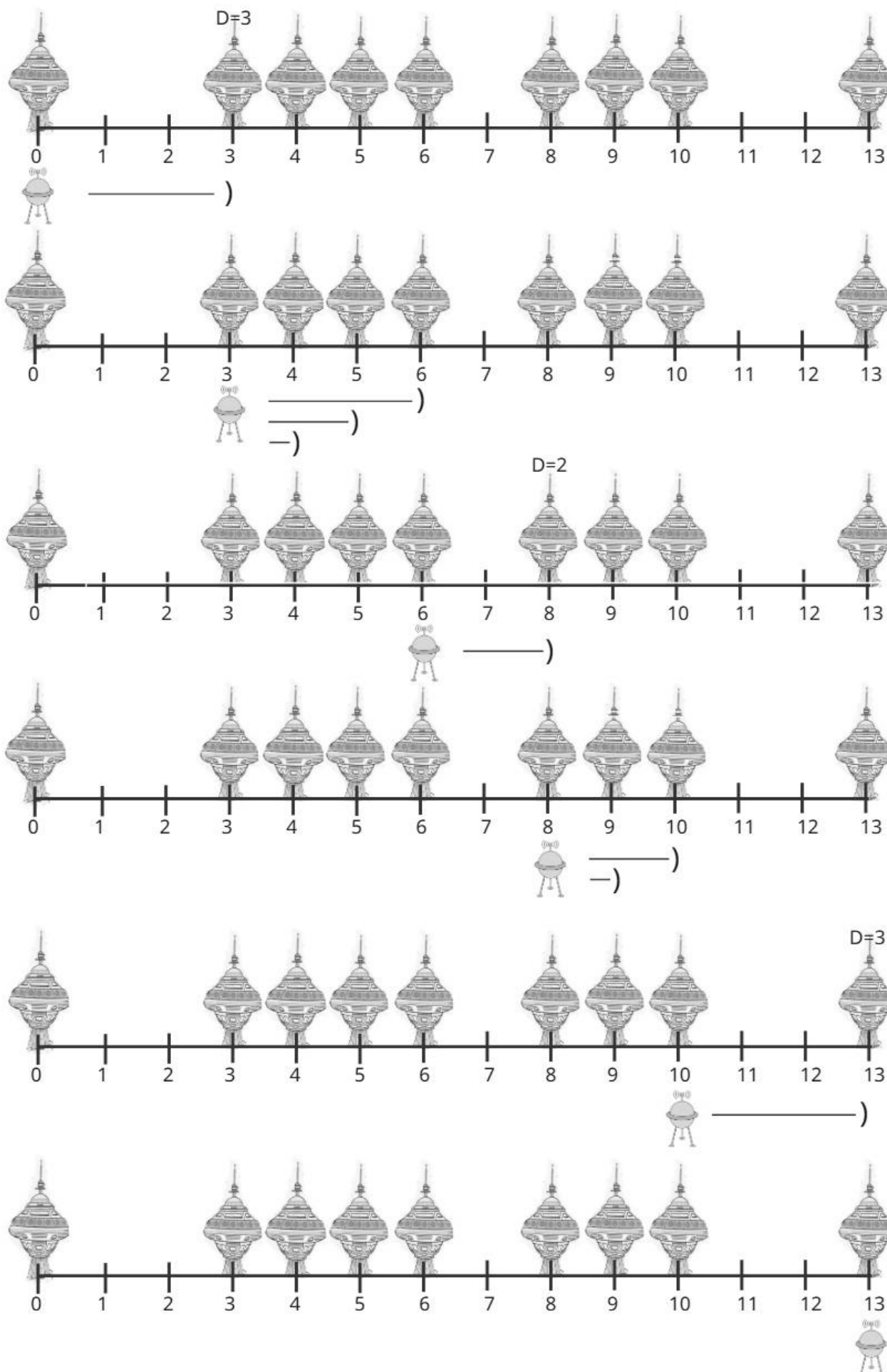


Иллюстрация ко второму примеру.





Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, обязательно **добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных, поясните решение. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
12 0 3 4 5 6 7 9 10 12 13 14 24	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

Смотрите решение в первом варианте.



Задача 6 (30 баллов)

Источник вечной энергии

Известный изобретатель Иван Иванович Иванов разработал принципиально новую установку, позволяющую генерировать сколько угодно безопасной энергии. Он поместил в точки прямой **0** и **D** два направленных друг на друга источника специально заряженных частиц. В момент времени **0** источник с координатой **0** генерирует **n** частиц, которые движутся по прямой в сторону второго генератора. Каждая из них имеет свою скорость v_i , скорости всех частиц попарно различны. Как только какая-то частица достигает источника с координатой **D**, она им поглощается и этот источник выпускает ровно такую же по скорости частицу в противоположном направлении. Эта частица движется обратно в сторону начала координат и снова поглощается источником с координатой **0**, после чего процесс испускания частицы в сторону второго источника снова повторяется.

Известно, что как только две частицы попадают в одну и ту же точку прямой, они генерируют одну единицу энергии. Если в одной точке оказываются несколько частиц, каждая пара среди встретившихся генерирует единицу энергии, в частности, в момент времени **0**, когда **n** частиц стартуют из точки **0**, они уже создают $\frac{n(n-1)}{2}$ единиц энергии. Если две частицы одновременно достигают какого-то из источников, то они генерируют одну единицу перед поглощением, и одну единицу уже после возрождения их двойников, движущихся в обратном направлении. За счет множественности встреч, процесс работы источников имеет большой положительный баланс по выработке энергии. Энергия может вырабатываться сколь угодно долго.

Теперь, для написания научной статьи про опыт использования источника вечной энергии, Иван Иванович хочет точно знать момент, когда будет выработана **m**-я по счету единица энергии.

Формат входных данных

В первой строке ввода через пробел содержатся два целых числа **D** и **n** ($0 \leq D \leq 200$, $2 \leq n \leq 2000$). Во второй строке содержатся **n** целых чисел v_i через пробел – скорости частиц ($1 \leq v_i \leq 2000$). Скорости даны в порядке строгого возрастания, то есть $v_1 < v_2 < \dots < v_n$.

В третьей строке содержится одно целое число **m** – номер требуемой по порядку единицы энергии, для которой следует найти точный момент её генерации ($1 \leq m \leq 10^9$).

Формат выходных данных

Вывести в ответ одно вещественное число – время, когда будет сгенерирована **m**-я по порядку единица энергии. Ответ вывести с точностью не менее 2 знаков после десятичной точки.

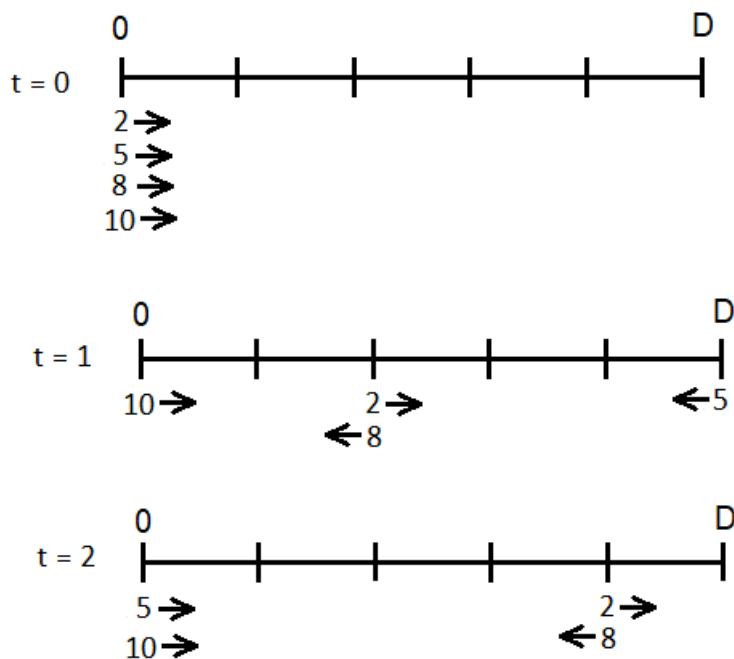


Примеры

стандартный ввод	стандартный вывод
5 4 2 5 8 10 6	0.00
5 4 2 5 8 10 7	0.56
5 4 2 5 8 10 11	1.00
5 4 2 5 8 10 17	1.67
5 4 2 5 8 10 19	1.67
5 4 2 5 8 10 21	2.00

Пояснения к примерам

На следующих рисунках приведено расположение частиц из примеров в моменты времени 0, 1 и 2. Частицы обозначены своими скоростями. Стрелками обозначены направления их движения в соответствующий момент. Если в этот момент частица была регенерирована, она имеет новое направление движения.



Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите



эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, **обязательно добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
5 4 2 5 8 10 16	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

Смотрите решение в первом варианте.

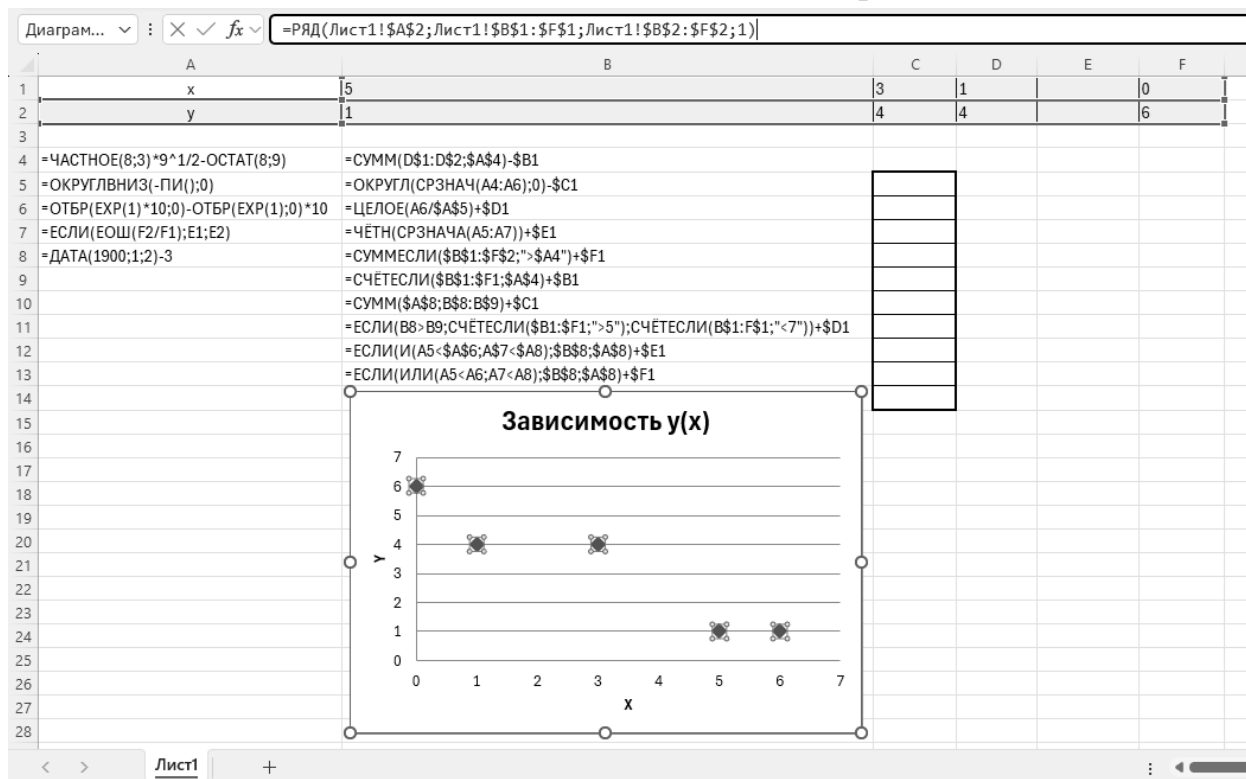


ВАРИАНТ 3

Задача 1 (5 баллов)

Разведчики

От разведчиков были получены сведения, закодированные в виде электронной таблицы. Определите числа в ячейках C5:C14, которые будут получены в результате копирования в них ячеек B4:B13. Также определите недостающие числа в блоке B1:F2 на основании диаграммы.



В ответе записать, в одну строчку через запятую, значения ячеек C5:C14, начиная с ячейки C5 и заканчивая ячейкой C14. Формат ячеек в таблице – «Общий».

Решение

На основании диаграммы определяем, что в ячейках A1:F2 не отображена точка (x=6, y=1) расположенная в ячейках E1:E2. Далее копируем формулы, расположенные в ячейках B4:B13 в ячейки C5:C14. В результате получаем следующие формулы:



	C	D
5	=СУММ(E\$1:E\$2;\$A\$4)-\$B2	
6	=ОКРУГЛ(СРЗНАЧ(B5:B7);0)-\$C2	
7	=ЦЕЛОЕ(B7/\$A\$5)+\$D2	
8	=ЧЁТН(СРЗНАЧА(B6:B8))+\$E2	
9	=СУММЕСЛИ(\$B\$1:\$F\$2;">\$A4")+ \$F2	
10	=СЧЁТЕСЛИ(\$B\$1:\$F2;\$A\$4)+\$B2	
11	=СУММ(\$A\$8;C\$8:C\$9)+\$C2	
12	=ЕСЛИ(C9>C10;СЧЁТЕСЛИ(\$B2:\$F2;">5");СЧЁТЕСЛИ(C\$1:G\$1;"<7"))+\$D2	
13	=ЕСЛИ(И(B6<\$A\$6;B\$7<\$A9);\$B\$8;\$A\$8)+\$E2	
14	=ЕСЛИ(ИЛИ(B6<B7;B8<B9);\$B\$8;\$A\$8)+\$F2	
15		

Вычисляем значения по формулам в ячейках A4:A8, затем переходим к вычислениям значений в ячейках B4:B13 и, наконец, в ячейках C5:C14. Получаем следующие результаты:

	A	B	C	D	E	F	G	H	I	J
1	x	5	3	1	6	0				
2	y	1	4	4	1	6				
3										
4	1	1								
5	-3	-1	7							
6	7	-2	-2							
7	6	10	0							
8	-1	0	5							
9		6	6							
10		8	4							
11		6	14							
12		5	5							
13		0	0							
14			6							
15										
16										
17										
18										
19										

Зависимость y(x)

Ответ: 7, -2, 0, 5, 6, 4, 14, 5, 0, 6

**Задача 2 (10 баллов)****Лаборатория кибербезопасности**

Алиса работает в отделе кибербезопасности и тестирует новый алгоритм фильтрации данных. Логическая функция F описывает условия допуска к защищённым файлам:

$$F = x \rightarrow (z \vee y \wedge w) \equiv (w \wedge z \rightarrow x)$$

Алиса составила таблицу истинности, но для проверки знаний своих коллег зашифровала некоторые значения логическими выражениями, в которых a, b, c – следующие высказывания:

a – «1 Кбайт = 1024 МБайт»

b – «27 делится на 3 без остатка»

c – « $7 \times 7 = 49$ »

Определите, какому столбцу таблицы истинности функции F соответствует каждая из переменных w, x, y, z .

?	?	?	?	F
	$\neg b \wedge c \wedge a$	$\neg b \rightarrow c \wedge a$		0
$a \rightarrow b \rightarrow c$		$b \wedge a \wedge c$	$\neg a \rightarrow b \rightarrow \neg c$	0
$\neg a \rightarrow b \vee c$		$\neg a \rightarrow b \wedge \neg c$	$a \rightarrow b \wedge \neg c$	0
	$\neg b \wedge a \vee c$	$b \wedge \neg a \vee c$		0

В ответе напишите буквы w, x, y, z в том порядке, в котором идут соответствующие им столбцы.

Решение

Определим значения логических высказываний. Таким образом, получим следующее:

$$a=0$$

$$b=1$$

$$c=1$$

Теперь определим значения логических выражений в таблице. В результате получим следующую таблицу:

?	?	?	?	F
	0	1		0
1		0	0	0
1		0	1	0
	1	1		0

Получаем следующую таблицу истинности:



w	x	y	z	$y \wedge w$	$z \vee y \wedge w$	$x \rightarrow (z \vee y \wedge w)$	$w \wedge z$	$w \wedge z \rightarrow x$	F
0	0	0	0	0	0	1	0	1	1
0	0	0	1	0	1	1	0	1	1
0	0	1	0	0	0	1	0	1	1
0	0	1	1	0	1	1	0	1	1
0	1	0	0	0	0	0	0	1	0
0	1	0	1	0	1	1	0	1	1
0	1	1	0	0	0	0	0	1	0
0	1	1	1	0	1	1	0	1	1
1	0	0	0	0	0	1	0	1	1
1	0	0	1	0	1	1	1	0	0
1	0	1	0	1	1	1	0	1	1
1	0	1	1	1	1	1	1	0	0
1	1	0	0	0	0	0	0	1	0
1	1	0	1	0	1	1	1	1	1
1	1	1	0	1	1	1	0	1	1
1	1	1	1	1	1	1	1	1	1

Выберем из нее строки, где функция $F = 0$.

w	x	y	z	F
0	1	0	0	0
0	1	1	0	0
1	0	0	1	0
1	0	1	1	0
1	1	0	0	0

Сравнивая эту таблицу с заданной в задании, и учитывая, что строки в этой таблице выписаны в случайном порядке, установим искомые наименования столбцов:

z	w	x	y	F
0	0	1	1	0
1	1	0	0	0
1	1	0	1	0
0	1	1	0	0

Ответ: x, z, y, w

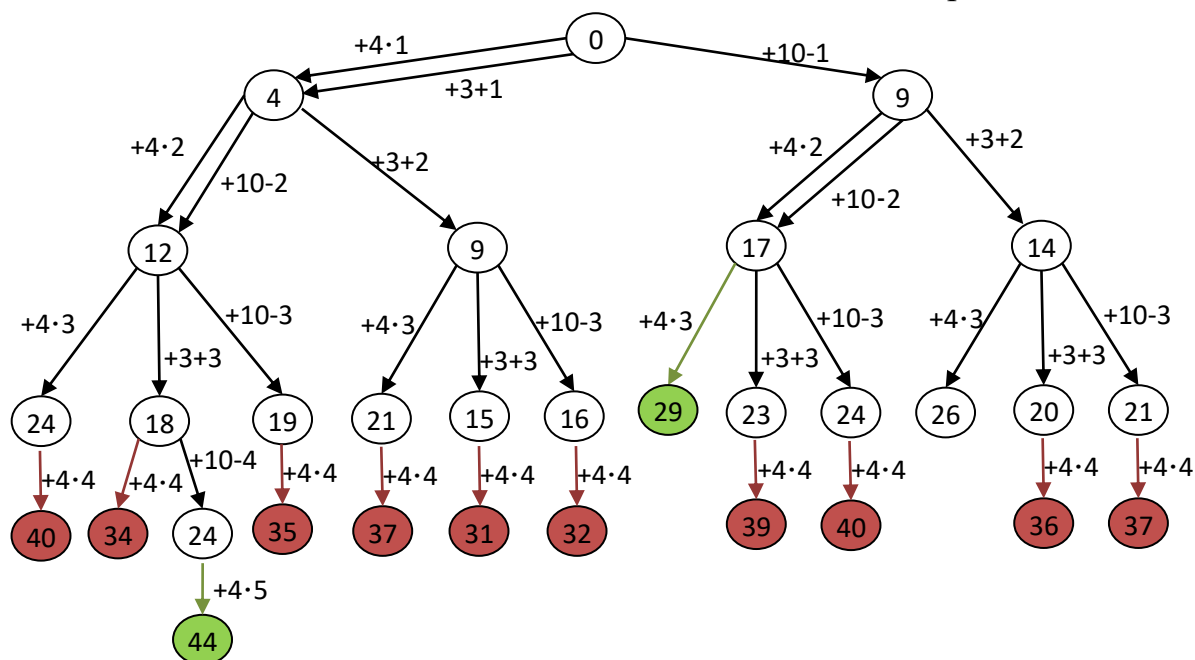
**Задача 3 (15 баллов)****Азартные игроки**

Два азартных игрока решили сыграть в следующую игру. По очереди они складывают игральные карты в колоду. За один ход можно положить в колоду количество карт, определяемое формулой: либо 4 умноженное на номер хода в игре, либо 3 плюс номер хода в игре, либо 10 минус номер хода в игре. При достижении или превышении количества 25 карт в колоде, игра заканчивается. Кто первым, сделав ход, получит в колоде 25 карт или больше – считается победителем.

Кто выигрывает при безошибочной игре – игрок, делающий первый ход, или игрок, делающий второй ход? Каково минимальное число ходов до победы, если выигрывающий игрок пытается закончить игру за меньшее число ходов, а проигрывающий пытается максимально задержать свое поражение? Какое максимальное число карт может оказаться в колоде в конце игры? Какой игрок при этом победит? Поясните алгоритм графически, используя ориентированные графы в виде деревьев, с обозначением выигрышных и проигрышных позиций как вершин, а действий игроков – дугами графа с указанием численных изменений.

Решение

Построим ориентированный граф, где зеленым цветом выделены выигрышные позиции первого игрока, красным цветом – выигрышные позиции второго игрока, а номер соответствует количеству карт в колоде. Рядом со стрелкой указано изменение числа карт в колоде на этом ходу. В случае, если один из вариантов хода приводит к победе, то второй вариант не отображается, за исключением ветви, которая демонстрирует игру с максимально возможным числом пачек на полке в конце игры.





Из графа видно, что при любой игре второго игрока, гарантированно победит первый игрок на 3 ходу, если первым ходом положит в колоду 9 карт. Максимально возможное число карт можно получить, если увеличивать количество карт в колоде таким образом, чтобы получить как можно более высокий номер хода (5) и количества карт в колоде (24) перед последним ходом, в котором берется вариант с большим значением (+4·5). При этом побеждает первый игрок с количеством карт 44.

Ответ: гарантированно победит первый игрок за 3 хода, максимум 44 карты при победе первого игрока.



Задача 4 (20 баллов)

Аномальный криокамень

В ходе арктической экспедиции обнаружена горная порода с аномальной кристаллической решеткой, демонстрирующая свойства высокотемпературной сверхпроводимости. Старший геолог Пётр, опасаясь промышленного шпионажа, передал коллеге Марии, используя многоуровневую систему защиты, зашифрованные данные, содержащие предварительное название породы из двух слов.

Мария получила два файла:

- 1) rock_sample.dat – файл с основным зашифрованным отчётом;
- 2) thin_section.png – микрофотография тонкого среза породы, в которую Петром был внедрён секретный ключ методом стеганографии, имитирующим природные включения. Микрофотография представляет собой 8-битное RGB-изображение.

Алгоритм действий для Марии выглядит следующим образом:

1) Извлечь скрытые данные из изображения thin_section.png. Для этого необходимо проанализировать третьи значащие биты цветовых каналов пикселей, записать их последовательно в том порядке, в котором они были закодированы, и получить битовую последовательность.

2) Эту битовую последовательность необходимо декодировать. В результате она должна превратиться в строку в формате JSON, которая и содержит секретные ключи для алгоритма RSA. В данной задаче строка в формате JSON выглядит следующим образом: {ключ1: значение1, ключ2: значение2, ключ3: значение3}.

3) Используя полученные ключи, можно приступить к дешифровке основного отчёта, хранящегося в файле rock_sample.dat.

Для сокрытия информации в изображении thin_section.png Пётр применил LSB-метод, производя замену третьего значащего младшего бита в компонентах R, G и B каждого пикселя.

Обычное LSB-стеганографирование заменяет младший бит (bit_0) в каждом цветовом канале R, G и B. Однако Пётр использует усложнённую модификацию метода, где он заменяет третий значащий бит.

Например, необходимо зашифровать бит сообщения = 0. Имеется цвет пикселя изображения: $R = 101$. Двоичное представление данного компонента канала: $101 \rightarrow 01100101_2$

1) Читаем текущий bit_2 для замены: $\text{bit}_2 = 1$

2) Записываем «0» вместо него, изменяя только данный бит.

Таким образом, до шифрования было: 01100101_2 (= 101). После шифрования станет: 01100001_2 (= 97)

После проведения спектрального и статистического анализа изображения thin_section.png Мария выделила последовательность пикселей,



в каждом цветовом канале которых наблюдались отклонения, характерные для внедрённого стеганографического сообщения. Мария извлекла из этих пикселей их значения каналов RGB, получив следующую последовательность.

Пиксель	R	G	B
1	79	149	232
2	211	48	194
3	102	165	243
4	73	221	113
5	163	139	68
6	70	39	122
7	104	229	38
8	14	211	170
9	171	126	188
10	52	204	214
11	90	197	31
12	130	236	195
13	107	160	136
14	187	157	62
15	123	194	221
16	247	189	142
17	102	83	19
18	94	133	226
19	177	54	117
20	44	95	249
21	158	134	116
22	110	251	236
23	201	218	32
24	28	145	118
25	98	186	43
26	143	180	240
27	65	189	205
28	191	157	140
29	233	149	88
30	254	182	120
31	58	62	98
32	149	116	226
33	152	208	76

Пётр, известный своей любовью к горным треккигам, в файле thin_section.png для последующей защиты дешифрованной битовой последовательности применил специальный «Горный код» стеганографии.



Его принцип основан на использовании неравномерного кодирования, где частым символам соответствуют короткие коды, что минимизирует общий размер секретного сообщения. Дополнительный уровень защиты обеспечивают «мусорные» символы – специальные ловушки, встроенные в код. При расшифровке легитимный получатель должен распознать и отфильтровать этот мусор, чтобы получить исходные данные. Таблица используемых Петром кодов представлена ниже.

Символ	Код	Символ	Код
{	110000	7	111100
}	110001	8	111101
"	110010	9	111110
:	110011	n	000
,	110100	e	001
0	110101	d	010
1	110110	p	0110
2	110111	q	0111
3	111000	x	1000
4	111001	z	1001
5	111010	y	1010
6	111011		

Неравномерный код – это код, в котором кодовые слова, например, последовательности битов, имеют разную длину.

Например, при декодировании битовой последовательности 000001010 с помощью вышепредставленной таблицы, процесс декодирования будет заключаться в следующем:

1) Читаем биты слева-направо: 0 → 00 → 000.

000 есть в таблице? Да, это n.

Результат: n.

2) Продолжаем со следующего бита: 0 → 00 → 001.

001 есть в таблице? Да, это e.

Результат: e.

3) Продолжаем со следующего бита: 0 → 01 → 010.

010 есть в таблице? Да, это d.

Результат: d.

Расшифрованное сообщение n e d.

Файл rock_sample.dat представляет собой последовательность десятичных чисел, каждая из которых – зашифрованная буква передаваемого сообщения. Исходный текст отчёта перед шифрованием был преобразован в числа по правилу: каждый символ кодировался своим порядковым номером в алфавите (a=1, б=2, ..., я=33). Для шифрования Петром использовалась формула: $C_i = M_i^e \bmod n$,

где C_i – i-й элемент последовательности шифротекста C;



M_i – i -й элемент последовательности шифра M ;
 e – открытая экспонента – параметр открытого ключа RSA – число, используемое при шифровании в качестве степени;
 n – модуль операции взятия остатка;
 mod – операция взятия остатка от деления.
Для дешифрования должна использоваться формула $M_i = C_i^d \text{mod } n$,
где M_i – i -й элемент последовательности шифра M ;
 C_i – i -й элемент последовательности шифротекста C ;
 d – секретная экспонента – параметр закрытого ключа RSA – число, используемое при дешифровке в качестве степени;
 n – модуль операции взятия остатка;
 mod – операция взятия остатка от деления.

Совокупность значений (e, n) и (d, n) является открытым и закрытым ключами, используемыми при шифровании и дешифровании соответственно.

Например, RSA-шифрование с открытым ключом: $(e, n) = (17, 3233)$ – и закрытым ключом: $(d, n) = (2753, 3233)$ заключается в следующем.
При шифровании сообщения $M = 65$ отправляемый шифротекст $C = M^e \text{mod } n = 65^{17} \text{mod } 3233 = 2790$
При дешифровке полученного шифротекста $C = 2790$ восстановленное сообщение $M = C^d \text{mod } n = 2790^{2753} \text{mod } 3233 = 65$.

Примечание

Чтобы вычислить выражение вида $M = C^d \text{mod } n$, иногда невыгодно сначала находить большое число C^d , ведь оно может быть огромным. Удобнее использовать алгоритм быстрого возведения в степень по модулю.

Идея алгоритма:

1. Разложить показатель степени на степени двойки.
2. Последовательно возводить число в квадрат.
3. После каждого шага брать остаток по модулю.
4. Умножать только те степени, которые нужны по разложению.

Например, если нужно найти значение выражения $11^5 \text{mod } 9$, то степень 5 можно представить как сумму степеней 4 и 1: $5 = 4 + 1$. Следовательно:

1. Находим остаток: $11^2 \text{mod } 9 = 4$, т.к. $9 \times 13 = 117$ (остаток 4).
 2. Используем найденный остаток, чтобы получить $11^4 \text{mod } 9$.
 $11^4 \text{mod } 9 = (11^2)^2 \text{mod } 9 \equiv 4^2 \text{mod } 9 = 7$, т.к. $9 \times 1 = 9$ (остаток 7)
 3. Перемножаем найденный остаток с 11 – и снова берём остаток.
 $11^5 \text{mod } 9 = (11^4 \times 11) \text{mod } 9 \equiv (7 \times 11) \text{mod } 9 = 77 \text{mod } 9 = 5$, т.к. $9 \times 8 = 72$ (остаток 5)
- Таким образом, $11^5 \text{mod } 9 = 5$.

Используя данные, извлечённые из стеганографического сообщения, расшифруйте `rock_sample.dat` и узнайте предварительное название новой горной породы. Содержимое `rock_sample.dat`:

Первое слово – 4 12 16 7 4 20 19; второе слово – 18 17 22 24 20 1 7.



В ответе укажите два слова расшифрованного сообщения в формате «Первое слово» «Второе слово» (например, Медное золото). Для получения дополнительных баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю, используемого при дешифровке.

Решение

На первом этапе необходимо определить секретный ключ, который был внедрен Петром методом стеганографии в файл `thin_section.png`. Для сокрытия информации в изображении `thin_section.png` Пётр применил LSB-метод, производя замену третьего значащего бита в компонентах R, G и B каждого пикселя. Следовательно, необходимо последовательно извлечь биты сначала из красного канала первого пикселя, затем из зеленого, затем из синего, записать в том порядке, в котором они были закодированы, а потом перейти к следующему пикселю.

Каждый цветовой канал пикселя представлен числом от 0 до 255. В связи с тем, что замена третьего значащего бита осуществляется в двоичной записи числа, необходимо перевести значения извлечённой последовательности каналов RGB в двоичную систему, выделить третий бит каждого числа и записать получившуюся последовательность.

Для перевода числа из десятичной системы счисления в двоичную необходимо разделить число в десятичной системе на 2 и записать остаток до тех пор, пока неполное частное не будет равно нулю. Полученные остатки записываются в обратном порядке. В рамках задачи можно ограничиться делением числа до момента нахождения значения третьего значащего бита.

Например, извлечение битов из первого пикселя выглядит следующим образом.

Пиксель	Каналы	Число в 10-ой системе счисления	Перевод в 2-ую систему счисления	Третий значащий бит
1	R	79_{10}	$\begin{array}{l} 1) \frac{79}{2} = 39 \text{ (остаток 1)} \\ 2) \frac{39}{2} = 19 \text{ (остаток 1)} \\ 3) \frac{19}{2} = 9 \text{ (остаток 1)} \end{array}$	1
	G	149_{10}	$\begin{array}{l} 1) \frac{149}{2} = 74 \text{ (остаток 1)} \\ 2) \frac{74}{2} = 37 \text{ (остаток 0)} \\ 3) \frac{37}{2} = 18 \text{ (остаток 1)} \end{array}$	1
	B	232_{10}	$\begin{array}{l} 1) \frac{232}{2} = 116 \text{ (остаток 0)} \\ 2) \frac{116}{2} = 58 \text{ (остаток 0)} \\ 3) \frac{58}{2} = 29 \text{ (остаток 0)} \end{array}$	0

Руководствуясь аналогичным алгоритмом, получаем следующие



значения.

Пиксель/ Каналы	Число в 10-ой системе счисления			Третий значащий бит		
	R	G	B	R	G	B
1	79	149	232	1	1	0
2	211	48	194	0	0	0
3	102	165	243	1	1	0
4	73	221	113	0	1	0
5	163	139	68	0	0	1
6	70	39	122	1	1	0
7	104	229	38	0	1	1
8	14	211	170	1	0	0
9	171	126	188	0	1	1
10	52	204	214	1	1	1
11	90	197	31	0	1	1
12	130	236	195	0	1	0
13	107	160	136	0	0	0
14	187	157	62	0	1	1
15	123	194	221	0	0	1
16	247	189	142	1	1	1
17	102	83	19	1	0	0
18	94	133	226	1	1	0
19	177	54	117	0	1	1
20	44	95	249	1	1	0
21	158	134	116	1	1	1
22	110	251	236	1	0	1
23	201	218	32	0	0	0
24	28	145	118	1	0	1
25	98	186	43	0	0	0
26	143	180	240	1	1	0
27	65	189	205	0	1	1
28	191	157	140	1	1	1
29	233	149	88	0	1	0
30	254	182	120	1	1	0
31	58	62	98	0	1	0
32	149	116	226	1	1	0
33	152	208	76	0	0	1

Таким образом, в результате дешифровки пикселей изображения получена следующая битовая строка:

1100001100100011100111000111110110100000110011111001100111101
11101000101000110011111010110010110001

В задаче сказано, что Пётр использовал для шифрования специальный вод, который основан на использовании неравномерного кодирования. Алгоритм дешифровки в данном случае заключается в следующем: читаем строку по битам слева направо, пошагово накапливая последовательность,



пока она не совпадёт с одним из кодов в таблице. Как только совпадение найдено, записываем символ и начинаем формировать следующую последовательность заново.

Начнём процесс декодирования. Если код не найден сразу (например, при чтении первой 1), то показан процесс накопления последовательности: $1 \rightarrow 11 \rightarrow 110$ и т.д., до момента совпадения с записью в таблице кодов, используемых Петром при шифровании. Максимальная длина кодового слова в таблице кодировки – 6 бит (например, 110000, 110011, 111110 и т.д.), поэтому в процессе накопления битовой последовательности последовательности длиннее 6 символов не рассматриваются.

Шаг	Индекс начала	Биты (извлечённые)	Найденный код	Символ
1	0	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11000 \rightarrow \mathbf{110000}$	110000	{
2	6	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow \mathbf{110010}$	110010	"
3	12	$0 \rightarrow 01 \rightarrow 010$	001	e
4	15	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
5	21	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1000$	1000	x
6	25	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1111 \rightarrow 11111 \rightarrow 111110$	111110	9
7	31	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1101 \rightarrow 11010 \rightarrow 110100$	110100	,
8	37	$0 \rightarrow 00 \rightarrow 000$	000	n
9	40	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
10	46	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 11100 \rightarrow 111001$	111001	4
11	52	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1001$	1001	z
12	56	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 11101 \rightarrow 111011$	111011	6
13	62	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1101 \rightarrow 11010 \rightarrow 110100$	000	,
14	68	$0 \rightarrow 01 \rightarrow 010$	010	d
15	71	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1000$	1000	x
16	75	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	1000	:
17	81	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1110 \rightarrow 11101 \rightarrow 111010$	111010	5
18	87	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow \mathbf{110010}$	110010	"
19	93	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow \mathbf{110001}$	110001	}

Собирая полученные символы в порядке их дешифровки, получаем точную расшифровку битовой строки: {"e:x9,n:4z6,d:x5"}. В условии задачи говорится про «мусорные символы» – ловушки, которые Пётр вставляет в сообщение. В таблице кодов есть набор букв-«включений» x, y, z (и т.д.), и они выступают как мусор в конкретном сообщении: они появляются внутри ключей/значений, но не несут смысла для конечного JSON.

Если удалить все мусорные буквы x, y, z из полученной строки {"e:x9,n:4z6,d:x5"}, то получим корректную JSON строку: {"e:9,n:46,d:5"}. Следовательно, получены данные для расшифровки текстового сообщения из файла rock_sample.dat, а именно модуль $n=46$ и секретная экспонента $d=5$.

Файл rock_sample.dat содержит две последовательности целых чисел, где каждое число – зашифрованный символ шифротекста:

- Первое слово: 4 12 16 7 4 20 19
- Второе слово: 18 17 22 24 20 1 7



Из условия задачи воспользуемся формулой дешифрования: $M_i = C_i^d \bmod n$,

где M_i – i -й элемент последовательности шифра M (восстановленный номер буквы ($1 = a, 2 = б, \dots, 33 = я$));

C_i – i -й элемент последовательности шифротекста C ;

d – секретная экспонента;

n – модуль операции взятия остатка;

$\bmod$ – операция взятия остатка от деления.

Дешифруем первое слово: $C = [4 \ 12 \ 16 \ 7 \ 4 \ 20 \ 19]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 4$

$$4^5 = 1024 \rightarrow 1024 \bmod 46 = 12, \text{ т. к. } 46 \times 22 = 1012 \text{ (остаток 12)}$$

$$\text{Таким образом, } M_1 = 4^5 \bmod 46 = 12$$

Следовательно, $M_1 = 12$, что соответствует букве алфавита «к».

2. $C_2 = 12$

$$12^2 = 144 \rightarrow 144 \bmod 46 = 6, \text{ т. к. } 46 \times 3 = 138 \text{ (остаток 6)}$$

$$\text{Таким образом, } 12^2 \equiv 6 \pmod{46}$$

$$12^4 = (12^2)^2 \equiv 6^2 = 36 \rightarrow 36 \bmod 46 = 36$$

$$\text{Таким образом, } 12^4 \equiv 36 \pmod{46}$$

$$12^5 = 12^4 \times 12 \equiv 36 \times 12 = 432 \rightarrow 432 \bmod 46 = 18, \text{ т. к. } 46 \times 9 = 414 \text{ (остаток 18)}$$

$$\text{Таким образом, } M_2 = 12^5 \bmod 46 = 18$$

Следовательно, $M_2 = 18$, что соответствует букве алфавита «р».

3. $C_3 = 16$

$$16^2 = 256 \rightarrow 256 \bmod 46 = 26, \text{ т. к. } 46 \times 5 = 230 \text{ (остаток 26)}$$

$$\text{Таким образом, } 16^2 \equiv 26 \pmod{46}$$

$$16^4 = (16^2)^2 \equiv 26^2 = 676 \rightarrow 676 \bmod 46 = 32, \text{ т. к. } 46 \times 14 = 644 \text{ (остаток 32)}$$

$$\text{Таким образом, } 16^4 \equiv 32 \pmod{46}$$

$$16^5 = 16^4 \times 16 \equiv 32 \times 16 = 512 \rightarrow 512 \bmod 46 = 6, \text{ т. к. } 46 \times 11 = 506 \text{ (остаток 6)}$$

$$\text{Таким образом, } M_3 = 16^5 \bmod 46 = 6$$

Следовательно, $M_3 = 6$, что соответствует букве алфавита «е».

4. $C_5 = 7$

$$7^2 = 49 \rightarrow 49 \bmod 46 = 3, \text{ т. к. } 46 \times 1 = 46 \text{ (остаток 3)}$$

$$\text{Таким образом, } 7^2 \equiv 3 \pmod{46}$$

$$7^4 = (7^2)^2 \equiv 3^2 = 9 \rightarrow 9 \bmod 46 = 9$$

$$\text{Таким образом, } 7^4 \equiv 9 \pmod{46}$$



$$7^5 = 7^4 \times 7 \equiv 9 \times 7 = 63 \rightarrow 63 \bmod 46 = 17, \text{ т. к. } 46 \times 1 = 46 \text{ (остаток 17)}$$

$$\text{Таким образом, } M_5 = 7^5 \bmod 46 = 17$$

Следовательно, $M_5 = 17$, что соответствует букве алфавита «п».

5. $C_5 = 4$

Аналогично $C_1, M_1 = M_5 = 12$, что соответствует букве алфавита «к».

6. $C_6 = 20$

$$20^2 = 400 \rightarrow 400 \bmod 46 = 32, \text{ т. к. } 46 \times 8 = 368 \text{ (остаток 32)}$$

$$\text{Таким образом, } 20^2 \equiv 32 \bmod 46$$

$$20^4 = (20^2)^2 \equiv 32^2 = 1024 \rightarrow 1024 \bmod 46 = 12, \text{ т. к. } 46 \times 22 = 1012 \text{ (остаток 12)}$$

$$\text{Таким образом, } 20^4 \equiv 12 \bmod 46$$

$$20^5 = 20^4 \times 20 \equiv 12 \times 20 = 240 \rightarrow 240 \bmod 46 = 10, \text{ т. к. } 46 \times 5 = 230 \text{ (остаток 10)}$$

$$\text{Таким образом, } M_6 = 20^5 \bmod 46 = 10$$

Следовательно, $M_6 = 10$, что соответствует букве алфавита «и».

7. $C_7 = 19$

$$19^2 = 361 \rightarrow 361 \bmod 46 = 39, \text{ т. к. } 46 \times 7 = 322 \text{ (остаток 39)}$$

$$\text{Таким образом, } 19^2 \equiv 39 \bmod 46$$

$$19^4 = (19^2)^2 \equiv 39^2 = 1521 \bmod 46 = 3, \text{ т. к. } 46 \times 33 = 1518 \text{ (остаток 3)}$$

$$\text{Таким образом, } 19^4 \equiv 3 \bmod 46$$

$$19^5 = 19^4 \times 19 \equiv 3 \times 19 = 57 \rightarrow 57 \bmod 46 = 11, \text{ т. к. } 46 \times 1 = 46 \text{ (остаток 11)}$$

$$\text{Таким образом, } M_7 = 19^5 \bmod 46 = 11$$

Следовательно, $M_7 = 11$, что соответствует букве алфавита «й».

Соединив расшифрованные буквы, получаем слово «крепкий».

Дешифруем второе слово: $C = [18 \ 17 \ 22 \ 24 \ 20 \ 1 \ 7]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 18$

$$18^2 = 324 \rightarrow 324 \bmod 46 = 2, \text{ т. к. } 46 \times 7 = 322 \text{ (остаток 2)}$$

$$\text{Таким образом, } 18^2 \equiv 2 \bmod 46$$

$$18^4 = (18^2)^2 \equiv 2^2 = 4 \bmod 46 = 4$$

$$\text{Таким образом, } 18^4 \equiv 4 \bmod 46$$

$$18^5 = 18^4 \times 18 \equiv 4 \times 18 = 72 \rightarrow 72 \bmod 46 = 26, \text{ т. к. } 46 \times 1 = 46 \text{ (остаток 26)}$$

$$\text{Таким образом, } M_1 = 18^5 \bmod 46 = 26$$

Следовательно, $M_1 = 26$, что соответствует букве алфавита «ш».

2. $C_2 = 17$

$$17^2 = 289 \rightarrow 289 \bmod 46 = 13, \text{ т. к. } 46 \times 7 = 276 \text{ (остаток 13)}$$

$$\text{Таким образом, } 17^2 \equiv 13 \bmod 46$$



$$17^4 = (17^2)^2 \equiv 13^2 = 169 \bmod 46 = 31, \text{ т. к. } 46 \times 3 = 138 \text{ (остаток 31)}$$

Таким образом, $17^4 \equiv 31 \pmod{46}$

$$17^5 = 17^4 \times 17 \equiv 31 \times 17 = 527 \rightarrow 527 \bmod 46 = 21, \text{ т. к. } 46 \times 11 = 506 \text{ (остаток 21)}$$

Таким образом, $M_2 = 17^5 \pmod{46} = 21$

Следовательно, $M_2 = 21$, что соответствует букве алфавита «у».

3. $C_3 = 22$

$$22^2 = 484 \rightarrow 289 \bmod 46 = 24, \text{ т. к. } 46 \times 10 = 460 \text{ (остаток 24)}$$

Таким образом, $22^2 \equiv 24 \pmod{46}$

$$22^4 = (22^2)^2 \equiv 24^2 = 576 \bmod 46 = 24, \text{ т. к. } 46 \times 12 = 552 \text{ (остаток 24)}$$

Таким образом, $22^4 \equiv 24 \pmod{46}$

$$22^5 = 22^4 \times 22 \equiv 24 \times 22 = 528 \rightarrow 528 \bmod 46 = 23, \text{ т. к. } 46 \times 11 = 506 \text{ (остаток 23)}$$

Таким образом, $M_3 = 22^5 \pmod{46} = 23$

Следовательно, $M_3 = 23$, что соответствует букве алфавита «ф».

4. $C_4 = 24$

$$24^2 = 576 \rightarrow 576 \bmod 46 = 24, \text{ т. к. } 46 \times 12 = 552 \text{ (остаток 24)}$$

Таким образом, $24^2 \equiv 24 \pmod{46}$

$$24^4 = (24^2)^2 \equiv 24^2 = 576 \bmod 46 = 24, \text{ т. к. } 46 \times 12 = 552 \text{ (остаток 24)}$$

Таким образом, $24^4 \equiv 24 \pmod{46}$

$$24^5 = 24^4 \times 24 \equiv 24 \times 24 = 576 \rightarrow 576 \bmod 46 = 24, \text{ т. к. } 46 \times 12 = 552 \text{ (остаток 24)}$$

Таким образом, $M_4 = 24^5 \pmod{46} = 24$

Следовательно, $M_4 = 24$, что соответствует букве алфавита «ц».

5. $C_5 = 20$

Аналогично C_6 для первого слова, $M_6 = M_5 = 10$, что соответствует букве алфавита «и».

6. $C_6 = 1$

$$1^5 = 1 \rightarrow 1 \bmod 46 = 1$$

Таким образом, $M_6 = 1^5 \pmod{46} = 1$

Следовательно, $M_6 = 1$, что соответствует букве алфавита «а».

7. $C_7 = 7$

Аналогично C_4 для первого слова, $M_4 = M_7 = 17$, что соответствует букве алфавита «п».

Соединив расшифрованные буквы, получаем слово «шуфциап».

В ответе сказано указать два слова расшифрованного сообщения в формате «Первое слово» «Второе слово», соединяя получаем крепкий шуфциап.

Ответ: крепкий шуфциап



Для получения максимального количества баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю при дешифровке.

C++

```
#include <iostream>

int mod_exp(int base, int exp, int mod) {
    int result = 1;
    while (exp > 0) {
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

int main() {
    int base = 33;
    int exp = 5;
    int mod = 46;

    int result = mod_exp(base, exp, mod);
    std::cout << result << std::endl;

    return 0;
}
```

Python

```
def mod_exp(base, exp, mod):
    result = 1
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp //= 2
    return result

base = 12
exp = 5
mod = 46
result = mod_exp(base, exp, mod)
print(result)
```

**Задача 5 (20 баллов)****Космический зонд на ледяной комете**

На длинной ледяной комете расположено $n \geq 2$ научно-исследовательских станций вдоль числовой оси с целочисленными координатами $b_1, b_2, \dots, b_n$, упорядоченными по возрастанию, где $b_1 = 0$. Космический зонд начинает свою миссию на станции b_1 и должен достичь финальной станции b_n , используя адаптивную систему навигации.

Принцип работы навигационной системы

Зонд использует двухфазную систему перемещения. Сначала происходит **калибровка**: зонд определяет ближайшую станцию впереди (обозначим её b_j), вычисляет эталонное расстояние D как разность между координатой этой станции и текущей позицией зонда, затем телепортируется на станцию b_j , фиксируя найденное значение D как текущий масштаб перемещения.

После калибровки начинается **поиск целевого расстояния**: зонд последовательно сканирует позиции впереди себя (текущая позиция + 1, текущая позиция + 2, текущая позиция + 3 и так далее). Когда зонд обнаруживает станцию на некоторой позиции X , возможны три сценария. Если X равно b_n , то зонд перемещается на нее и успешно завершает миссию. Если на позиции $(X - D)$ также существует станция, то зонд перемещается на станцию X и выполняет новую калибровку в этой точке. В противном случае зонд продолжает сканирование позиций дальше.

Формат входных данных

В первой строке задано целое число n — количество станций, различных целочисленных точек ($2 \leq n \leq 300000$). Во второй строке через пробел записаны n целых чисел $b_1, b_2, \dots, b_n$ — координаты станций в порядке возрастания. Самая левая из них имеет значение 0, самая правая не превосходит 10^9 .

Формат выходных данных

Требуется вывести одно целое число D — итоговое эталонное расстояние, при котором зонд достигает финальной станции b_n .

Примеры

стандартный ввод	стандартный вывод
7 0 3 4 5 6 8 13	2
9 0 3 4 5 6 8 9 10 13	3
12 0 3 4 5 7 8 11 12 14 17 18 24	2

Пояснения к примерам

Иллюстрация к первому примеру.

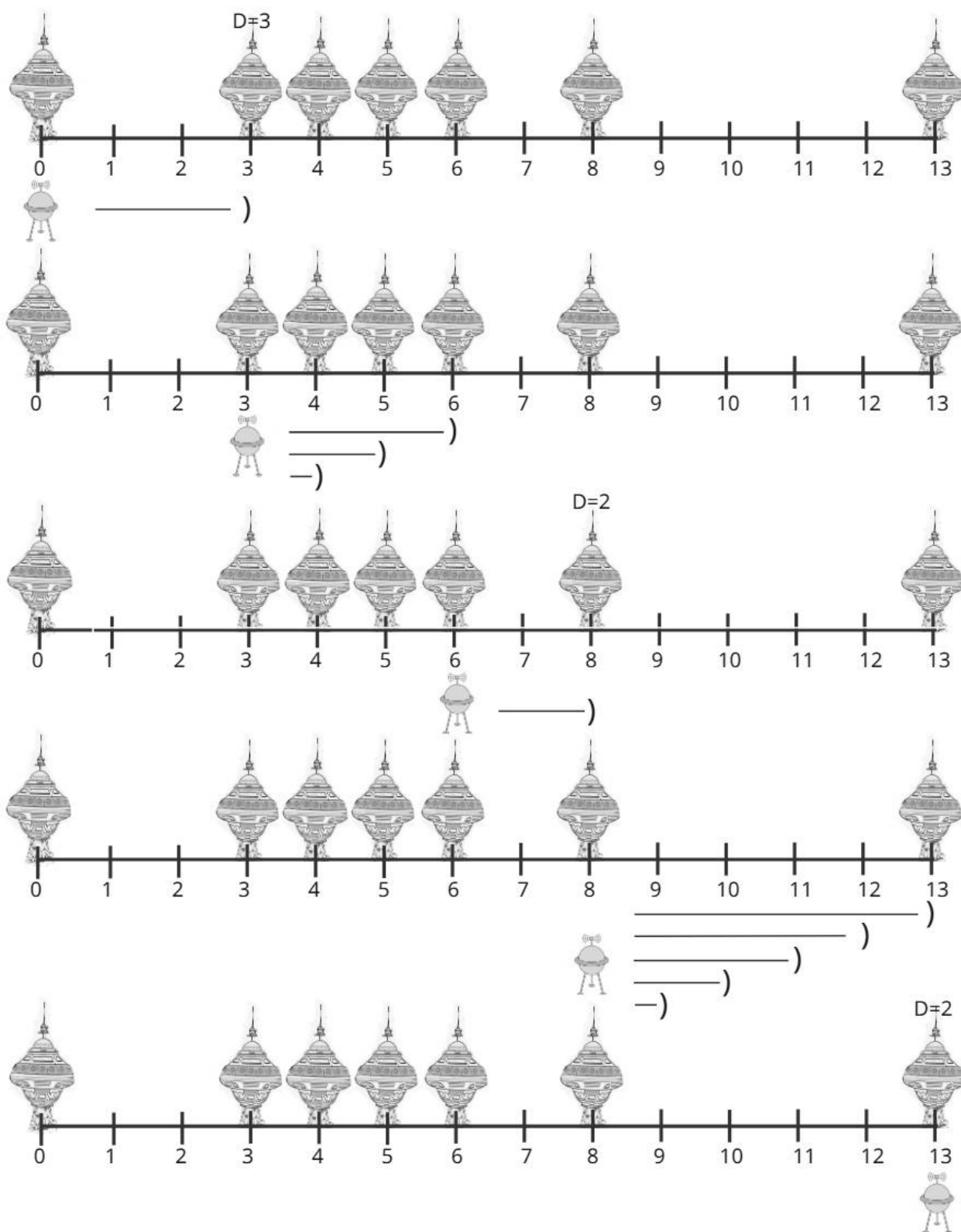
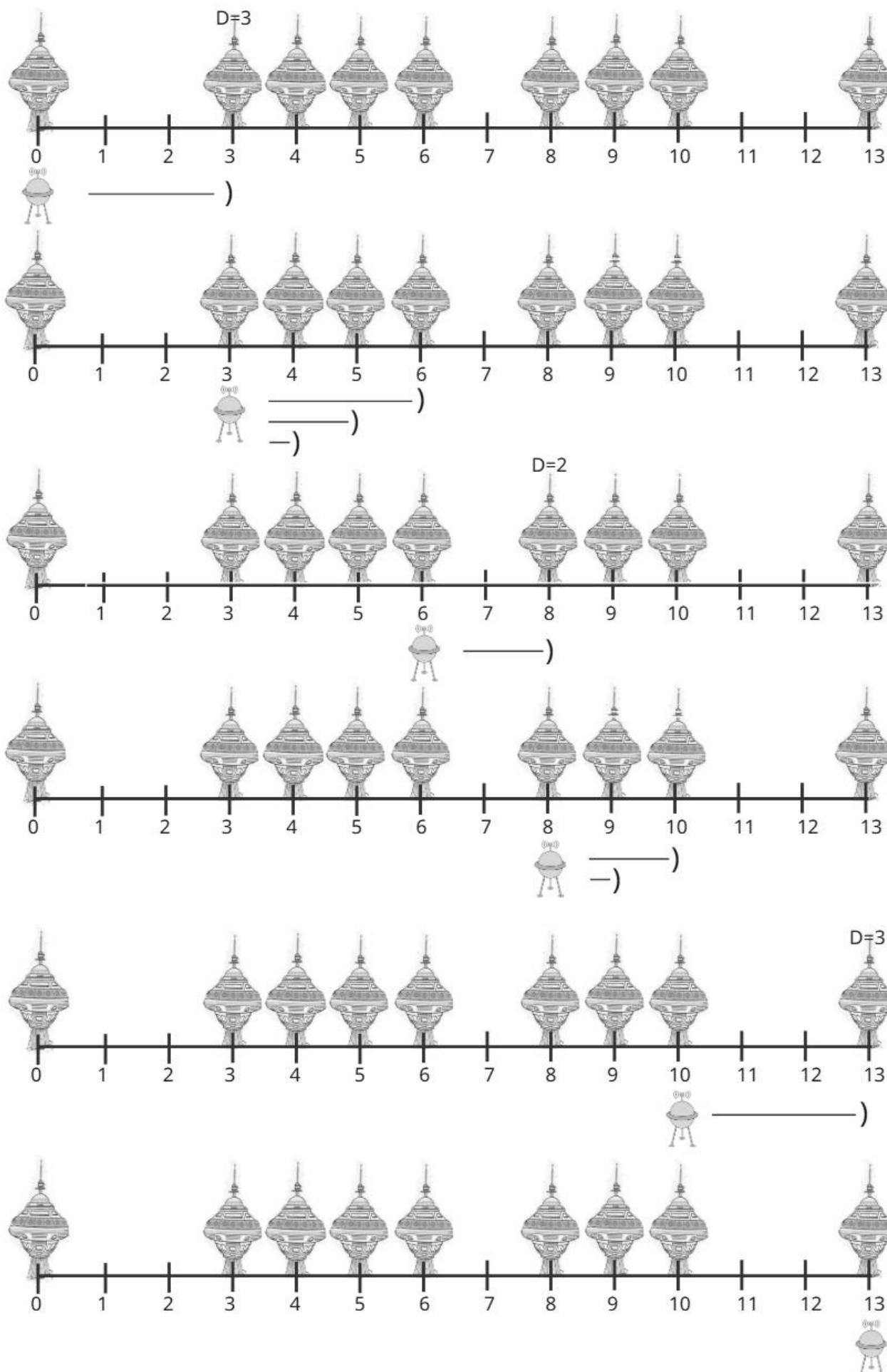


Иллюстрация ко второму примеру.





Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, обязательно **добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных, поясните решение. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
12 0 3 4 7 9 10 12 13 14 18 19 24	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

Смотрите решение в первом варианте.



Задача 6 (30 баллов)

Источник вечной энергии

Известный изобретатель Иван Иванович Иванов разработал принципиально новую установку, позволяющую генерировать сколько угодно безопасной энергии. Он поместил в точки прямой **0** и **D** два направленных друг на друга источника специально заряженных частиц. В момент времени **0** источник с координатой **0** генерирует **n** частиц, которые движутся по прямой в сторону второго генератора. Каждая из них имеет свою скорость v_i , скорости всех частиц попарно различны. Как только какая-то частица достигает источника с координатой **D**, она им поглощается и этот источник выпускает ровно такую же по скорости частицу в противоположном направлении. Эта частица движется обратно в сторону начала координат и снова поглощается источником с координатой **0**, после чего процесс испускания частицы в сторону второго источника снова повторяется.

Известно, что как только две частицы попадают в одну и ту же точку прямой, они генерируют одну единицу энергии. Если в одной точке оказываются несколько частиц, каждая пара среди встретившихся генерирует единицу энергии, в частности, в момент времени **0**, когда **n** частиц стартуют из точки **0**, они уже создают $\frac{n(n-1)}{2}$ единиц энергии. Если две частицы одновременно достигают какого-то из источников, то они генерируют одну единицу перед поглощением, и одну единицу уже после возрождения их двойников, движущихся в обратном направлении. За счет множественности встреч, процесс работы источников имеет большой положительный баланс по выработке энергии. Энергия может вырабатываться сколь угодно долго.

Теперь, для написания научной статьи про опыт использования источника вечной энергии, Иван Иванович хочет точно знать момент, когда будет выработана **m**-я по счету единица энергии.

Формат входных данных

В первой строке ввода через пробел содержатся два целых числа **D** и **n** ($0 \leq D \leq 200$, $2 \leq n \leq 2000$). Во второй строке содержатся **n** целых чисел v_i через пробел – скорости частиц ($1 \leq v_i \leq 2000$). Скорости даны в порядке строгого возрастания, то есть $v_1 < v_2 < \dots < v_n$.

В третьей строке содержится одно целое число **m** – номер требуемой по порядку единицы энергии, для которой следует найти точный момент её генерации ($1 \leq m \leq 10^9$).

Формат выходных данных

Вывести в ответ одно вещественное число – время, когда будет сгенерирована **m**-я по порядку единица энергии. Ответ вывести с точностью не менее 2 знаков после десятичной точки.

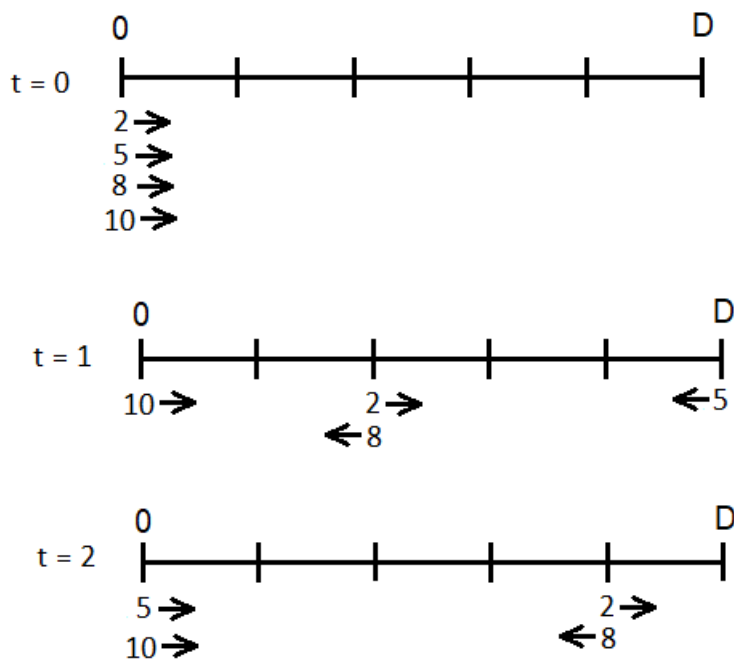


Примеры

стандартный ввод	стандартный вывод
5 4 2 5 8 10 6	0.00
5 4 2 5 8 10 7	0.56
5 4 2 5 8 10 11	1.00
5 4 2 5 8 10 17	1.67
5 4 2 5 8 10 19	1.67
5 4 2 5 8 10 21	2.00

Пояснения к примерам

На следующих рисунках приведено расположение частиц из примеров в моменты времени 0, 1 и 2. Частицы обозначены своими скоростями. Стрелками обозначены направления их движения в соответствующий момент. Если в этот момент частица была регенерирована, она имеет новое направление движения.



Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите



эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, **обязательно добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
5 4 2 5 8 10 14	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

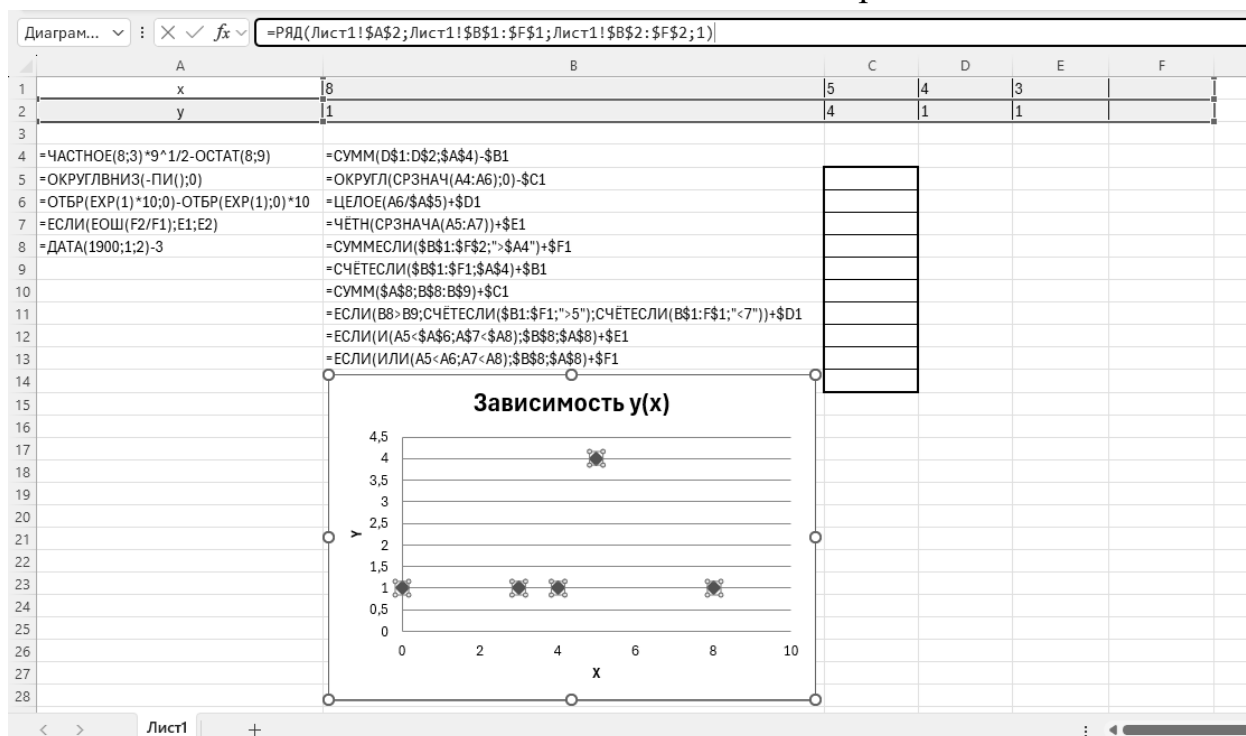
Смотрите решение в первом варианте.



ВАРИАНТ 4

Разведчики

От разведчиков были получены сведения, закодированные в виде электронной таблицы. Определите числа в ячейках C5:C14, которые будут получены в результате копирования в них ячеек B4:B13. Также определите недостающие числа в блоке B1:F2 на основании диаграммы.



В ответе записать, в одну строчку через запятую, значения ячеек C5:C14, начиная с ячейки C5 и заканчивая ячейкой C14. Формат ячеек в таблице – «Общий».

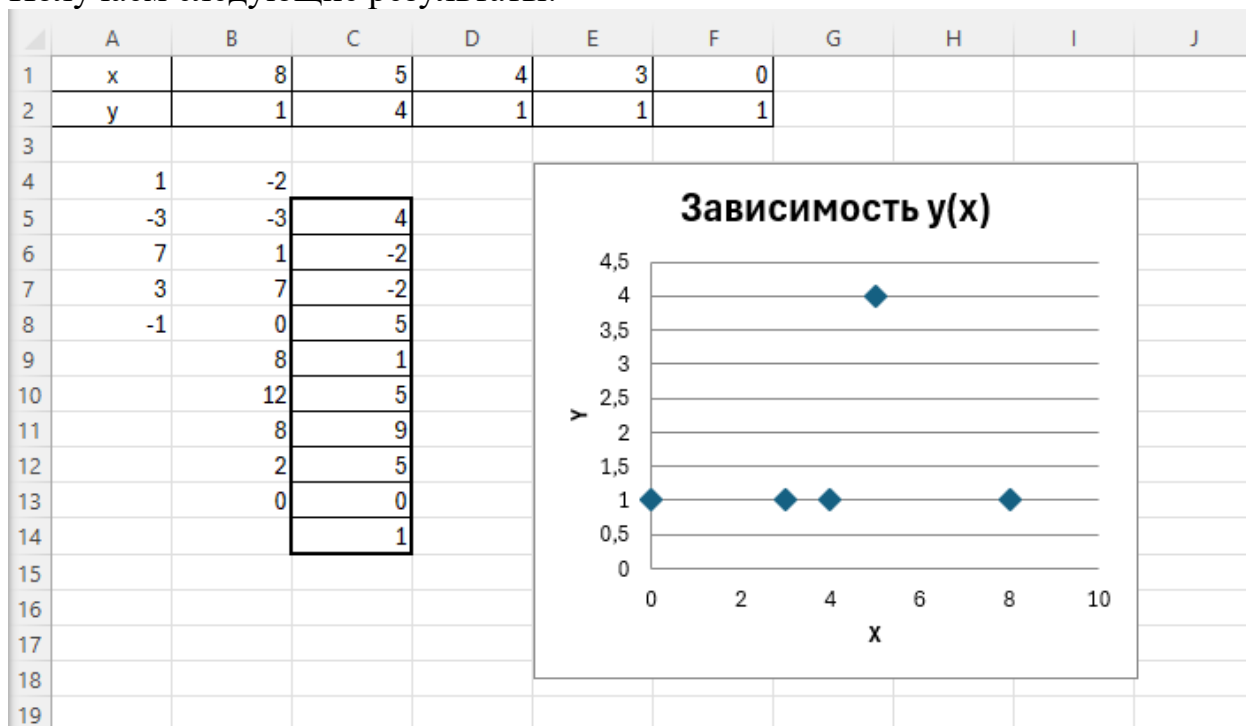
Решение

На основании диаграммы определяем, что в ячейках A1:F2 не отображена точка (x=0, y=1) расположенная в ячейках F1:F2. Далее копируем формулы, расположенные в ячейках B4:B13 в ячейки C5:C14. В результате получаем следующие формулы:

	C	D
5	=СУММ(E\$1:E\$2;\$A\$4)-\$B2	
6	=ОКРУГЛ(СРЗНАЧ(B5:B7);0)-\$C2	
7	=ЦЕЛОЕ(B7/\$A\$5)+\$D2	
8	=ЧЁТН(СРЗНАЧА(B6:B8))+\$E2	
9	=СУММЕСЛИ(\$B\$1:\$F\$2;">\$A4")+ \$F2	
10	=СЧЁТЕСЛИ(\$B\$1:\$F2;\$A\$4)+\$B2	
11	=СУММ(\$A\$8;C\$8:C\$9)+\$C2	
12	=ЕСЛИ(C9>C10;СЧЁТЕСЛИ(\$B2:\$F2;">5");СЧЁТЕСЛИ(C\$1:G\$1;"<7"))+\$D2	
13	=ЕСЛИ(И(B6<\$A\$6;B\$7<\$A9);\$B\$8;\$A\$8)+\$E2	
14	=ЕСЛИ(ИЛИ(B6<B7;B8<B9);\$B\$8;\$A\$8)+\$F2	
15		



Вычисляем значения по формулам в ячейках A4:A8, затем переходим к вычислениям значений в ячейках B4:B13 и, наконец, в ячейках C5:C14. Получаем следующие результаты:



Ответ: 4, -2, -2, 5, 1, 5, 9, 5, 0, 1

**Задача 2 (10 баллов)****Лаборатория кибербезопасности**

Алиса работает в отделе кибербезопасности и тестирует новый алгоритм фильтрации данных. Логическая функция F описывает условия допуска к защищённым файлам:

$$F = z \wedge x \oplus z \rightarrow y \wedge w \wedge x$$

Алиса составила таблицу истинности, но для проверки знаний своих коллег зашифровала некоторые значения логическими выражениями, в которых a, b, c – следующие высказывания:

a – «1 КБ = 2^{10} Б»

b – «24 делится на 4 без остатка»

c – « $7 \times 3 = 20$ »

Определите, какому столбцу таблицы истинности функции F соответствует каждая из переменных w, x, y, z .

?	?	?	?	F
	$\neg a \vee \neg b \rightarrow \neg c$	$a \wedge \neg c \wedge \neg b$		0
$a \vee b \wedge c$	$\neg a \vee b \wedge \neg c$		$a \wedge b \wedge c$	0
$\neg a \wedge c \vee \neg b$		$c \rightarrow a \equiv b$		0
$c \rightarrow b \rightarrow a$		$\neg a \rightarrow c \vee \neg b$		0

В ответе напишите буквы w, x, y, z в том порядке, в котором идут соответствующие им столбцы.

Решение

Определим значения логических высказываний. Таким образом, получим следующее:

$$a=1$$

$$b=1$$

$$c=0$$

Теперь определим значения логических выражений в таблице. В результате получим следующую таблицу:

?	?	?	?	F
0	1	0		0
1	1		0	0
0		1		0
1		1		0

Получаем следующую таблицу истинности:



w	x	y	z	$z \wedge x$	$z \wedge x \oplus z$	$y \wedge w$	$y \wedge w \wedge x$	F
0	0	0	0	0	0	0	0	1
0	0	0	1	0	1	0	0	0
0	0	1	0	0	0	0	0	1
0	0	1	1	0	1	0	0	0
0	1	0	0	0	0	0	0	1
0	1	0	1	1	0	0	0	1
0	1	1	0	0	0	0	0	1
0	1	1	1	1	0	0	0	1
1	0	0	0	0	0	0	0	1
1	0	0	1	0	1	0	0	0
1	0	1	0	0	0	1	0	1
1	0	1	1	0	1	1	0	0
1	1	0	0	0	0	0	0	1
1	1	0	1	1	0	0	0	1
1	1	1	0	0	0	1	1	1
1	1	1	1	1	0	1	1	1

Выберем из нее строки, где функция $F = 0$.

w	x	y	z	F
0	0	0	1	0
0	0	1	1	0
1	0	0	1	0
1	0	1	1	0

Сравнивая эту таблицу с заданной в задании, и учитывая, что строки в этой таблице выписаны в случайном порядке, установим искомые наименования столбцов:

y	z	w	x	F
0	1	0	0	0
1	1	0	0	0
0	1	1	0	0
1	1	1	0	0

или

w	z	y	x	F
0	1	0	0	0
1	1	0	0	0
0	1	1	0	0
1	1	1	0	0

Ответ: y, z, w, x или w, z, y, x

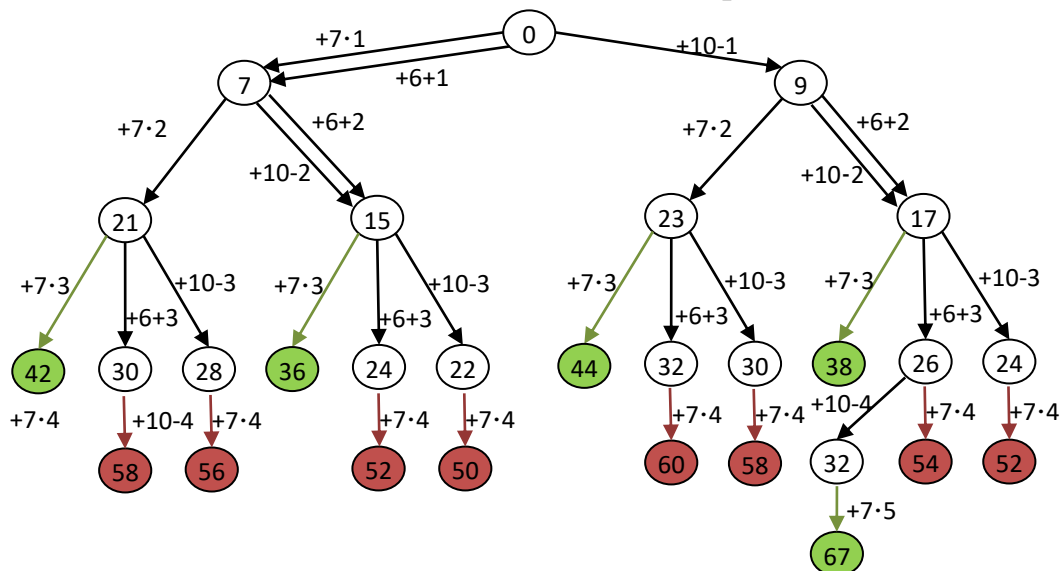
**Задача 3 (15 баллов)****Сокровища пиратов: дуэль за 40 монет**

Два пирата захотели закопать клад. Для этого они должны сначала заполнить сундук золотыми монетами, а чтобы дело шло веселее, они решили сыграть при этом в следующую игру. По очереди они складывают монеты в сундук. За один ход можно либо положить в сундук количество монет, определяемое формулой: либо 7 умноженное на номер хода в игре, либо 6 плюс номер хода в игре, либо 10 минус номер хода в игре. При достижении или превышении количества 33 монеты в сундуке, игра заканчивается. Кто первым, сделав ход, получит в сундуке 33 монеты или больше – считается победителем.

Кто выигрывает при безошибочной игре – пират, делающий первый ход, или пират, делающий второй ход? Каково минимальное число ходов до победы, если выигрывающий пират пытается закончить игру за меньшее число ходов, а проигрывающий пытается максимально задержать свое поражение? Какое максимальное число монет может оказаться в сундуке в конце игры? Какой пират при этом победит? Поясните алгоритм графически, используя ориентированные графы в виде деревьев, с обозначением выигрышных и проигрышных позиций как вершин, а действий игроков – дугами графа с указанием численных изменений

Решение

Построим ориентированный граф, где зеленым цветом выделены выигрышные позиции первого пирата, красным цветом – выигрышные позиции второго пирата, а номер соответствует количеству монет в сундуке. Рядом со стрелкой указано изменение числа монет в сундуке на этом ходу. В случае, если один из вариантов хода приводит к победе, то второй вариант не отображается, за исключением ветвей, которые демонстрирует игру с максимально возможным числом монет в конце игры.





Из графа видно, что при любой игре второго пирата, гарантированно победит первый пират на 3 ходу, если вторым ходом положит 21 монету. Максимально возможное число монет можно получить, если увеличивать количество монет в сундуке таким образом, чтобы получить как можно более высокий номер хода (5) и количества монет в сундуке (32) перед последним ходом, в котором берется вариант с большим значением $(+7 \cdot 5)$. При этом побеждает первый пират с количеством монет 67.

Ответ: гарантированно победит первый пират за 3 хода, максимум 67 монеты при победе первого пирата.



Задача 4 (20 баллов)

Аномальный криокамень

В ходе арктической экспедиции обнаружена горная порода с аномальной кристаллической решеткой, демонстрирующая свойства высокотемпературной сверхпроводимости. Старший геолог Пётр, опасаясь промышленного шпионажа, передал коллеге Марии, используя многоуровневую систему защиты, зашифрованные данные, содержащие предварительное название породы из двух слов.

Мария получила два файла:

- 1) rock_sample.dat – файл с основным зашифрованным отчётом;
- 2) thin_section.png – микрофотография тонкого среза породы, в которую Петром был внедрён секретный ключ методом стеганографии, имитирующим природные включения.

Алгоритм действий для Марии выглядит следующим образом:

1) Извлечь скрытые данные из изображения thin_section.png. Для этого необходимо проанализировать третьи значащие биты цветовых каналов пикселей, записать их последовательно в том порядке, в котором они были закодированы, и получить битовую последовательность. Микрофотография представляет собой 8-битное RGB-изображение.

2) Эту битовую последовательность необходимо декодировать. В результате она должна превратиться в строку в формате JSON, которая и содержит секретные ключи для алгоритма RSA. В данной задаче строка в формате JSON выглядит следующим образом: {ключ1: значение1, ключ2: значение2, ключ3: значение3}.

3) Используя полученные ключи, можно приступить к дешифровке основного отчёта, хранящегося в файле rock_sample.dat.

Для сокрытия информации в изображении thin_section.png Пётр применил LSB-метод, производя замену третьего значащего младшего бита в компонентах R, G и B каждого пикселя.

Обычное LSB-стеганографирование заменяет младший бит (bit_0) в каждом цветовом канале R, G и B. Однако Пётр использует усложнённую модификацию метода, где он заменяет третий значащий бит.

Например, необходимо зашифровать бит сообщения = 0. Имеется цвет пикселя изображения: R = 101. Двоичное представление данного компонента канала: $101 \rightarrow 01100101_2$

1) Читаем текущий bit_2 для замены: $\text{bit}_2 = 1$

2) Записываем «0» вместо него, изменяя только данный бит.

Таким образом, до шифрования было: 01100101_2 (= 101). После шифрования станет: 01100001_2 (= 97)

После проведения спектрального и статистического анализа изображения thin_section.png Мария выделила последовательность пикселей,



в каждом цветовом канале которых наблюдались отклонения, характерные для внедрённого стеганографического сообщения. Мария извлекла из этих пикселей их значения каналов RGB, получив следующую последовательность.

Пиксель	R	G	B
1	189	62	163
2	225	97	138
3	30	252	177
4	216	86	43
5	105	111	235
6	60	239	195
7	209	244	94
8	204	8	160
9	125	220	134
10	188	49	214
11	121	118	229
12	193	180	34
13	82	202	178
14	237	22	157
15	113	42	204
16	254	100	165
17	166	101	53
18	216	127	228
19	81	157	74
20	130	142	242
21	198	1	136
22	169	152	103
23	117	184	83
24	47	221	190
25	223	108	122
26	172	173	38
27	233	89	59
28	70	182	212
29	247	142	66
30	77	175	130
31	145	92	18
32	236	207	153
33	56	251	126

Пётр, известный своей любовью к горным треккигам, в файле thin_section.png для последующей защиты дешифрованной битовой последовательности применил специальный «Горный код» стеганографии.



Его принцип основан на использовании неравномерного кодирования, где частым символам соответствуют короткие коды, что минимизирует общий размер секретного сообщения. Дополнительный уровень защиты обеспечивают «мусорные» символы – специальные ловушки, встроенные в код. При расшифровке легитимный получатель должен распознать и отфильтровать этот мусор, чтобы получить исходные данные. Таблица используемых Петром кодов представлена ниже.

Символ	Код	Символ	Код
{	110000	7	111100
}	110001	8	111101
"	110010	9	111110
:	110011	n	000
,	110100	e	001
0	110101	d	010
1	110110	p	0110
2	110111	q	0111
3	111000	x	1000
4	111001	z	1001
5	111010	y	1010
6	111011		

Неравномерный код – это код, в котором кодовые слова, например, последовательности битов, имеют разную длину.

Например, при декодировании битовой последовательности 000001010 с помощью вышепредставленной таблицы, процесс декодирования будет заключаться в следующем:

1) Читаем биты слева-направо: 0 → 00 → 000.

000 есть в таблице? Да, это n.

Результат: n.

2) Продолжаем со следующего бита: 0 → 00 → 001.

001 есть в таблице? Да, это e.

Результат: e.

3) Продолжаем со следующего бита: 0 → 01 → 010.

010 есть в таблице? Да, это d.

Результат: d.

Расшифрованное сообщение n e d.

Файл rock_sample.dat представляет собой последовательность десятичных чисел, каждая из которых – зашифрованная буква передаваемого сообщения. Исходный текст отчёта перед шифрованием был преобразован в числа по правилу: каждый символ кодировался своим порядковым номером в алфавите (a=1, б=2, ..., я=33). Для шифрования Петром использовалась формула: $C_i = M_i^e \bmod n$,

где C_i – i-й элемент последовательности шифротекста С;

M_i – i-й элемент последовательности шифра М;



e – открытая экспонента – параметр открытого ключа RSA – число, используемое при шифровании в качестве степени;

n – модуль операции взятия остатка;

mod – операция взятия остатка от деления.

Для дешифрования должна использоваться формула $M_i = C_i^d \text{mod } n$,

где M_i – i -й элемент последовательности шифра M ;

C_i – i -й элемент последовательности шифротекста C ;

d – секретная экспонента – параметр закрытого ключа RSA – число, используемое при дешифровке в качестве степени;

n – модуль операции взятия остатка;

mod – операция взятия остатка от деления.

Совокупность значений (e, n) и (d, n) является открытым и закрытым ключами, используемыми при шифровании и дешифровании соответственно.

Например, RSA-шифрование с открытым ключом: $(e, n) = (17, 3233)$ – и закрытым ключом: $(d, n) = (2753, 3233)$ заключается в следующем.	
При шифровании сообщения $M = 65$ отправляемый шифротекст $C = M^e \text{mod } n = 65^{17} \text{mod } 3233 = 2790$	
При дешифровке полученного шифротекста $C = 2790$ восстановленное сообщение $M = C^d \text{mod } n = 2790^{2753} \text{mod } 3233 = 65$.	

Примечание

Чтобы вычислить выражение вида $M = C^d \text{mod } n$, иногда невыгодно сначала находить большое число C^d , ведь оно может быть огромным. Удобнее использовать алгоритм быстрого возведения в степень по модулю.

Идея алгоритма:

1. Разложить показатель степени на степени двойки.
2. Последовательно возводить число в квадрат.
3. После каждого шага брать остаток по модулю.
4. Умножать только те степени, которые нужны по разложению.

Например, если нужно найти значение выражения $11^5 \text{mod } 9$, то степень 5 можно представить как сумму степеней 4 и 1: $5 = 4 + 1$. Следовательно:

1. Находим остаток: $11^2 \text{mod } 9 = 4$, т.к. $9 \times 13 = 117$ (остаток 4).
 2. Используем найденный остаток, чтобы получить $11^4 \text{mod } 9$.
 $11^4 \text{mod } 9 = (11^2)^2 \text{mod } 9 \equiv 4^2 \text{mod } 9 = 7$, т.к. $9 \times 1 = 9$ (остаток 7)
 3. Перемножаем найденный остаток с 11 – и снова берём остаток.
 $11^5 \text{mod } 9 = (11^4 \times 11) \text{mod } 9 \equiv (7 \times 11) \text{mod } 9 = 77 \text{mod } 9 = 5$, т.к. $9 \times 8 = 72$ (остаток 5)
- Таким образом, $11^5 \text{mod } 9 = 5$.

Используя данные, извлечённые из стеганографического сообщения, расшифруйте `rock_sample.dat` и узнайте предварительное название новой горной породы. Содержимое `rock_sample.dat`:

Первое слово – 10 31 49 61 60 62 65; второе слово – 13 60 39 65 10 43 5.

В ответе укажите два слова расшифрованного сообщения в формате



«Первое слово» «Второе слово» (например, Медное золото).

Для получения дополнительных баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю, используемого при дешифровке.

Решение

На первом этапе необходимо определить секретный ключ, который был внедрен Петром методом стеганографии в файл `thin_section.png`. Для сокрытия информации в изображении `thin_section.png` Пётр применил LSB-метод, производя замену третьего значащего бита в компонентах R, G и B каждого пикселя. Следовательно, необходимо последовательно извлечь биты сначала из красного канала первого пикселя, затем из зеленого, затем из синего, записать в том порядке, в котором они были закодированы, а потом перейти к следующему пикселю.

Каждый цветовой канал пикселя представлен числом от 0 до 255. В связи с тем, что замена третьего значащего бита осуществляется в двоичной записи числа, необходимо перевести значения извлечённой последовательности каналов RGB в двоичную систему, выделить третий бит каждого числа и записать получившуюся последовательность.

Для перевода числа из десятичной системы счисления в двоичную необходимо разделить число в десятичной системе на 2 и записать остаток до тех пор, пока неполное частное не будет равно нулю. Полученные остатки записываются в обратном порядке. В рамках задачи можно ограничиться делением числа до момента нахождения значения третьего значащего бита.

Например, извлечение битов из первого пикселя выглядит следующим образом.

Пиксель	Каналы	Число в 10-ой системе счисления	Перевод в 2-ую систему счисления	Третий значащий бит
1	R	189_{10}	$\begin{array}{l} 1) \frac{189}{2} = 94 \text{ (остаток 1)} \\ 2) \frac{94}{2} = 47 \text{ (остаток 0)} \\ 3) \frac{47}{2} = 23 \text{ (остаток 1)} \end{array}$	1
	G	62_{10}	$\begin{array}{l} 1) \frac{62}{2} = 31 \text{ (остаток 0)} \\ 2) \frac{31}{2} = 15 \text{ (остаток 1)} \\ 3) \frac{15}{2} = 7 \text{ (остаток 1)} \end{array}$	1
	B	163_{10}	$\begin{array}{l} 1) \frac{163}{2} = 81 \text{ (остаток 1)} \\ 2) \frac{81}{2} = 40 \text{ (остаток 1)} \\ 3) \frac{40}{2} = 20 \text{ (остаток 0)} \end{array}$	0

Руководствуясь аналогичным алгоритмом, получаем следующие значения.



Пиксель/ Каналы	Число в 10-ой системе счисления			Третий значащий бит		
	R	G	B	R	G	B
1	189	62	163	1	1	0
2	225	97	138	0	0	0
3	30	252	177	1	1	0
4	216	86	43	0	1	0
5	105	111	235	0	1	0
6	60	239	195	1	1	0
7	209	244	94	0	1	1
8	204	8	160	1	0	0
9	125	220	134	1	1	1
10	188	49	214	1	0	1
11	121	118	229	0	1	1
12	193	180	34	0	1	0
13	82	202	178	0	0	0
14	237	22	157	1	1	1
15	113	42	204	0	0	1
16	254	100	165	1	1	1
17	166	101	53	1	1	1
18	216	127	228	0	1	1
19	81	157	74	0	1	0
20	130	142	242	0	1	0
21	198	1	136	1	0	0
22	169	152	103	0	0	1
23	117	184	83	1	0	0
24	47	221	190	1	1	1
25	223	108	122	1	1	0
26	172	173	38	1	1	1
27	233	89	59	0	0	0
28	70	182	212	1	1	1
29	247	142	66	1	1	0
30	77	175	130	1	1	0
31	145	92	18	0	1	0
32	236	207	153	1	1	0
33	56	251	126	0	0	1

Таким образом, в результате дешифровки пикселей изображения получена следующая битовая строка:

1100001100100101100111001111010110100001110011111110100011010
00001100111110111010111110110010110001

В задаче сказано, что Пётр использовал для шифрования специальный вод, который основан на использовании неравномерного кодирования. Алгоритм дешифровки в данном случае заключается в следующем: читаем строку по битам слева направо, пошагово накапливая последовательность, пока она не совпадёт с одним из кодов в таблице. Как только совпадение



найденно, записываем символ и начинаем формировать следующую последовательность заново.

Начнём процесс декодирования. Если код не найден сразу (например, при чтении первой 1), то показан процесс накопления последовательности: $1 \rightarrow 11 \rightarrow 110$ и т.д., до момента совпадения с записью в таблице кодов, используемых Петром при шифровании. Максимальная длина кодового слова в таблице кодировки – 6 бит (например, 110000, 110011, 111110 и т.д.), поэтому в процессе накопления битовой последовательности последовательности длиннее 6 символов не рассматриваются.

Шаг	Индекс начала	Биты (извлечённые)	Найденный код	Символ
1	0	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11000 \rightarrow 110000$	110000	{
2	6	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110010$	110010	"
3	12	$0 \rightarrow 01 \rightarrow 010$	010	d
4	15	$110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
5	21	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1001$	1001	z
6	25	$111 \rightarrow 1110 \rightarrow 11101 \rightarrow 111010$	111010	5
7	31	$1101 \rightarrow 11010 \rightarrow 110100$	110100	,
8	37	$0 \rightarrow 00 \rightarrow 001$	001	e
9	40	$110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
10	46	$111 \rightarrow 1111 \rightarrow 11111 \rightarrow 111110$	111110	9
11	52	$1101 \rightarrow 11010 \rightarrow 110100$	110100	,
12	58	$1 \rightarrow 10 \rightarrow 101 \rightarrow 1010$	1010	y
13	62	$0 \rightarrow 00 \rightarrow 000$	000	n
14	65	$110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110011$	110011	:
15	71	$111 \rightarrow 1110 \rightarrow 11101 \rightarrow 111011$	111011	6
16	77	$1 \rightarrow 10 \rightarrow 100 \rightarrow 1000$	1000	x
17	81	$1 \rightarrow 11 \rightarrow 111 \rightarrow 1111 \rightarrow 11111 \rightarrow 111110$	111110	9
18	87	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11001 \rightarrow 110010$	110010	"
19	93	$1 \rightarrow 11 \rightarrow 110 \rightarrow 1100 \rightarrow 11000 \rightarrow 110001$	110001	}

Собирая полученные символы в порядке их дешифровки, получаем точную расшифровку битовой строки: {"d:z5,e:9,yn:6x9"}. В условии задачи говорится про «мусорные символы» – ловушки, которые Пётр вставляет в сообщение. В таблице кодов есть набор букв-«включений» x, y, z (и т.д.), и они выступают как мусор в конкретном сообщении: они появляются внутри ключей/значений, но не несут смысла для конечного JSON.

Если удалить все мусорные буквы x, y, z из полученной строки {"d:z5,e:9,yn:6x9"}, то получим корректную JSON строку: {"d:5,e:9,y:69"}. Следовательно, получены данные для расшифровки текстового сообщения из файла rock_sample.dat, а именно модуль $n=69$ и секретная экспонента $d=5$.

Файл rock_sample.dat содержит две последовательности целых чисел, где каждое число – зашифрованный символ шифротекста:

- Первое слово: 10 31 49 61 60 62 65
- Второе слово: 13 60 39 65 10 43 5



Из условия задачи воспользуемся формулой дешифрования: $M_i = C_i^d \bmod n$,

где M_i – i -й элемент последовательности шифра M (восстановленный номер буквы ($1 = a, 2 = б, \dots, 33 = я$));

C_i – i -й элемент последовательности шифротекста C ;

d – секретная экспонента;

n – модуль операции взятия остатка;

$\bmod$ – операция взятия остатка от деления.

Дешифруем первое слово: $C = [10 \ 31 \ 49 \ 61 \ 60 \ 62 \ 65]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 10$

$$10^2 = 100 \rightarrow 100 \bmod 69 = 31, \text{ т. к. } 69 \times 1 = 69 \text{ (остаток 31)}$$

Таким образом, $10^2 \equiv 31 \pmod{69}$

$$10^4 = (10^2)^2 \equiv 31^2 = 961 \rightarrow 961 \bmod 69 = 64, \text{ т. к. } 69 \times 13 = 897 \text{ (остаток 64)}$$

Таким образом, $10^4 \equiv 64 \pmod{69}$

$$10^5 = 10^4 \times 10 \equiv 64 \times 10 = 640 \rightarrow 640 \bmod 69 = 19, \text{ т. к. } 69 \times 9 = 621 \text{ (остаток 19)}$$

Таким образом, $M_1 = 10^5 \pmod{69} = 19$

Следовательно, $M_1 = 19$, что соответствует букве алфавита «с».

2. $C_2 = 31$

$$31^2 = 961 \rightarrow 961 \bmod 69 = 64, \text{ т. к. } 69 \times 13 = 897 \text{ (остаток 64)}$$

Таким образом, $31^2 \equiv 64 \pmod{69}$

$$31^4 = (31^2)^2 \equiv 64^2 = 4096 \rightarrow 4096 \bmod 69 = 25, \text{ т. к. } 69 \times 59 = 4071 \text{ (остаток 25)}$$

Таким образом, $31^4 \equiv 25 \pmod{69}$

$$31^5 = 31^4 \times 31 \equiv 25 \times 31 = 775 \rightarrow 775 \bmod 69 = 16, \text{ т. к. } 69 \times 11 = 759 \text{ (остаток 16)}$$

Таким образом, $M_2 = 31^5 \pmod{69} = 16$

Следовательно, $M_2 = 16$, что соответствует букве алфавита «о».

3. $C_3 = 49$

$$49^2 = 2401 \rightarrow 2401 \bmod 69 = 55, \text{ т. к. } 69 \times 34 = 2346 \text{ (остаток 55)}$$

Таким образом, $49^2 \equiv 55 \pmod{69}$

$$49^4 = (49^2)^2 \equiv 55^2 = 3025 \rightarrow 3025 \bmod 69 = 58, \text{ т. к. } 69 \times 43 = 2967 \text{ (остаток 58)}$$

Таким образом, $49^4 \equiv 58 \pmod{69}$

$$49^5 = 49^4 \times 49 \equiv 58 \times 49 = 2842 \rightarrow 2842 \bmod 69 = 13, \text{ т. к. } 69 \times 41 = 2829 \text{ (остаток 13)}$$

Таким образом, $M_3 = 49^5 \pmod{69} = 13$

Следовательно, $M_3 = 13$, что соответствует букве алфавита «л».

4. $C_4 = 61$

$$61^2 = 3721 \rightarrow 3721 \bmod 69 = 64, \text{ т. к. } 69 \times 53 = 3657 \text{ (остаток 64)}$$

Таким образом, $61^2 \equiv 64 \pmod{69}$



$$61^4 = (61^2)^2 \equiv 64^2 = 4096 \rightarrow 4096 \bmod 69 = 25, \text{ т. к. } 69 \times 59 = 4071 \text{ (остаток 25)}$$

Таким образом, $61^4 \equiv 25 \pmod{69}$

$$61^5 = 61^4 \times 61 \equiv 25 \times 61 = 1525 \rightarrow 1525 \bmod 69 = 7, \text{ т. к. } 69 \times 22 = 1518 \text{ (остаток 7)}$$

Таким образом, $M_4 = 61^5 \pmod{69} = 7$

Следовательно, $M_4 = 7$, что соответствует букве алфавита «ё».

5. $C_5 = 60$

$$60^2 = 3600 \rightarrow 3600 \bmod 69 = 12, \text{ т. к. } 69 \times 52 = 3588 \text{ (остаток 12)}$$

Таким образом, $60^2 \equiv 12 \pmod{69}$

$$60^4 = (60^2)^2 \equiv 12^2 = 144 \rightarrow 144 \bmod 69 = 6, \text{ т. к. } 69 \times 2 = 138 \text{ (остаток 6)}$$

Таким образом, $60^4 \equiv 6 \pmod{69}$

$$60^5 = 60^4 \times 60 \equiv 6 \times 60 = 360 \rightarrow 360 \bmod 69 = 15, \text{ т. к. } 69 \times 5 = 345 \text{ (остаток 15)}$$

Таким образом, $M_5 = 60^5 \pmod{69} = 15$

Следовательно, $M_5 = 15$, что соответствует букве алфавита «н».

6. $C_6 = 62$

$$62^2 = 3844 \rightarrow 3844 \bmod 69 = 49, \text{ т. к. } 69 \times 55 = 3795 \text{ (остаток 49)}$$

Таким образом, $62^2 \equiv 49 \pmod{69}$

$$62^4 = (62^2)^2 \equiv 49^2 = 2401 \rightarrow 2401 \bmod 69 = 55, \text{ т. к. } 69 \times 34 = 2346 \text{ (остаток 55)}$$

Таким образом, $62^4 \equiv 55 \pmod{69}$

$$62^5 = 62^4 \times 62 \equiv 55 \times 62 = 3410 \rightarrow 3410 \bmod 69 = 29, \text{ т. к. } 69 \times 49 = 3381 \text{ (остаток 29)}$$

Таким образом, $M_6 = 62^5 \pmod{69} = 29$

Следовательно, $M_6 = 29$, что соответствует букве алфавита «ы».

7. $C_7 = 65$

$$65^2 = 4225 \rightarrow 4225 \bmod 69 = 16, \text{ т. к. } 69 \times 61 = 4209 \text{ (остаток 16)}$$

Таким образом, $65^2 \equiv 16 \pmod{69}$

$$65^4 = (65^2)^2 \equiv 16^2 = 256 \rightarrow 256 \bmod 69 = 49, \text{ т. к. } 69 \times 3 = 207 \text{ (остаток 49)}$$

Таким образом, $65^4 \equiv 49 \pmod{69}$

$$65^5 = 65^4 \times 65 \equiv 49 \times 65 = 3185 \rightarrow 3185 \bmod 69 = 11, \text{ т. к. } 69 \times 46 = 3174 \text{ (остаток 11)}$$

Таким образом, $M_7 = 65^5 \pmod{69} = 11$

Следовательно, $M_8 = 11$, что соответствует букве алфавита «й».

Соединив расшифрованные буквы, получаем слово «солёный».

Дешифруем второе слово: $C = [13 \ 60 \ 39 \ 65 \ 10 \ 43 \ 5]$. Можно производить дешифровку напрямую возводя числа в степень $d=5$ или же воспользоваться быстрым возведением в степень через квадраты.

1. $C_1 = 13$

$$13^2 = 169 \rightarrow 169 \bmod 69 = 31, \text{ т. к. } 69 \times 2 = 138 \text{ (остаток 31)}$$

Таким образом, $13^2 \equiv 31 \pmod{69}$



$$13^4 = (13^2)^2 \equiv 31^2 = 961 \rightarrow 961 \bmod 69 = 64, \text{ т. к. } 69 \times 13 = 897 \text{ (остаток 64)}$$

Таким образом, $13^4 \equiv 64 \pmod{69}$

$$13^5 = 13^4 \times 13 \equiv 64 \times 13 = 832 \rightarrow 832 \bmod 69 = 4, \text{ т. к. } 69 \times 12 = 828 \text{ (остаток 4)}$$

Таким образом, $M_1 = 13^5 \pmod{69} = 4$

Следовательно, $M_1 = 4$, что соответствует букве алфавита «г».

2. $C_2 = 60$

Аналогично C_5 для первого слова, $M_5 = M_2 = 15$, что соответствует букве алфавита «н».

3. $C_3 = 39$

$$39^2 = 1521 \rightarrow 1521 \bmod 69 = 3, \text{ т. к. } 69 \times 22 = 1518 \text{ (остаток 3)}$$

Таким образом, $39^2 \equiv 3 \pmod{69}$

$$39^4 = (39^2)^2 \equiv 3^2 = 9 \rightarrow 9 \bmod 69 = 9$$

Таким образом, $39^4 \equiv 9 \pmod{69}$

$$39^5 = 39^4 \times 39 \equiv 9 \times 39 = 351 \rightarrow 351 \bmod 69 = 6, \text{ т. к. } 69 \times 5 = 345 \text{ (остаток 6)}$$

Таким образом, $M_3 = 39^5 \pmod{69} = 6$

Следовательно, $M_3 = 6$, что соответствует букве алфавита «е».

4. $C_4 = 65$

Аналогично C_7 для первого слова, $M_7 = M_4 = 11$, что соответствует букве алфавита «й».

5. $C_5 = 10$

Аналогично C_1 для первого слова, $M_1 = M_4 = 19$, что соответствует букве алфавита «с».

6. $C_6 = 43$

$$43^2 = 1849 \rightarrow 1849 \bmod 69 = 55, \text{ т. к. } 69 \times 26 = 1794 \text{ (остаток 55)}$$

Таким образом, $43^2 \equiv 55 \pmod{69}$

$$43^4 = (43^2)^2 \equiv 55^2 = 3025 \rightarrow 3025 \bmod 69 = 58, \text{ т. к. } 69 \times 43 = 2967 \text{ (остаток 58)}$$

Таким образом, $43^4 \equiv 58 \pmod{69}$

$$43^5 = 43^4 \times 43 \equiv 58 \times 43 = 2494 \rightarrow 2494 \bmod 69 = 10, \text{ т. к. } 69 \times 36 = 2484 \text{ (остаток 10)}$$

Таким образом, $M_6 = 43^5 \pmod{69} = 10$

Следовательно, $M_6 = 10$, что соответствует букве алфавита «и».

7. $C_7 = 5$

$$5^5 = 3125 \rightarrow 3125 \bmod 69 = 20, \text{ т. к. } 69 \times 45 = 3105 \text{ (остаток 20)}$$

Таким образом, $M_7 = 43^5 \pmod{69} = 20$

Следовательно, $M_7 = 20$, что соответствует букве алфавита «т».

Соединив расшифрованные буквы, получаем слово «гнейсит».



В ответе сказано указать два слова расшифрованного сообщения в формате «Первое слово» «Второе слово», соединяя получаем солёный гнейсит.

Ответ: солёный гнейсит

Для получения максимального количества баллов за задачу необходимо написать программу алгоритма быстрого возведения в степень по модулю при дешифровке.

C++

```
#include <iostream>

int mod_exp(int base, int exp, int mod) {
    int result = 1;
    while (exp > 0) {
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

int main() {
    int base = 33;
    int exp = 5;
    int mod = 46;

    int result = mod_exp(base, exp, mod);
    std::cout << result << std::endl;

    return 0;
}
```

Python

```
def mod_exp(base, exp, mod):
    result = 1
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp //= 2
    return result

base = 12
exp = 5
mod = 46
result = mod_exp(base, exp, mod)
print(result)
```

**Задача 5 (20 баллов)****Космический зонд на ледяной комете**

На длинной ледяной комете расположено $n \geq 2$ научно-исследовательских станций вдоль числовой оси с целочисленными координатами $b_1, b_2, \dots, b_n$, упорядоченными по возрастанию, где $b_1 = 0$. Космический зонд начинает свою миссию на станции b_1 и должен достичь финальной станции b_n , используя адаптивную систему навигации.

Принцип работы навигационной системы

Зонд использует двухфазную систему перемещения. Сначала происходит **калибровка**: зонд определяет ближайшую станцию впереди (обозначим её b_j), вычисляет эталонное расстояние D как разность между координатой этой станции и текущей позицией зонда, затем телепортируется на станцию b_j , фиксируя найденное значение D как текущий масштаб перемещения.

После калибровки начинается **поиск целевого расстояния**: зонд последовательно сканирует позиции впереди себя (текущая позиция + 1, текущая позиция + 2, текущая позиция + 3 и так далее). Когда зонд обнаруживает станцию на некоторой позиции X , возможны три сценария. Если X равно b_n , то зонд перемещается на нее и успешно завершает миссию. Если на позиции $(X - D)$ также существует станция, то зонд перемещается на станцию X и выполняет новую калибровку в этой точке. В противном случае зонд продолжает сканирование позиций дальше.

Формат входных данных

В первой строке задано целое число n — количество станций, различных целочисленных точек ($2 \leq n \leq 300000$). Во второй строке через пробел записаны n целых чисел $b_1, b_2, \dots, b_n$ — координаты станций в порядке возрастания. Самая левая из них имеет значение 0, самая правая не превосходит 10^9 .

Формат выходных данных

Требуется вывести одно целое число D — итоговое эталонное расстояние, при котором зонд достигает финальной станции b_n .

Примеры

стандартный ввод	стандартный вывод
7 0 3 4 5 6 8 13	2
9 0 3 4 5 6 8 9 10 13	3
12 0 3 4 5 7 8 11 12 14 17 18 24	2

Пояснения к примерам



Иллюстрация к первому примеру.

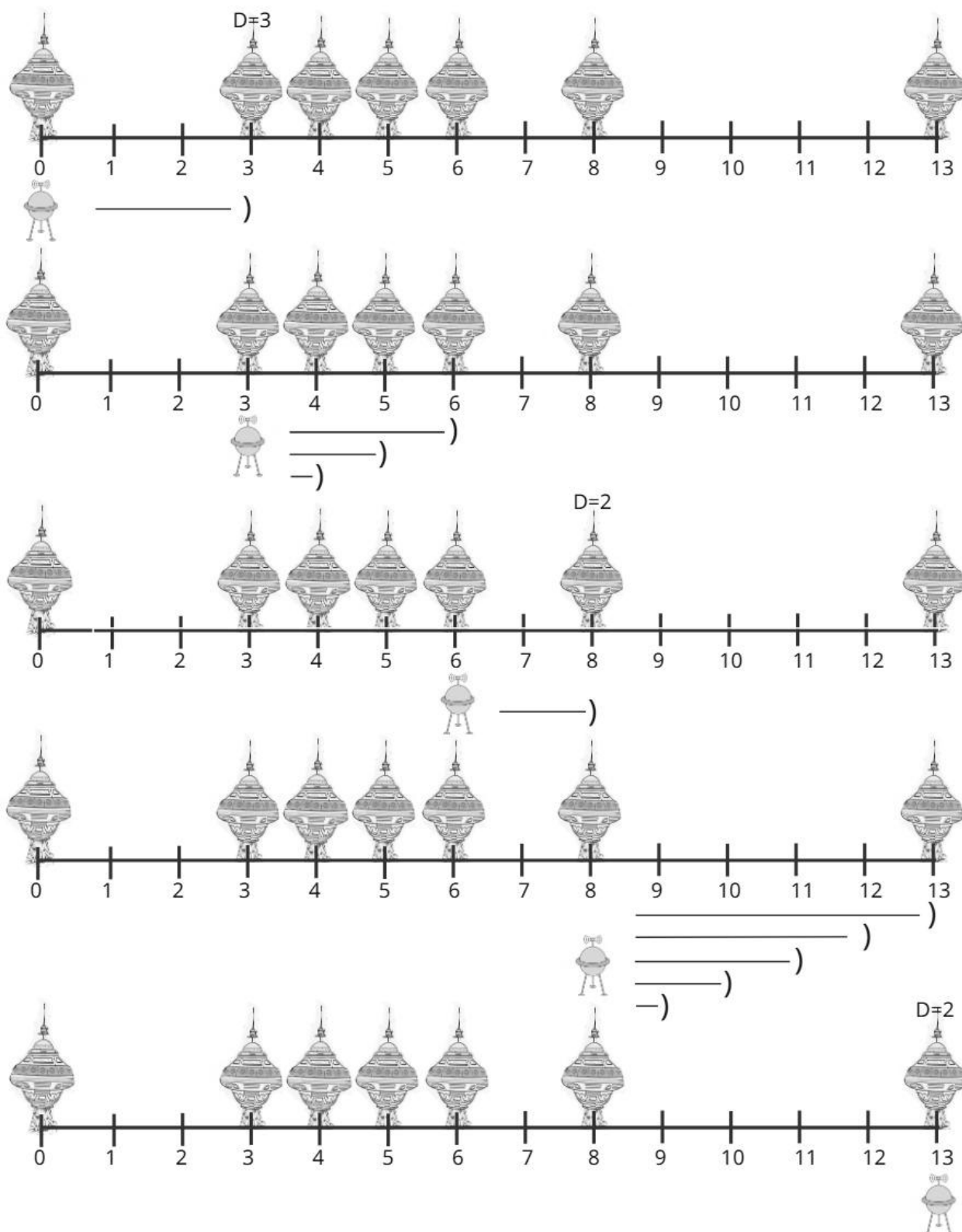
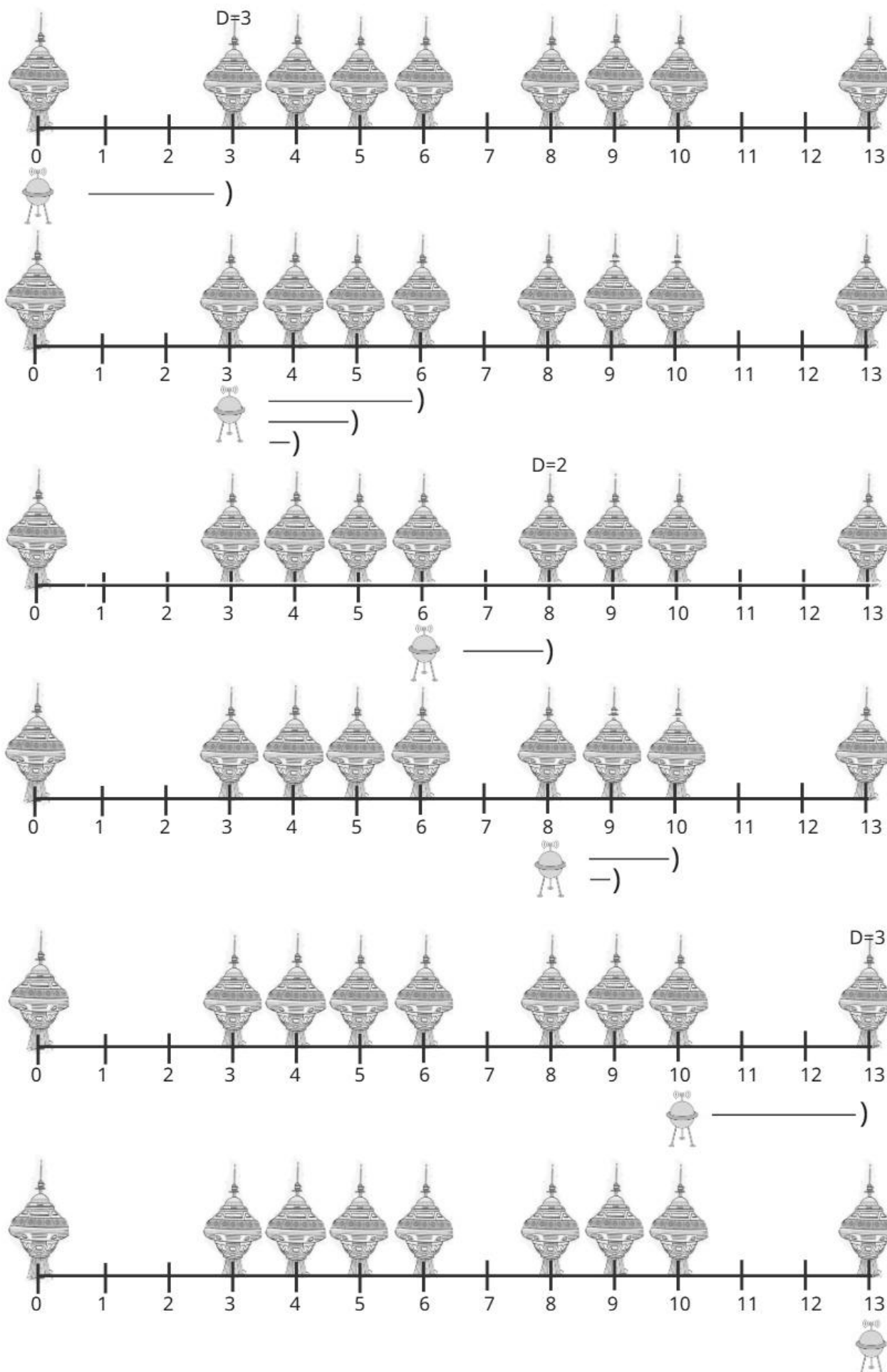


Иллюстрация ко второму примеру.





Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, обязательно **добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных, поясните решение. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
12 0 2 3 4 7 9 10 11 15 17 18 24	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

Смотрите решение в первом варианте.



Задача 6 (30 баллов)

Источник вечной энергии

Известный изобретатель Иван Иванович Иванов разработал принципиально новую установку, позволяющую генерировать сколько угодно безопасной энергии. Он поместил в точки прямой **0** и **D** два направленных друг на друга источника специально заряженных частиц. В момент времени **0** источник с координатой **0** генерирует **n** частиц, которые движутся по прямой в сторону второго генератора. Каждая из них имеет свою скорость v_i , скорости всех частиц попарно различны. Как только какая-то частица достигает источника с координатой **D**, она им поглощается и этот источник выпускает ровно такую же по скорости частицу в противоположном направлении. Эта частица движется обратно в сторону начала координат и снова поглощается источником с координатой **0**, после чего процесс испускания частицы в сторону второго источника снова повторяется.

Известно, что как только две частицы попадают в одну и ту же точку прямой, они генерируют одну единицу энергии. Если в одной точке оказываются несколько частиц, каждая пара среди встретившихся генерирует единицу энергии, в частности, в момент времени **0**, когда **n** частиц стартуют из точки **0**, они уже создают $\frac{n(n-1)}{2}$ единиц энергии. Если две частицы одновременно достигают какого-то из источников, то они генерируют одну единицу перед поглощением, и одну единицу уже после возрождения их двойников, движущихся в обратном направлении. За счет множественности встреч, процесс работы источников имеет большой положительный баланс по выработке энергии. Энергия может вырабатываться сколь угодно долго.

Теперь, для написания научной статьи про опыт использования источника вечной энергии, Иван Иванович хочет точно знать момент, когда будет выработана **m**-я по счету единица энергии.

Формат входных данных

В первой строке ввода через пробел содержатся два целых числа **D** и **n** ($0 \leq D \leq 200$, $2 \leq n \leq 2000$). Во второй строке содержатся **n** целых чисел v_i через пробел – скорости частиц ($1 \leq v_i \leq 2000$). Скорости даны в порядке строгого возрастания, то есть $v_1 < v_2 < \dots < v_n$.

В третьей строке содержится одно целое число **m** – номер требуемой по порядку единицы энергии, для которой следует найти точный момент её генерации ($1 \leq m \leq 10^9$).

Формат выходных данных

Вывести в ответ одно вещественное число – время, когда будет сгенерирована **m**-я по порядку единица энергии. Ответ вывести с точностью не менее 2 знаков после десятичной точки.

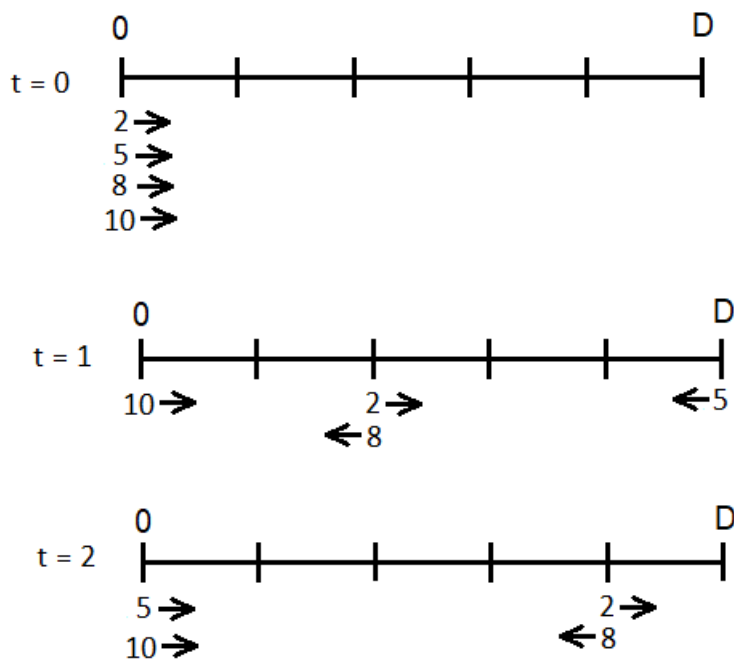


Примеры

стандартный ввод	стандартный вывод
5 4 2 5 8 10 6	0.00
5 4 2 5 8 10 7	0.56
5 4 2 5 8 10 11	1.00
5 4 2 5 8 10 17	1.67
5 4 2 5 8 10 19	1.67
5 4 2 5 8 10 21	2.00

Пояснения к примерам

На следующих рисунках приведено расположение частиц из примеров в моменты времени 0, 1 и 2. Частицы обозначены своими скоростями. Стрелками обозначены направления их движения в соответствующий момент. Если в этот момент частица была регенерирована, она имеет новое направление движения.



Решите указанную задачу, используя один из языков программирования, и укажите, какой именно язык был выбран. Предложите



эффективное решение, учитывая минимизацию времени выполнения программы и объема памяти, необходимого для её работы. Постарайтесь использовать алгоритмы и структуры данных, которые обеспечивают оптимальную производительность.

Реализуйте решение в виде программного кода, **обязательно добавив подробные комментарии, которые поясняют ключевые шаги алгоритма и его идею** – это важно для понимания проверяющему. Допускается использование только стандартных библиотек, встроенных в язык.

Приведите результат выполнения программы для заданных исходных данных. Оцените сложность предложенного алгоритма.

Исходные данные для выполнения задания

стандартный ввод	стандартный вывод
5 4 2 5 8 10 13	?

Ваше решение должно быть **аккуратно оформлено, читабельно** и соответствовать стандартам выбранного языка программирования.

Решение

Смотрите решение в первом варианте.

